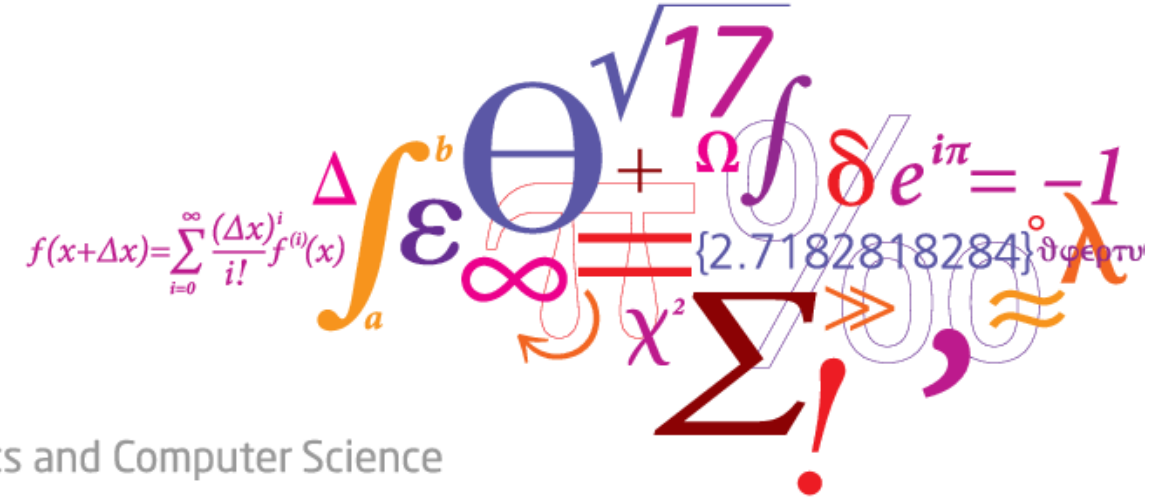
Decision Making under Uncertainty (02435)

Section for Dynamical Systems, DTU Compute.

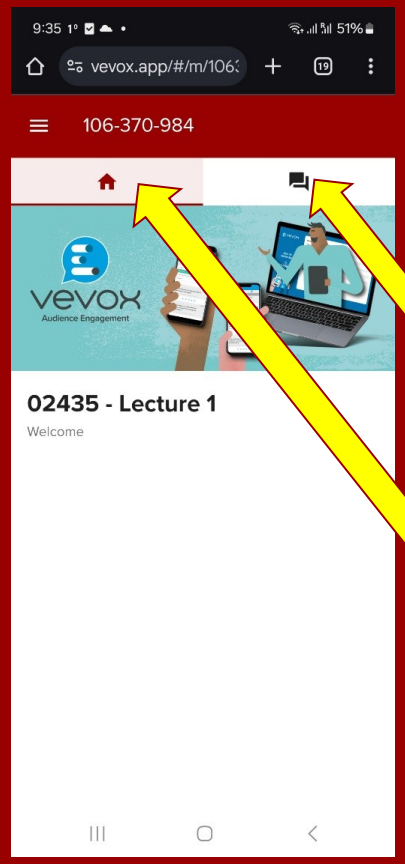


DTU Compute
Department of Applied Mathematics and Computer Science

Lecture 08: Approximate Dynamic Programming, visually explained

Scan me:

vevox.app
184-905-598



Anonymous Questions (during or after the lecture)

Quizzes

The Lecture will be recorded

Course Plan

Process	Concepts	Lectures	Assignment
	Mathematical Optimization	Week 1	Task 1
Defining the Problem	Markov Decision Process	Week 2	Task 2 & 6
Designing a Solution	Deterministic Lookahead (Model Predictive Control)	Week 3	Task 3
	Multi-stage Stochastic Programming	Weeks 4-5	Task 3
	Approximate Dynamic Programming	Weeks 6-8	Task 4
	Other Methods	Weeks 9-10	Task 5
Evaluating a Solution	Evaluation in Simulation Environment	Week 11	Task 6
Coordinating Multiple Agents	Multi-Agent Systems	Weeks 12-13	Task 7

Agenda for today

1. ADP general formulation (recap and visual explanations)
2. Variants / extensions

The Bellman Equation

If you have the Value Function, making a decision is easy.

$$V(x_t) = \max_{u_t} \left\{ r(x_t, u_t) + \sum_{x_{t+1}} P(x_{t+1} | x_t, u_t) \cdot V(x_{t+1}) \right\}$$

Value Function Approximation / Approximate Dynamic Programming

1. Sample states $x^{(1)}, \ldots, x^{(N)}$ → 2. Compute / estimate value targets $y^{(i)} \approx V(x^{(i)})$ → 3. Train parametric model $\hat{V}(x;\theta)$ using regression

Step 1: Value Function Approximation

Current state x_t → Policy execution: $\max_{u_t} R(x_t, u_t) + \hat{V}(x_{t+1}(x_t, u_t))$ → Choose action u_t^*

Step 2: Policy Optimization

Value Function Approximation / Approximate Dynamic Programming

Our setting:

- MDP with state $\boldsymbol{x}_t = (\boldsymbol{y}_t, \boldsymbol{w}_t)$
- Known transition:

$$\boldsymbol{y}_{t+1} = f(\boldsymbol{x}_t, \boldsymbol{w}_{t+1}, \boldsymbol{u}_t),$$
$$\boldsymbol{w}_{t+1} \sim P(\cdot \mid \boldsymbol{w}_t)$$

We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$…
…and learn $\boldsymbol{\eta}_t$.

Step 1: Value Function Approximation

Given $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$ and $\boldsymbol{x}_t = (\boldsymbol{y}_t, \boldsymbol{w}_t)$:

1. We create K samples $\boldsymbol{w}_{k,t+1}$
2. We solve:

$$\max_{\boldsymbol{u}_t, \boldsymbol{y}_{k,t+1}} \left\{ r(\boldsymbol{x}_t, \boldsymbol{u}_t) + \sum p_k \, \hat{V}\left((\boldsymbol{y}_{k,t+1}, \boldsymbol{w}_{k,t+1}); \boldsymbol{\eta}_{t+1} \right) \right\}$$
$$\text{s.t. } \boldsymbol{y}_{k,t+1} = f\left(\boldsymbol{y}_t, \boldsymbol{w}_{k,t+1}, \boldsymbol{u}_t\right) \; \forall k$$

…to get $\boldsymbol{u}_t^*$

Step 2: Policy Optimization

Value Function Approximation / Approximate Dynamic Programming

Our setting:

- MDP with state $\boldsymbol{x}_t = (\boldsymbol{y}_t, \boldsymbol{w}_t)$
- Known transition:

$$\boldsymbol{y}_{t+1} = f(\boldsymbol{x}_t, \boldsymbol{w}_{t+1}, \boldsymbol{u}_t),$$
$$\boldsymbol{w}_{t+1} \sim P(\cdot \mid \boldsymbol{w}_t)$$

The entire lecture will be focusing on Step 1

We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$…
…and learn $\boldsymbol{\eta}_t$.

Step 1: Value Function Approximation

Given $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$ and $\boldsymbol{x}_t = (\boldsymbol{y}_t, \boldsymbol{w}_t)$:

1. We create K samples $\boldsymbol{w}_{k,t+1}$
2. We solve:

$$\max_{\boldsymbol{u}_t, \boldsymbol{y}_{k,t+1}} \left\{ r(\boldsymbol{x}_t, \boldsymbol{u}_t) + \sum p_k \, \hat{V}\left((\boldsymbol{y}_{k,t+1}, \boldsymbol{w}_{k,t+1}); \boldsymbol{\eta}_{t+1} \right) \right\}$$
$$\text{s.t. } \boldsymbol{y}_{k,t+1} = f\left(\boldsymbol{y}_t, \boldsymbol{w}_{k,t+1}, \boldsymbol{u}_t\right) \; \forall k$$

…to get $\boldsymbol{u}_t^*$

Step 2: Policy Optimization

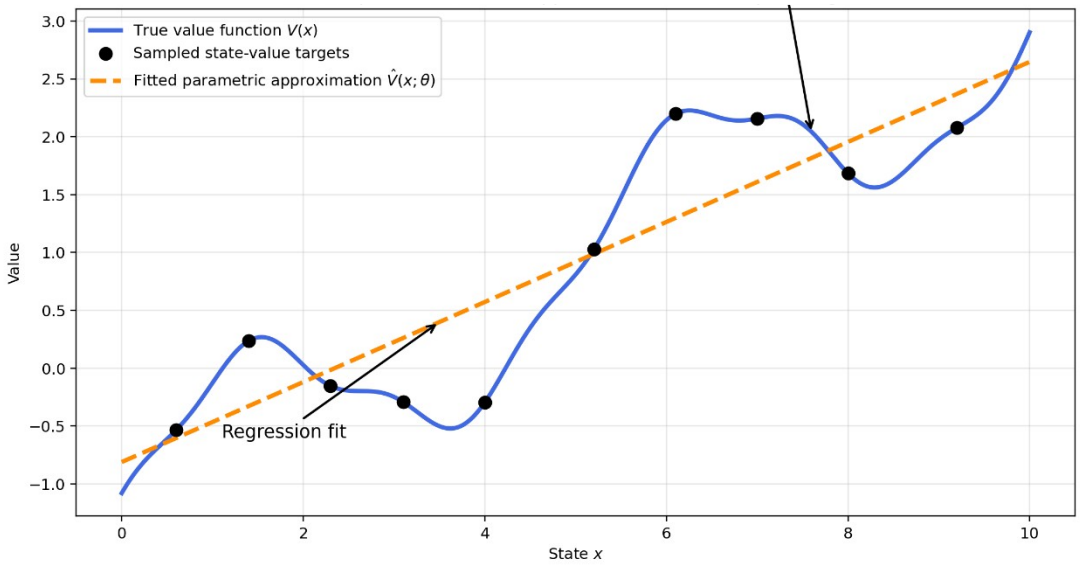
Step 1: Approximating the Value Function

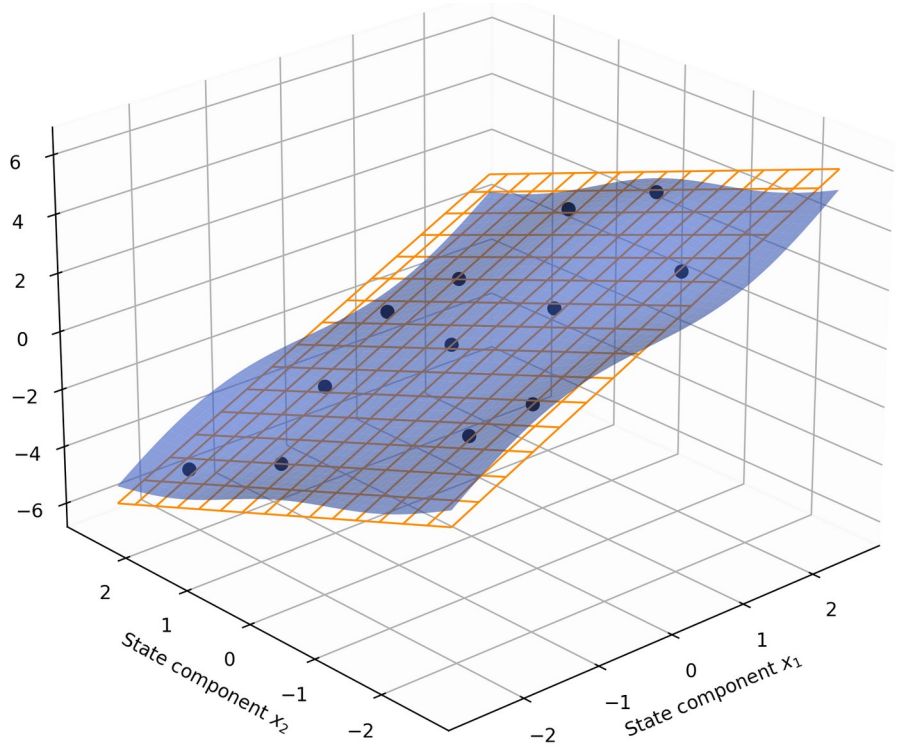
We parameterize the Value Function: $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$

1. We *would like to* compute samples $V(\boldsymbol{x}_{n,t})$ of the optimal Value Function.

2. And use them as Targets, to train $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$:

$$\min_{\boldsymbol{\eta}_t} \left\{ \sum_n \left(\hat{V}(\boldsymbol{x}_{n,t}; \boldsymbol{\eta}_t) - V(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$





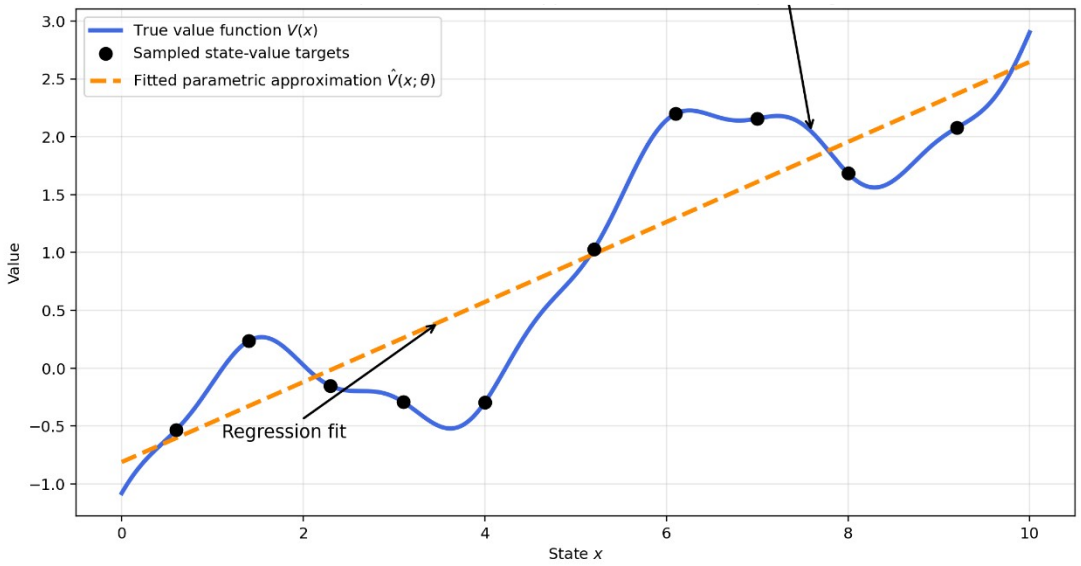
Step 1: Approximating the Value Function

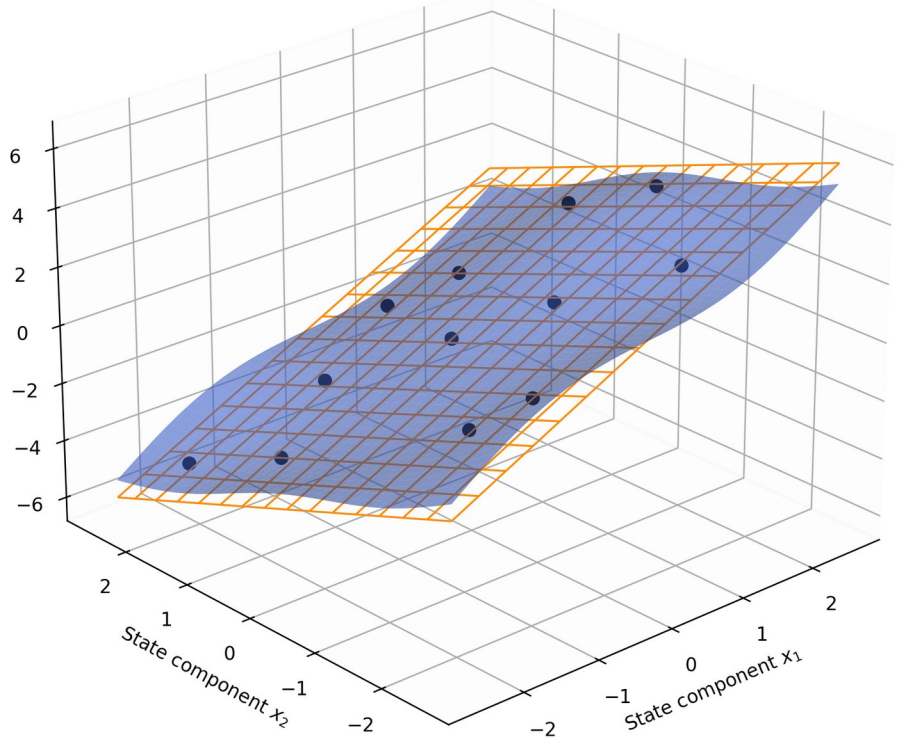
We parameterize the Value Function: $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$

1. We *would like to* compute samples $V(\boldsymbol{x}_{n,t})$ of the optimal Value Function.

2. And use them as Targets, to train $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$:

$$\min_{\boldsymbol{\eta}_t} \left\{ \sum_n \left(\hat{V}(\boldsymbol{x}_{n,t}; \boldsymbol{\eta}_t) - V(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

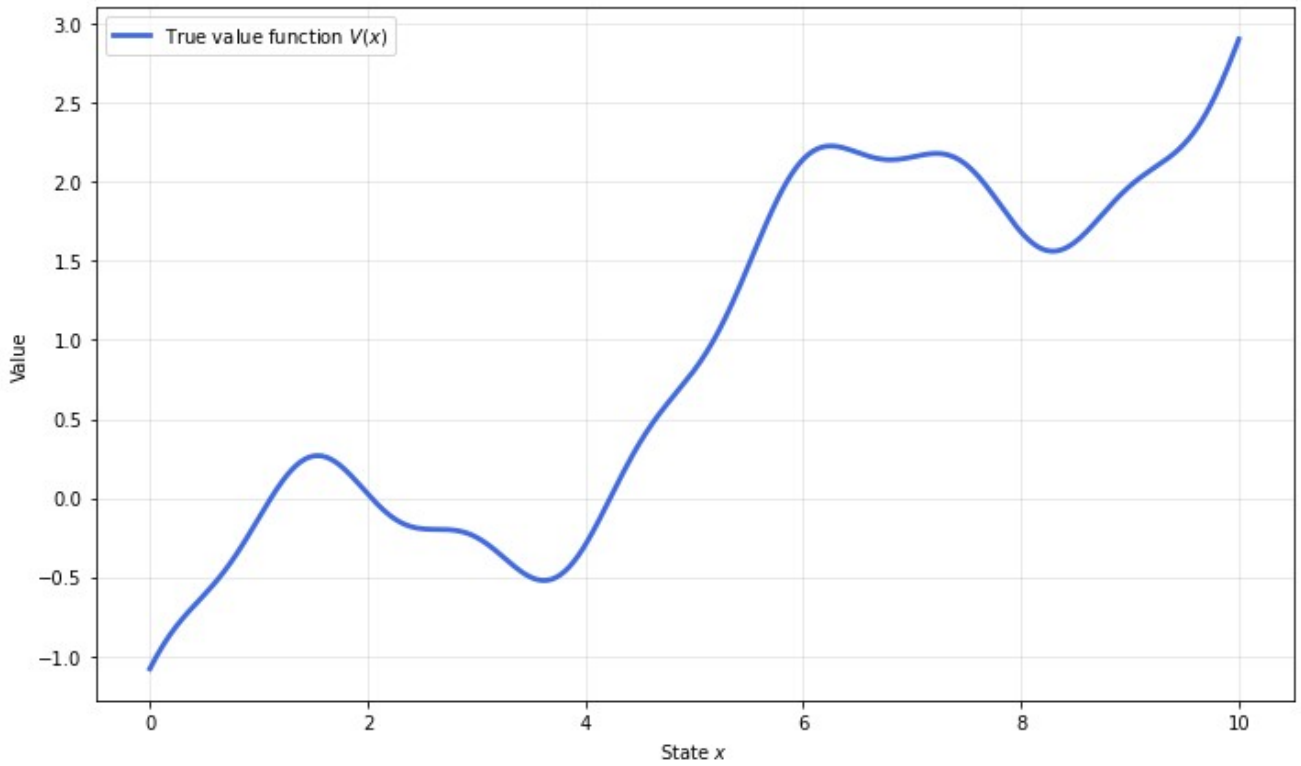




But we cannot evaluate $V(\boldsymbol{x}_{n,t})$, because we don't have $V(\cdot)$.

Duh...

Approximating the Value Function with Fitted Value Iteration



But we cannot evaluate $V(\boldsymbol{x}_{n,t})$, because we don't have $V(\cdot)$.

Duh…

Approximating the Value Function with Fitted Value Iteration

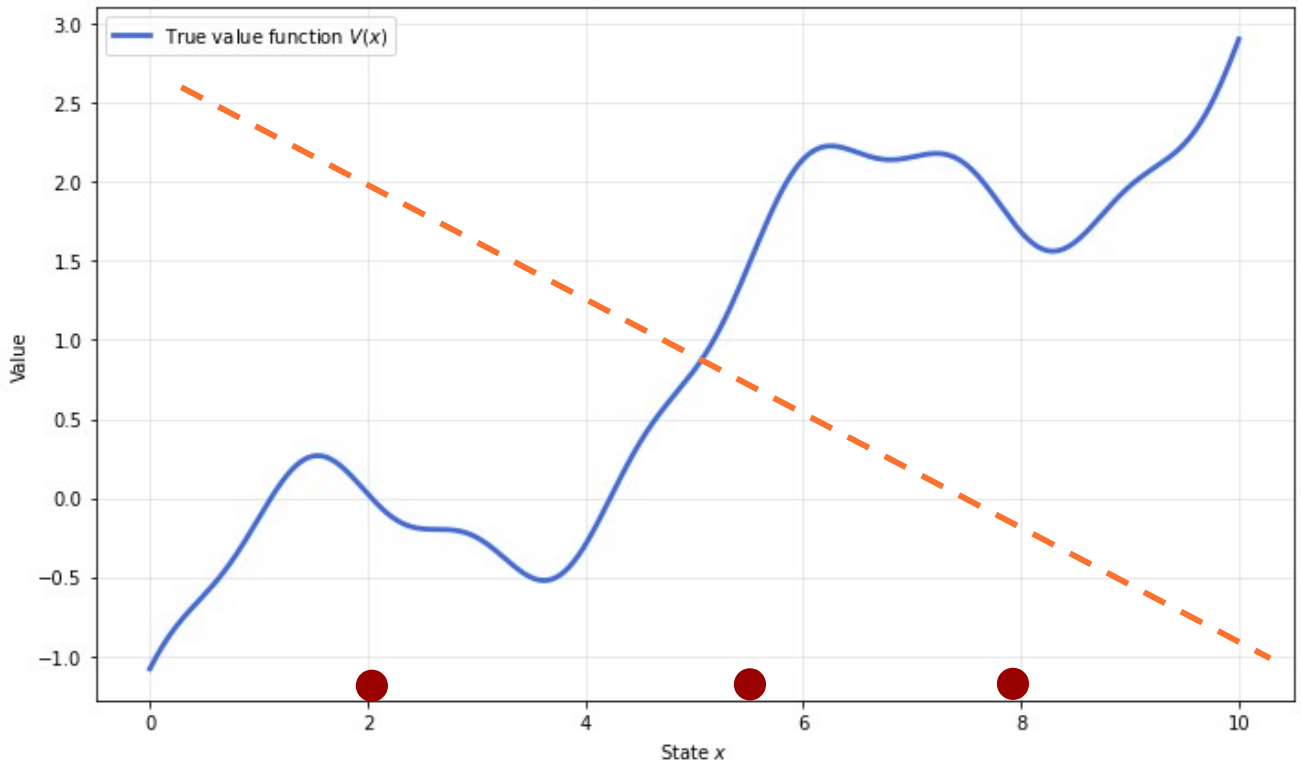
*Visual example for one timeslot

Input: MDP

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$



*this visualization, implicitly assumes that the true Value Function is the same at each stage, which in general is not true.

But we cannot evaluate $V(\boldsymbol{x}_{n,t})$, because we don't have $V(\cdot)$.

Duh…

Approximating the Value Function with Fitted Value Iteration

*Visual example for one timeslot

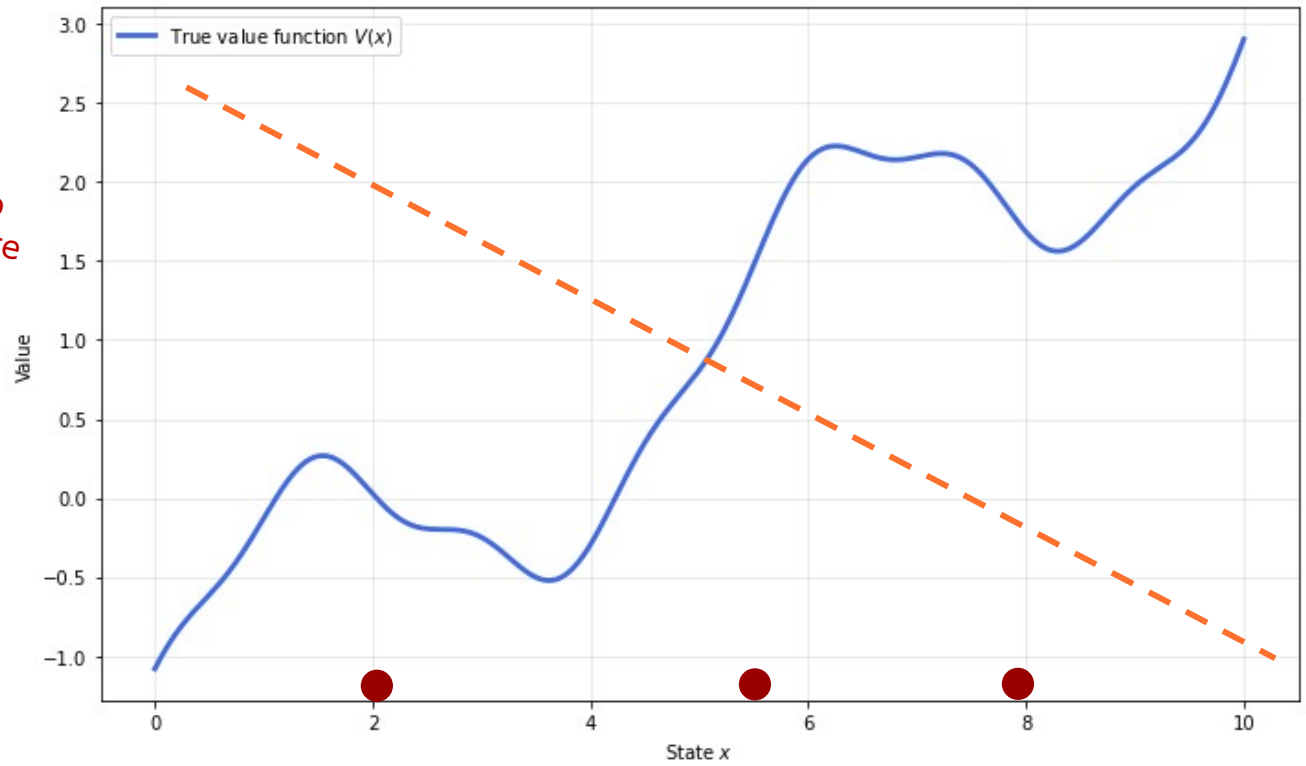
Input: MDP

For each $t \in T$:

Initialize η_t

Sample N states $\{x_{n,t}\}_{n,t} = \{(y_{n,t}, w_{n,t})\}_{n,t}$

To estimate the value of a state, I need to reason about the reward and the next state



But we cannot evaluate $V(x_{n,t})$, because we don't have $V(\cdot)$.

Duh…

Approximating the Value Function with Fitted Value Iteration

*Visual example for one timeslot

Input: MDP

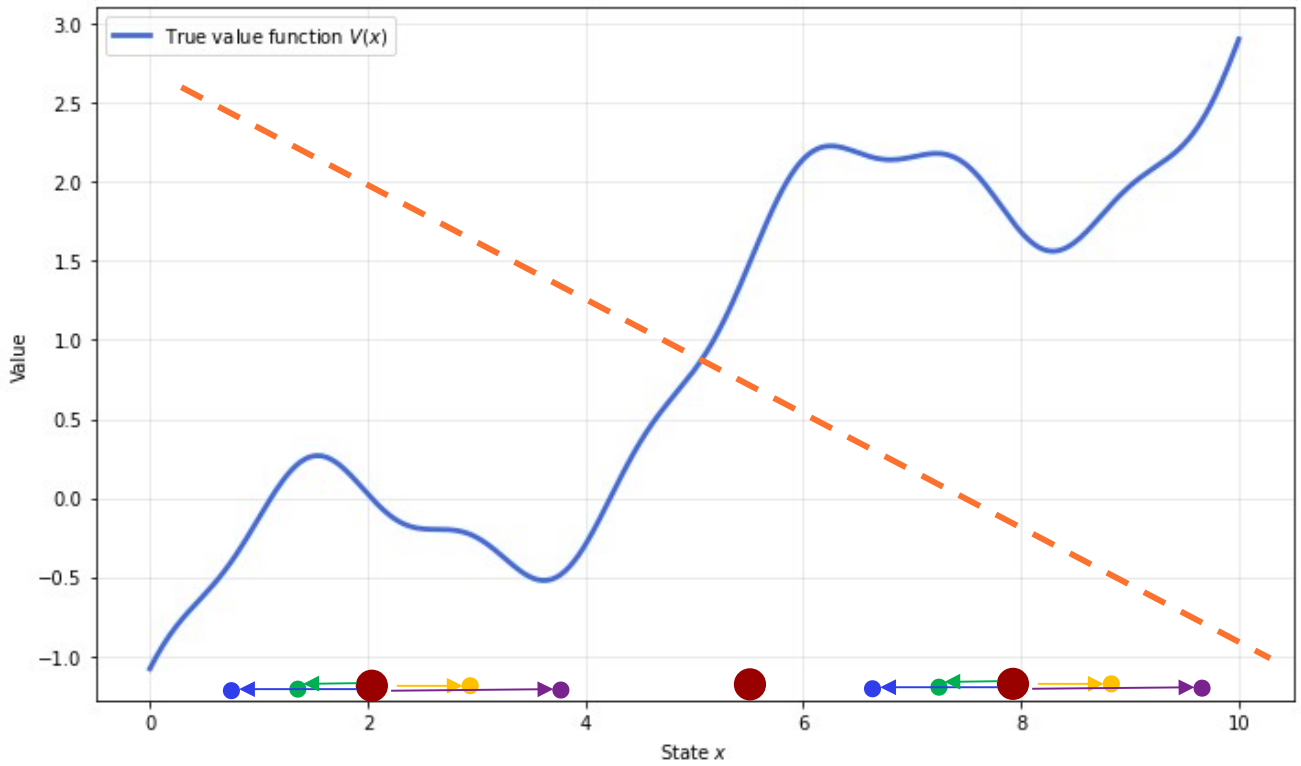
For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$



But we cannot evaluate $V(\boldsymbol{x}_{n,t})$, because we don't have $V(\cdot)$.

Duh…

Approximating the Value Function with Fitted Value Iteration

*Visual example for one timeslot

Input: MDP

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Repeat for I iterations:

For each $t \in T$:

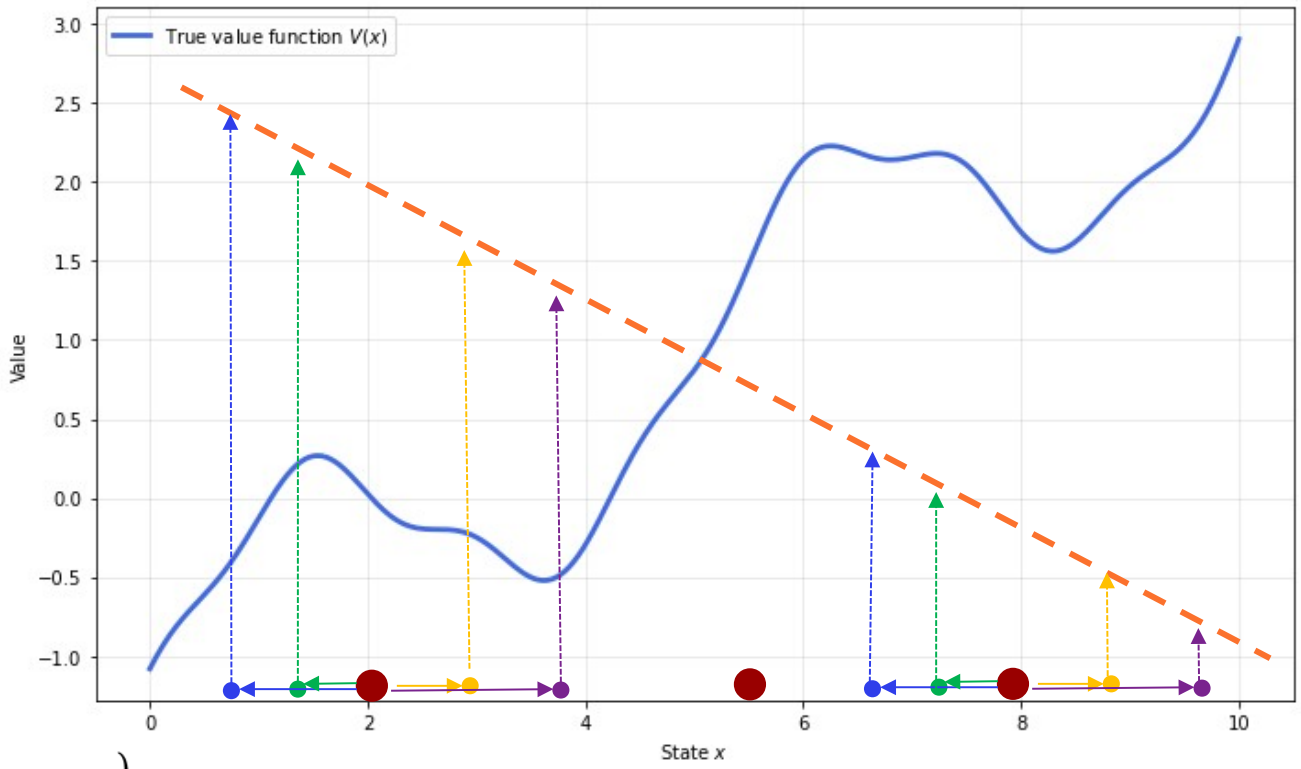
For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \, \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$$

For each sampled state, the next state is partly sampled ($\boldsymbol{w}_{n,k,t+1}$) and partly selected ($\boldsymbol{y}_{n,k,t+1}$)



But we cannot evaluate $V(\boldsymbol{x}_{n,t})$, because we don't have $V(\cdot)$.

Duh…

Approximating the Value Function with Fitted Value Iteration

*Visual example for one timeslot

Input: MDP

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Repeat for I iterations:

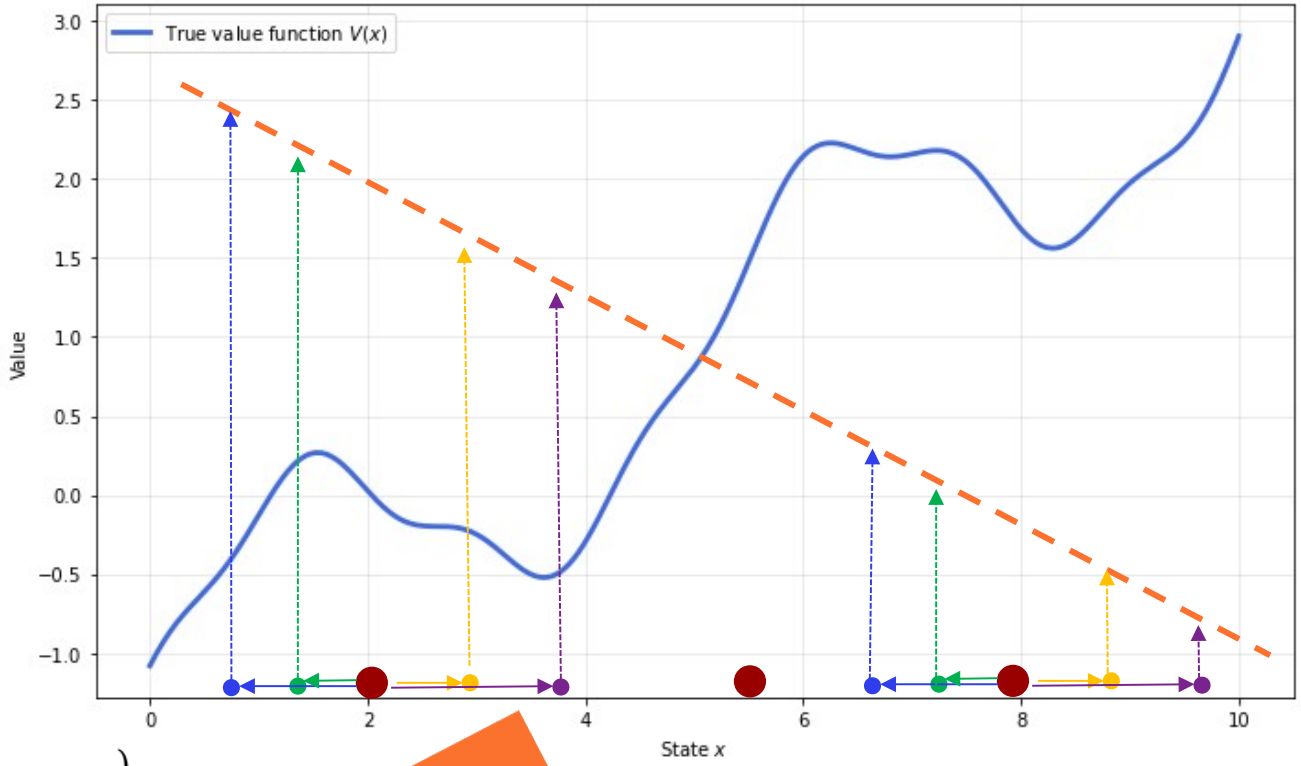
For each $t \in T$:

For each n:

Compute target $V^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$



And each $x_{n,k,t+1}$ is evaluated using the approximate Value Function itself!

But we cannot evaluate $V(\boldsymbol{x}_{n,t})$, because we don't have $V(\cdot)$.

Duh…

Approximating the Value Function with Fitted Value Iteration

*Visual example for one timeslot

Input: MDP

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Repeat for I iterations:

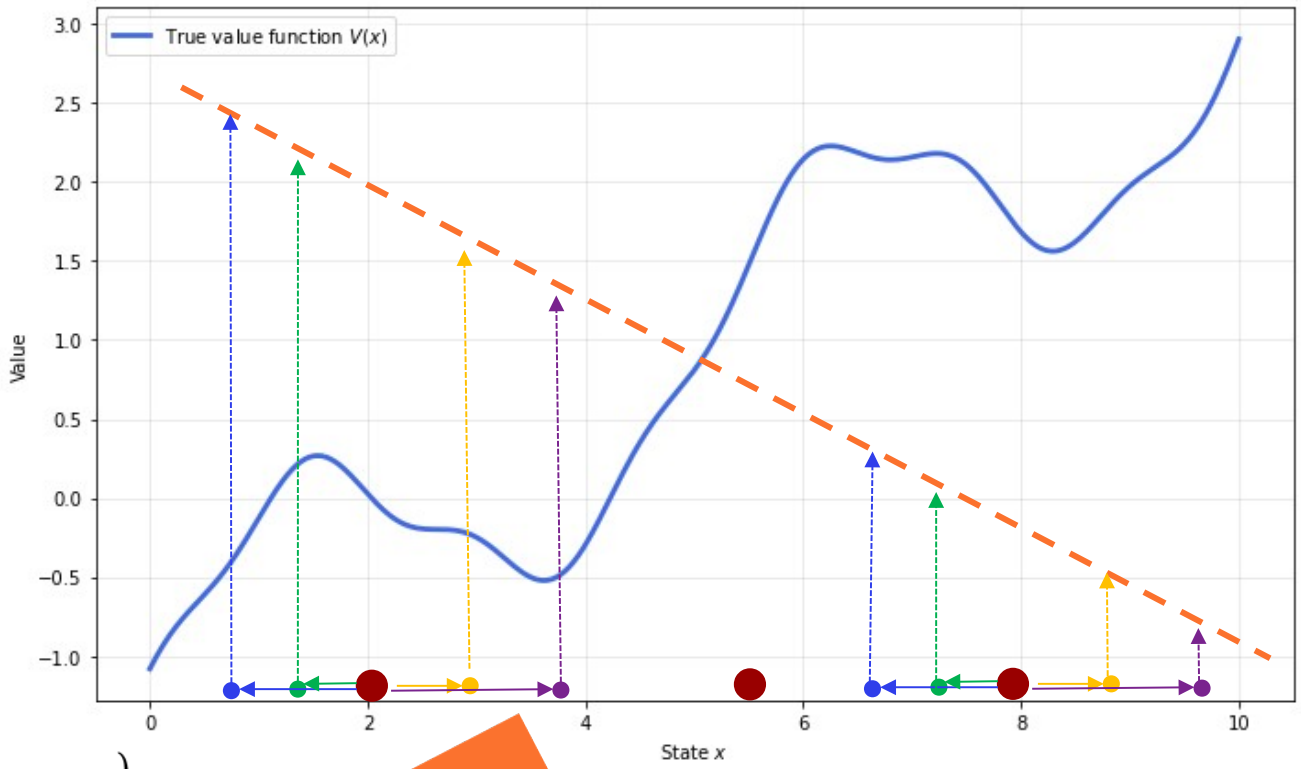
For each $t \in T$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \, \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$$



And each $x_{n,k,t+1}$ is evaluated using the approximate Value Function itself!

which is a bad estimator, but the targets also get informed of the true reward

Approximating the Value Function with Fitted Value Iteration

*Visual example for one timeslot

Input: MDP

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Repeat for I iterations:

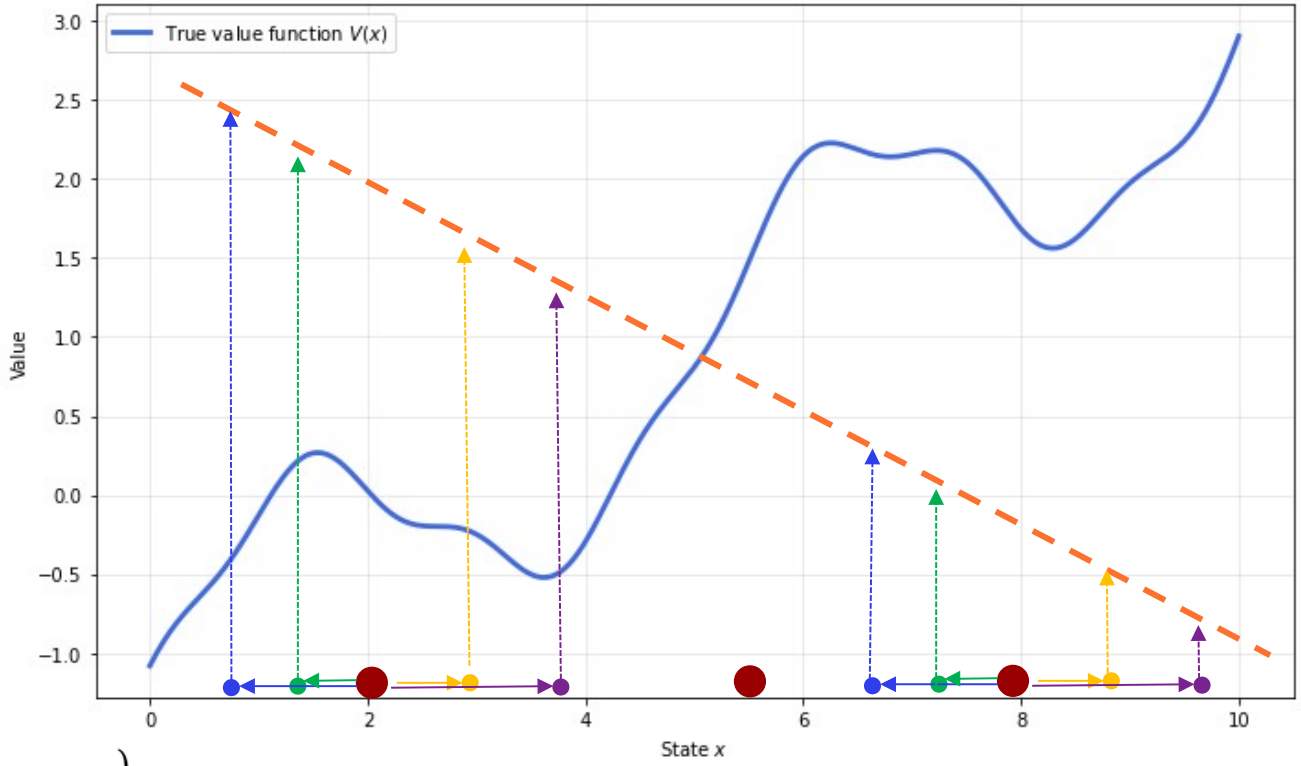
For each $t \in T$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \, \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$$



Where approximately will each target be?
Mark them on the figure

A target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ is the presumed value for state $\boldsymbol{x}_{n,t}$

Approximating the Value Function with Fitted Value Iteration

*Visual example for one timeslot

Input: MDP

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Repeat for I iterations:

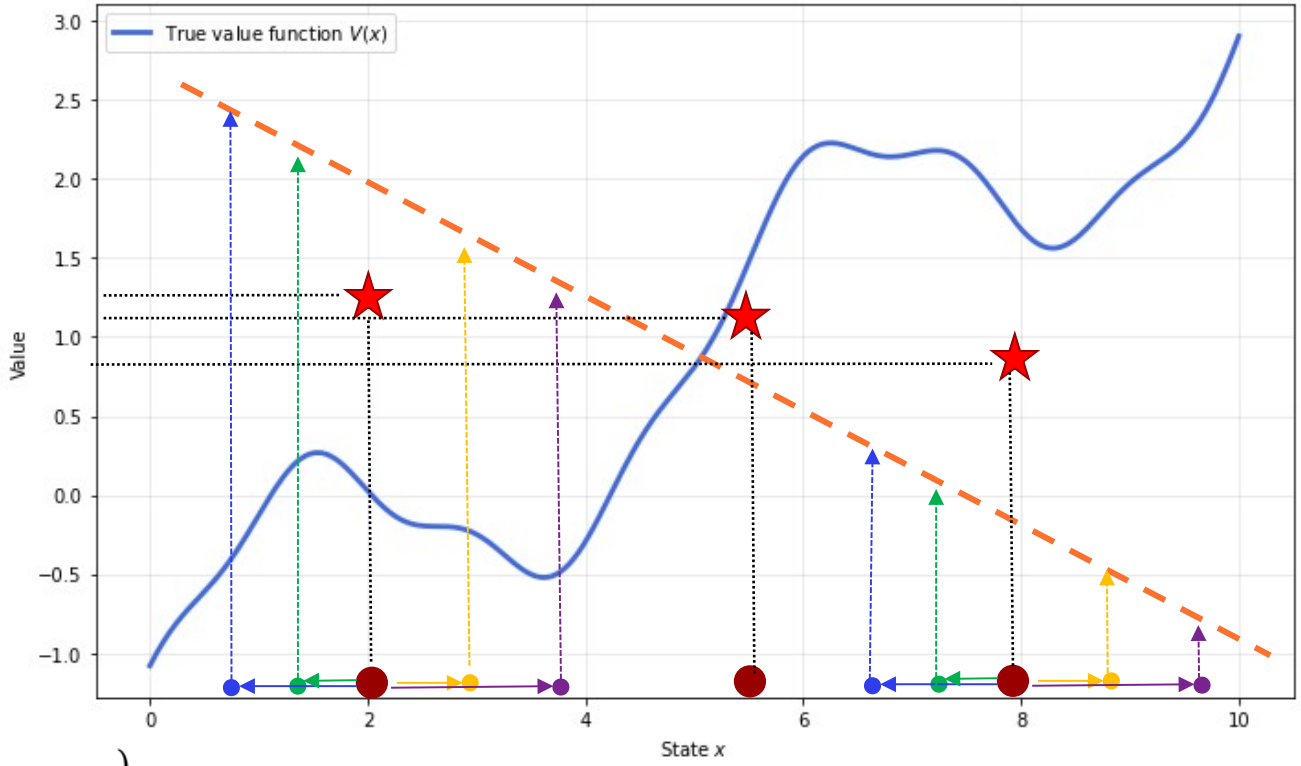
For each $t \in T$:

For each n:

Compute target $V^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$$



which is a bad estimator, but the targets also get informed of the true reward

Approximating the Value Function with Fitted Value Iteration

*Visual example for one timeslot

Input: MDP

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Repeat for I iterations:

For each $t \in T$:

For each n:

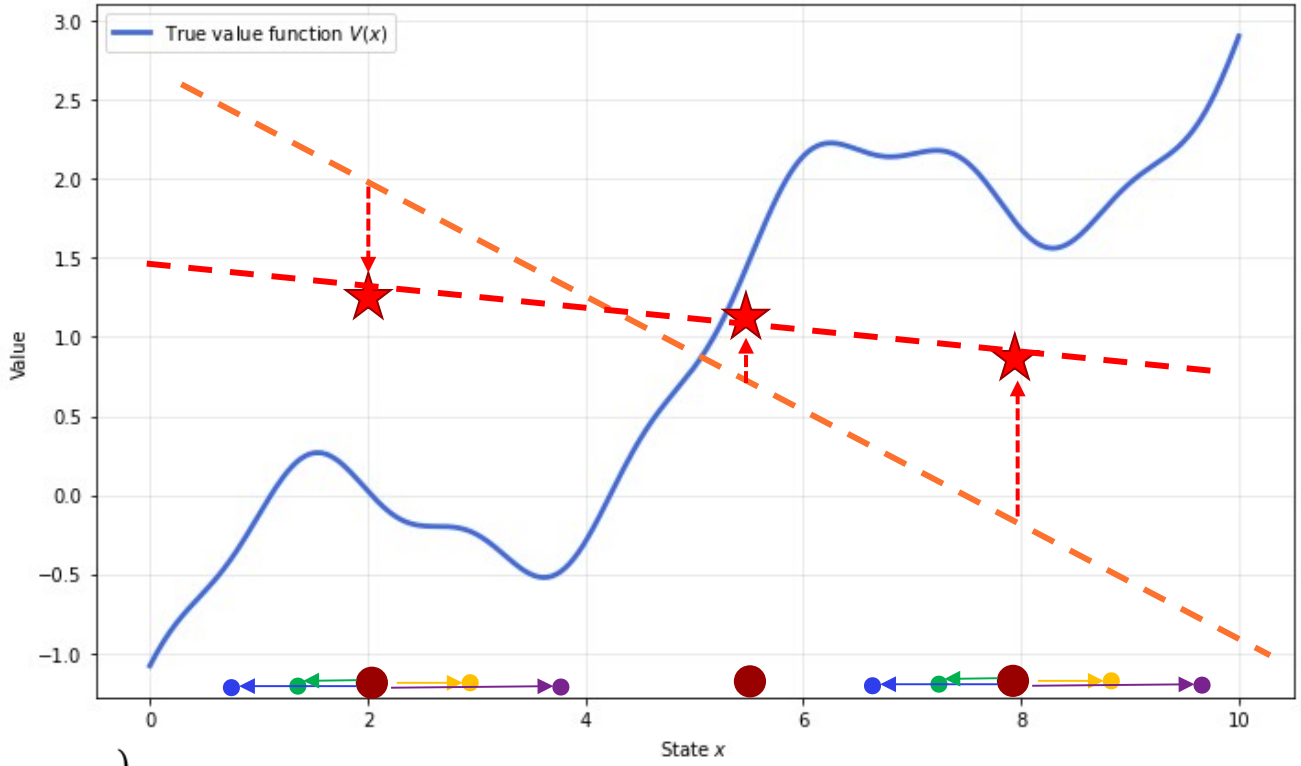
Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \, \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$



Approximating the Value Function with Fitted Value Iteration

*Visual example for one timeslot

Input: MDP

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Repeat for I iterations:

For each $t \in T$:

For each n:

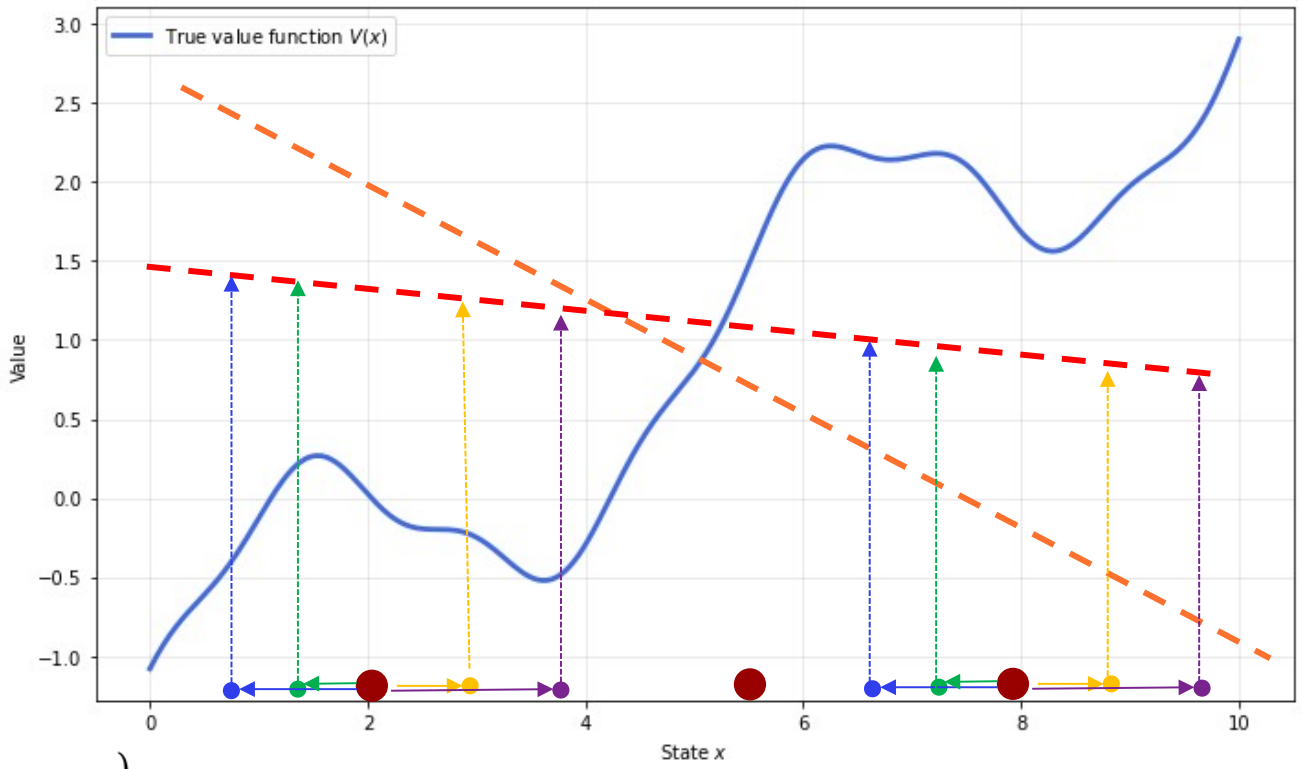
Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \, \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$$

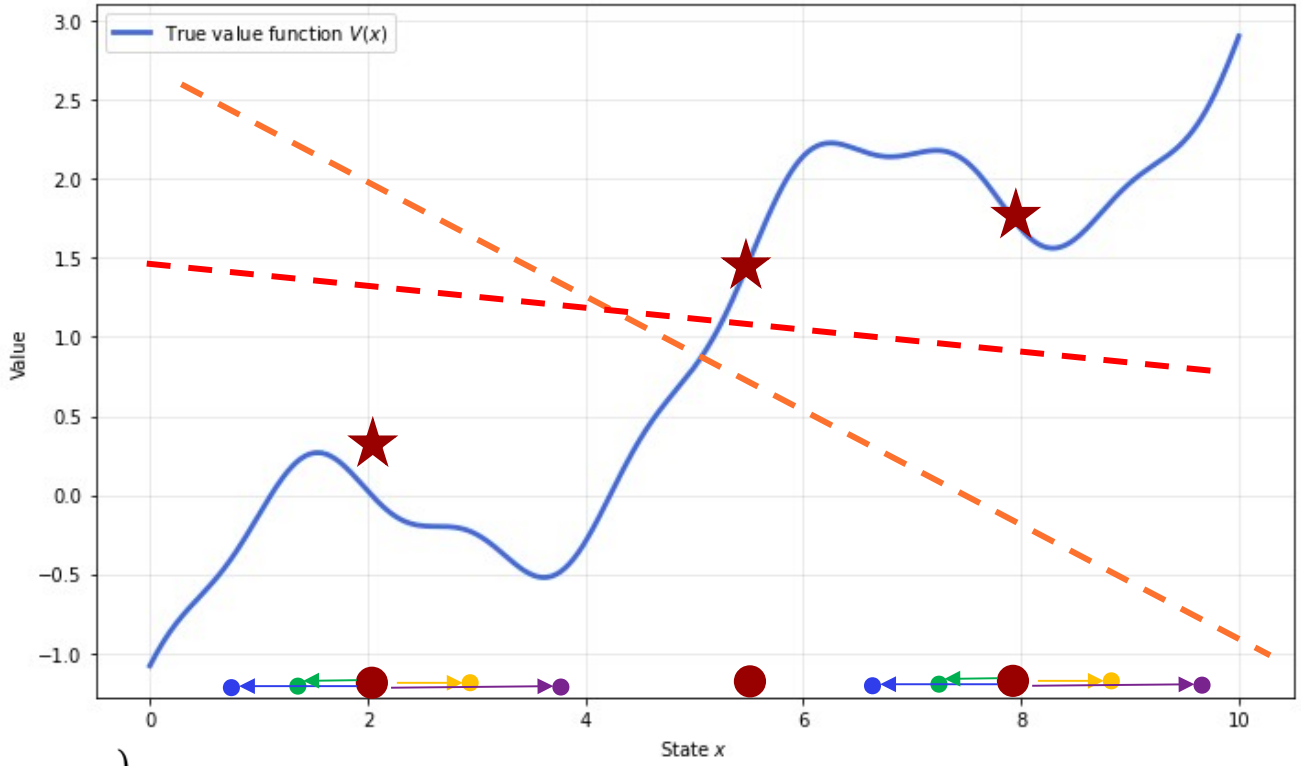
Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$



Approximating the Value Function with Fitted Value Iteration

*Visual example for one timeslot



Input: MDP

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Repeat for I iterations:

For each $t \in T$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

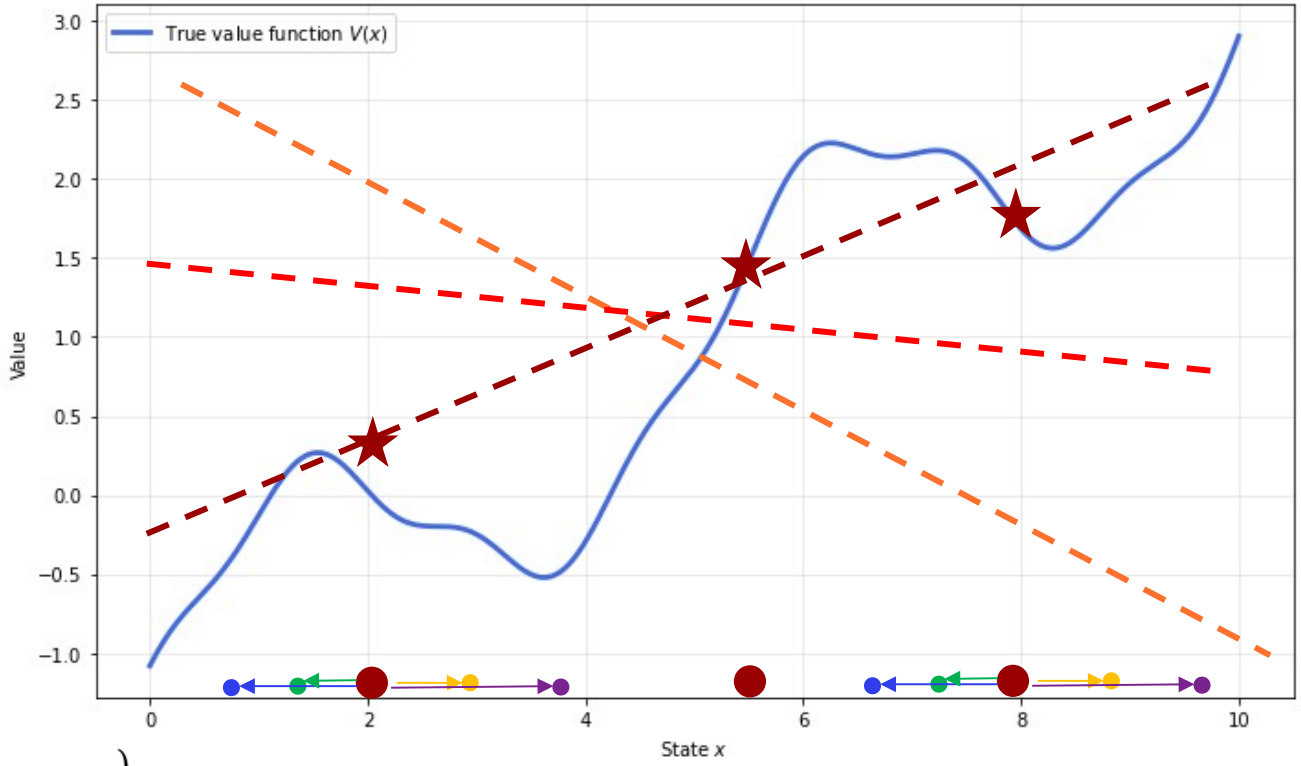
$$\text{subject to: } \boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

Approximating the Value Function with Fitted Value Iteration

*Visual example for one timeslot



Input: MDP

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Repeat for I iterations:

For each $t \in T$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

Approximating the Value Function with Fitted Value Iteration

*Visual example for one timeslot

Input: MDP

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Repeat for I iterations:

For each $t \in T$:

For each n:

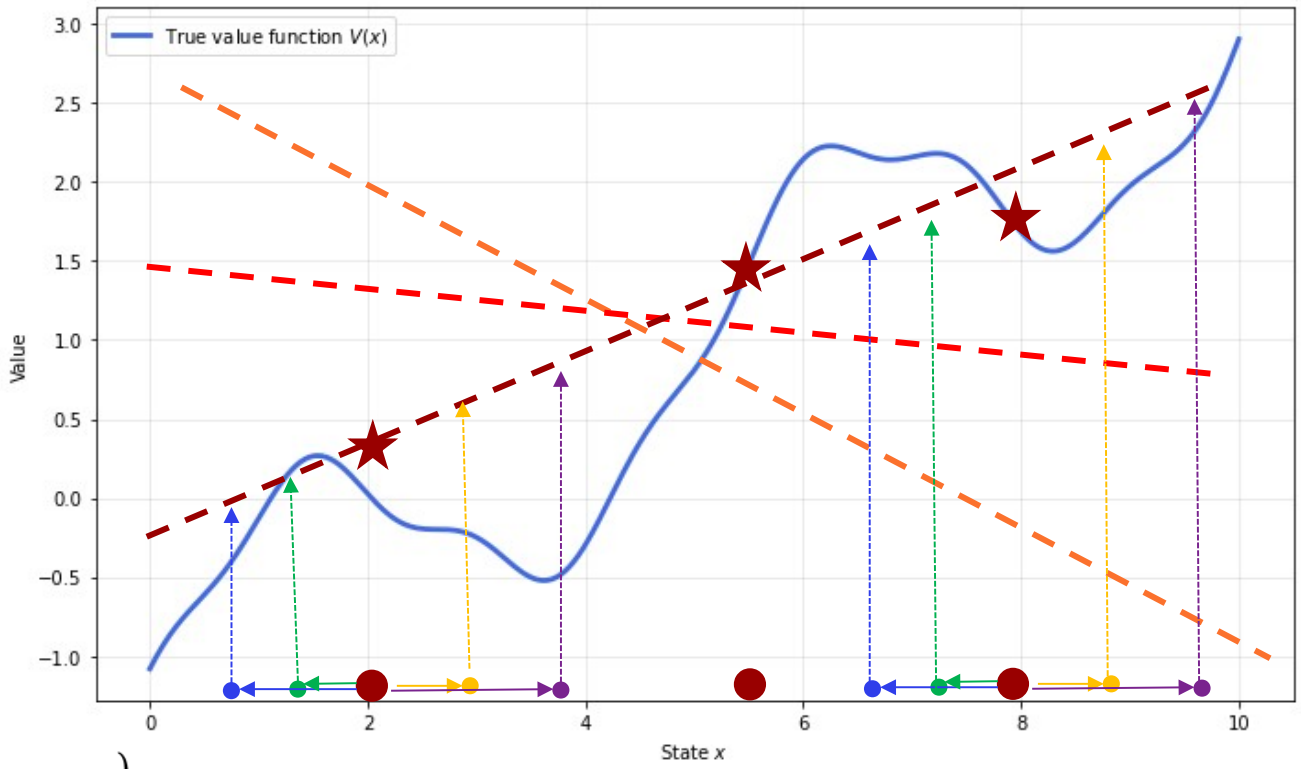
Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$



Approximating the Value Function with Fitted Value Iteration

*Visual example for one timeslot

Input: MDP,

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Repeat for I iterations:

For each $t \in T$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

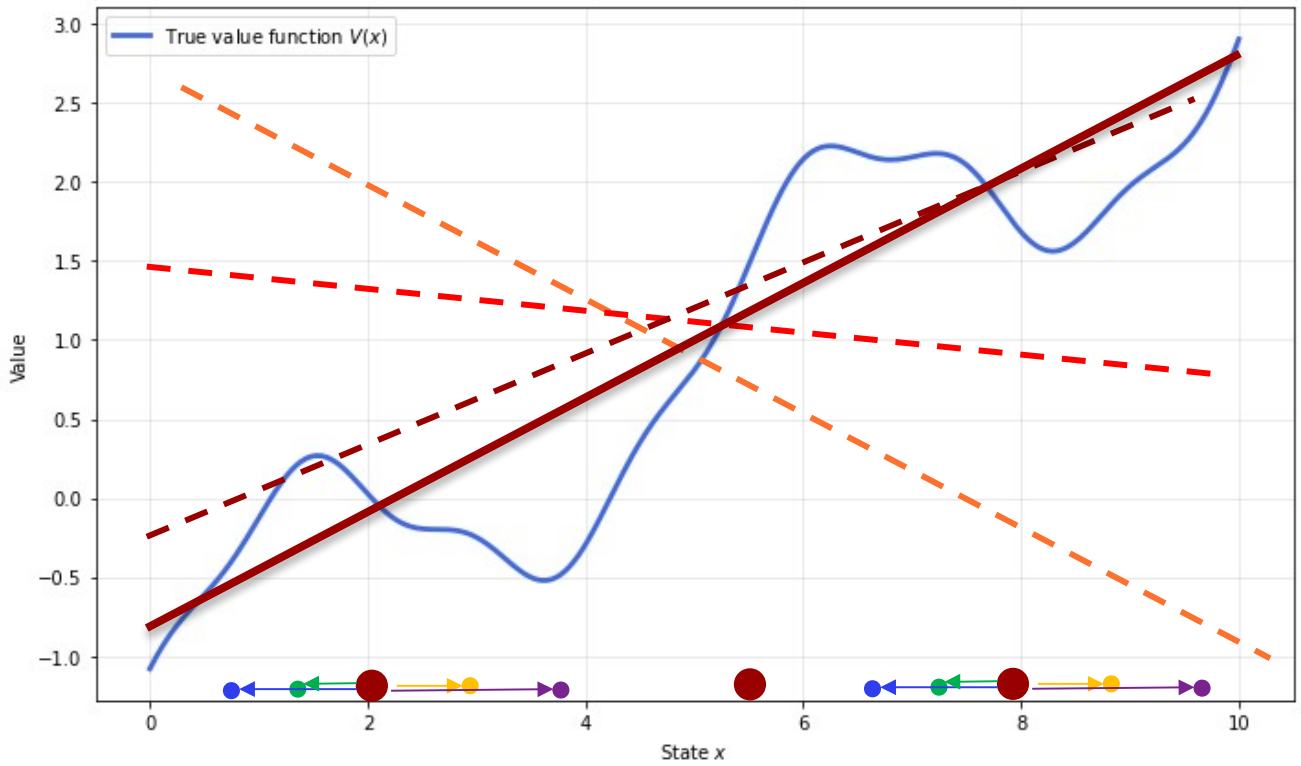
$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

Output: $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$



Approximating the Value Function with Fitted Value Iteration

*Visual example for one timeslot

Input: MDP,

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Repeat for I iterations:

For each $t \in T$:

For each n:

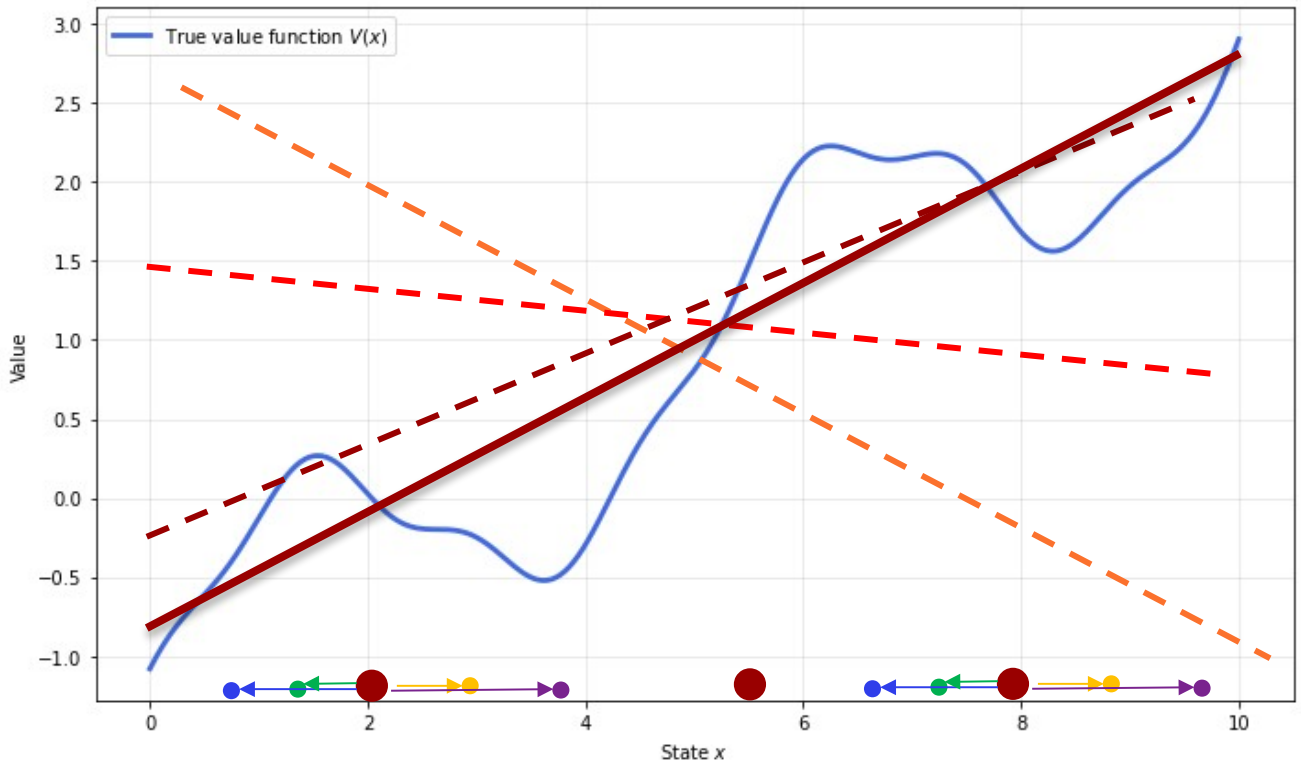
Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$



Would this converge?

Output: $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$

Approximating the Value Function with Fitted Value Iteration

Input: MDP,

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Repeat for I iterations:

For each $t \in T$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

Output: $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$

We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$…

…and learn $\boldsymbol{\eta}_t$.

Step 1: Value Function Approximation

Given $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$ and $\boldsymbol{x}_t = (\boldsymbol{y}_t, \boldsymbol{w}_t)$:

1. We create K samples $\boldsymbol{w}_{k,t+1}$
2. We solve:

$$\max_{\boldsymbol{u}_t, \boldsymbol{y}_{k,t+1}} \left\{ r(\boldsymbol{x}_t, \boldsymbol{u}_t) + \sum p_k \hat{V}\left((\boldsymbol{y}_{k,t+1}, \boldsymbol{w}_{k,t+1}); \boldsymbol{\eta}_{t+1} \right) \right\}$$

s.t. $\boldsymbol{y}_{k,t+1} = f(\boldsymbol{y}_t, \boldsymbol{w}_{k,t+1}, \boldsymbol{u}_t), \forall k$

…to get $\boldsymbol{u}_t^*$

Step 2: Policy Optimization

Fitted Value Iteration vs Approximate Backward Induction

Input: MDP,

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Repeat for I iterations: **vs** ~~**Repeat** for I iterations~~

For each $t \in T$: **vs** **For** each $t \in T, T-1, T-2, \ldots$

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$$

Update $\boldsymbol{\eta}_t$ as:

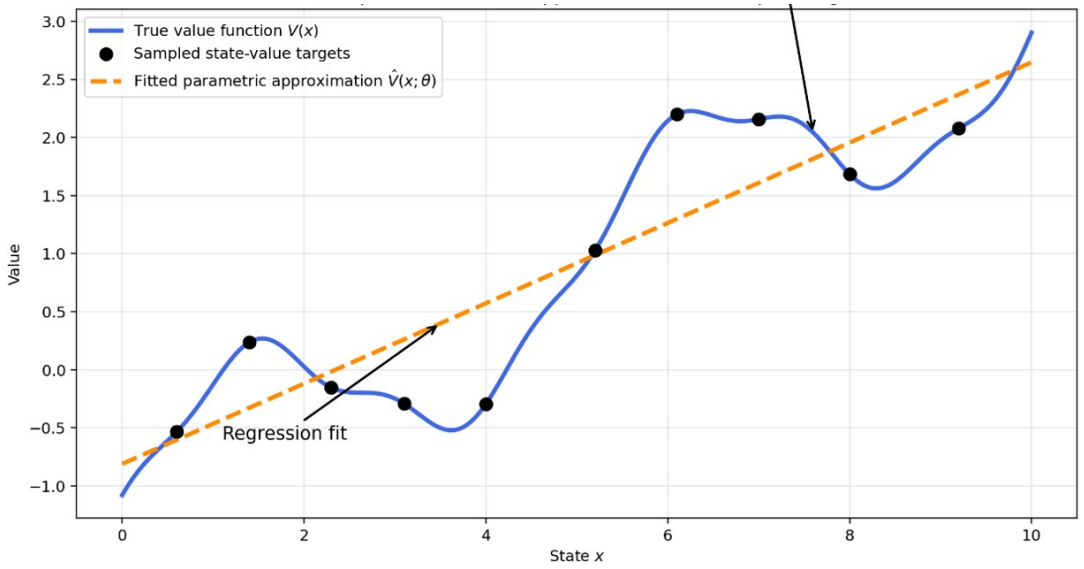
$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)\ldots$
…and learn $\boldsymbol{\eta}_t$.

Step 1:
Value Function Approximation

For the last stage, the target values can be computed exactly. So we get a very good fit with one strike – without iterating.



Fitted Value Iteration vs Approximate Backward Induction

Input: MDP,

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Repeat for I iterations: **vs** ~~**Repeat** for I iterations~~

For each $t \in T$: **vs** **For** each $t \in T, T-1, T-2, \ldots$

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

Backward Pass

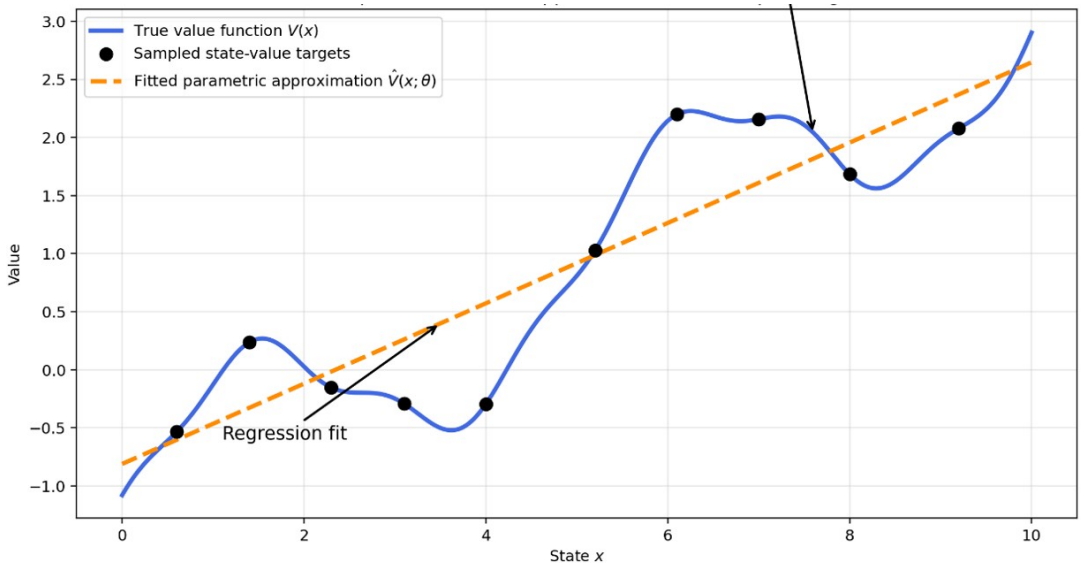
Which one is better?

We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)\ldots$
…and learn $\boldsymbol{\eta}_t$.

Step 1:
Value Function Approximation

For the last stage, the target values can be computed exactly. So we get a very good fit with one strike – without iterating.



Fitted Value Iteration vs Approximate Backward Induction

Input: MDP,

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Repeat for I iterations: **vs** ~~**Repeat** for I iterations~~

For each $t \in T$: **vs** **For** each $t \in T, T-1, T-2, \ldots$

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)\ldots$
…and learn $\boldsymbol{\eta}_t$.

Step 1:
Value Function Approximation

	Fitted Value Iteration	Approx. Backward Induction
Computation	Expensive	Cheaper
	Parallelizeable	Sequential
Interruptibility	Anytime	Needs to finish
	Better for long horizons	Weak for long horizons
Robustness	Can handle noisy targets better	Noisy targets can render it poor

Variants & Extensions

Variant A:
A Forward-Backward Algorithm

The problem with random sampling

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

For each $t \in \{T, T-1,\ T-2, \dots\}$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)\dots$
…and learn $\boldsymbol{\eta}_t$.

Step 1:
Value Function Approximation

There a problem with this algorithm.
Do you see it?

The problem with random sampling

Random samples can be inconsistent across stages.

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

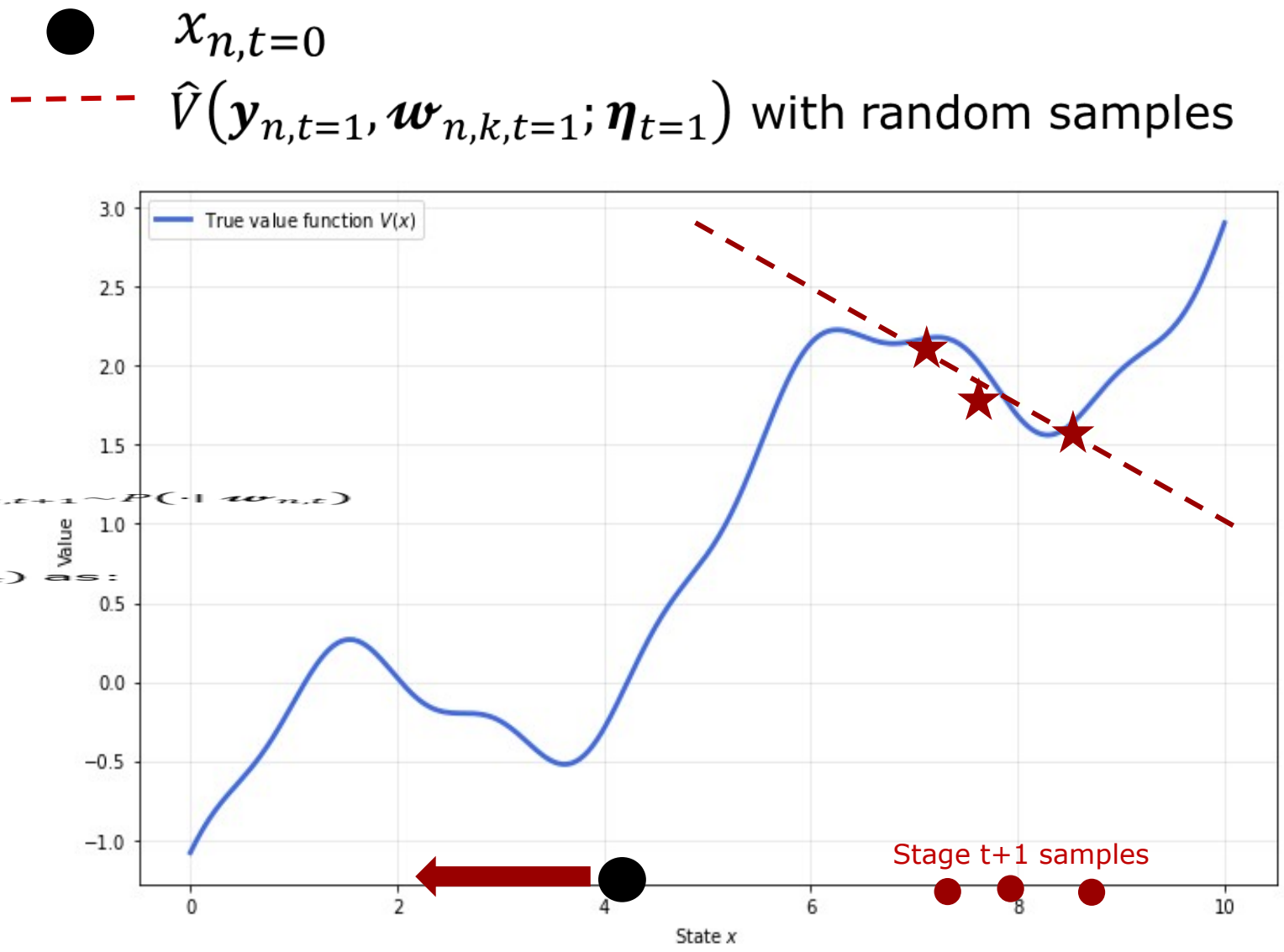
For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$



Forward Pass

Random samples can be inconsistent across stages.

For each $t \in T$:

Initialize $\boldsymbol{\eta}_t$

Sample N states $\{\boldsymbol{x}_{n,t}\}_{n,t} = \{(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t})\}_{n,t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

For each $t \in \{T, T-1,\ T-2, \dots\}$:

For each n:

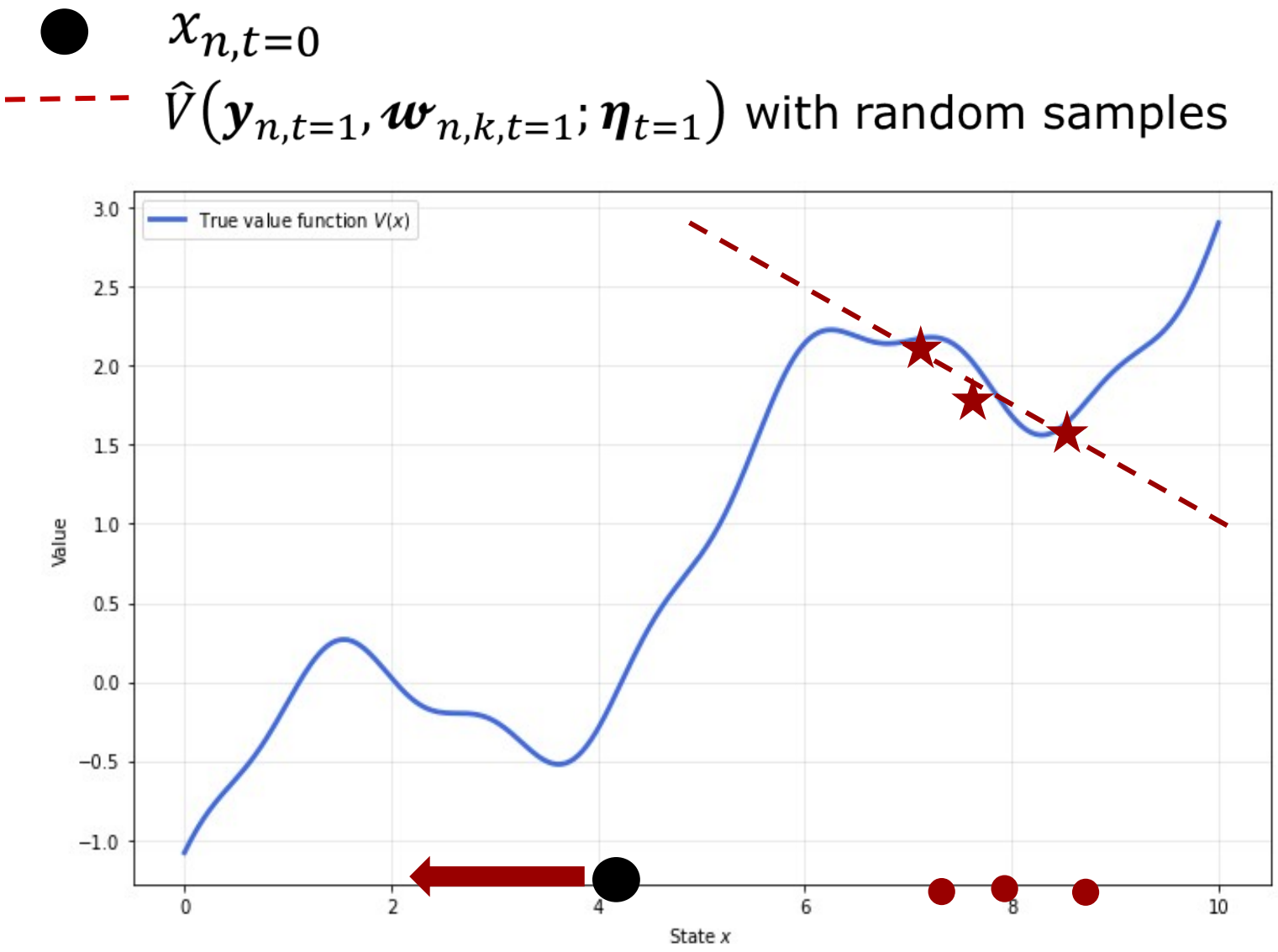
Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$



Forward Pass

Random samples can be inconsistent across stages.

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

Initialize $\boldsymbol{\eta}_t$

For each n:

Forward pass

For each $t \in \{T, T-1,\ T-2, \ldots\}$:

For each n:

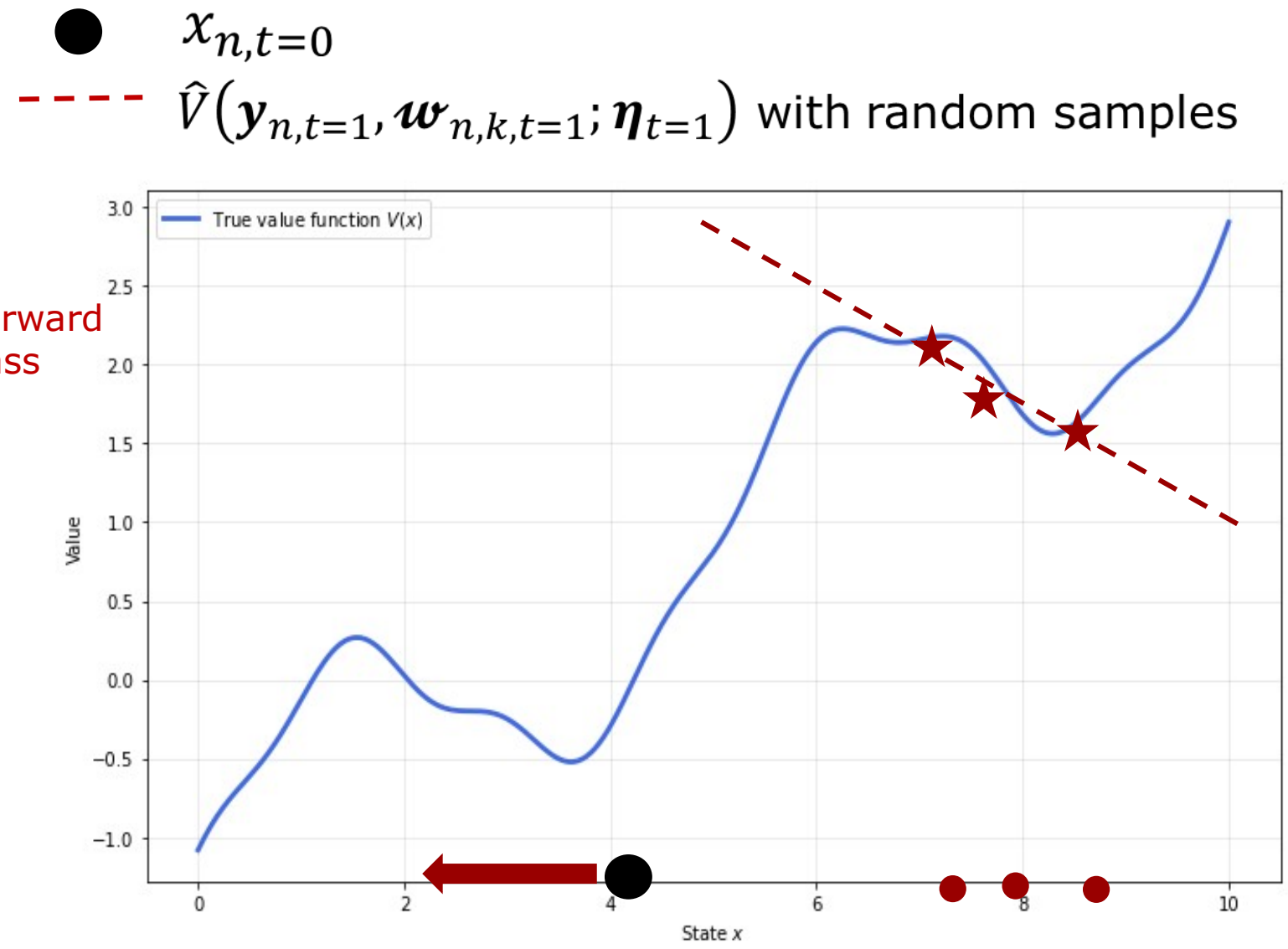
Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$



Forward Pass

Random samples can be inconsistent across stages.

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

Initialize $\boldsymbol{\eta}_t$

For each n:

Given a state $\boldsymbol{x}_{n,t}$, what do I need to get a next state?

Forward pass

For each $t \in \{T, T-1,\ T-2, \ldots\}$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

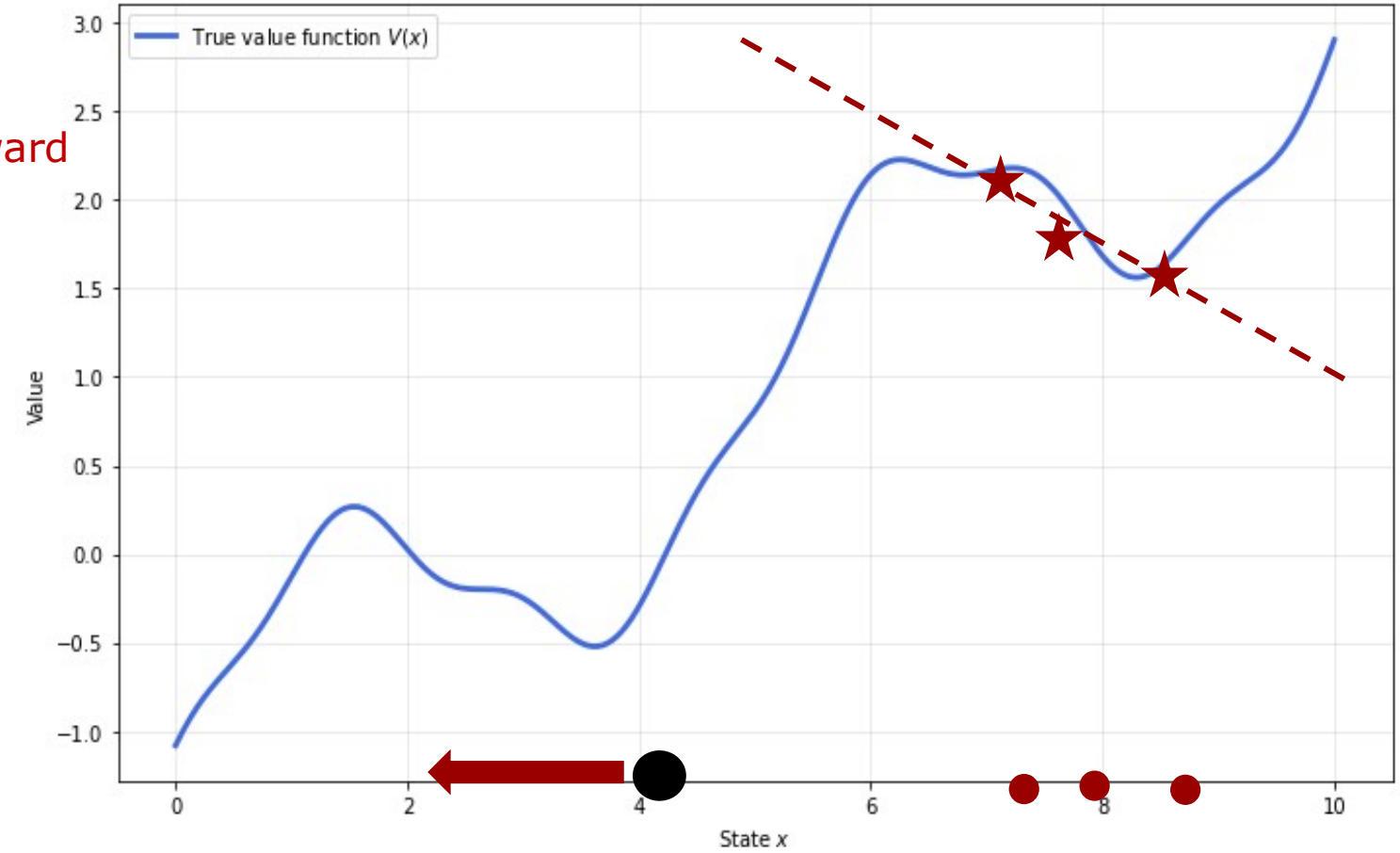
subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

$x_{n,t=0}$

$\hat{V}(\boldsymbol{y}_{n,t=1}, \boldsymbol{w}_{n,k,t=1}; \boldsymbol{\eta}_{t=1})$ with random samples



Forward Pass

Random samples can be inconsistent across stages.

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \dots\}$:

Initialize $\boldsymbol{\eta}_t$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,t+1} = f(\boldsymbol{y}_{n,t}, \sum_k p_k \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Forward pass

For each $t \in \{T, T-1, T-2, \dots\}$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

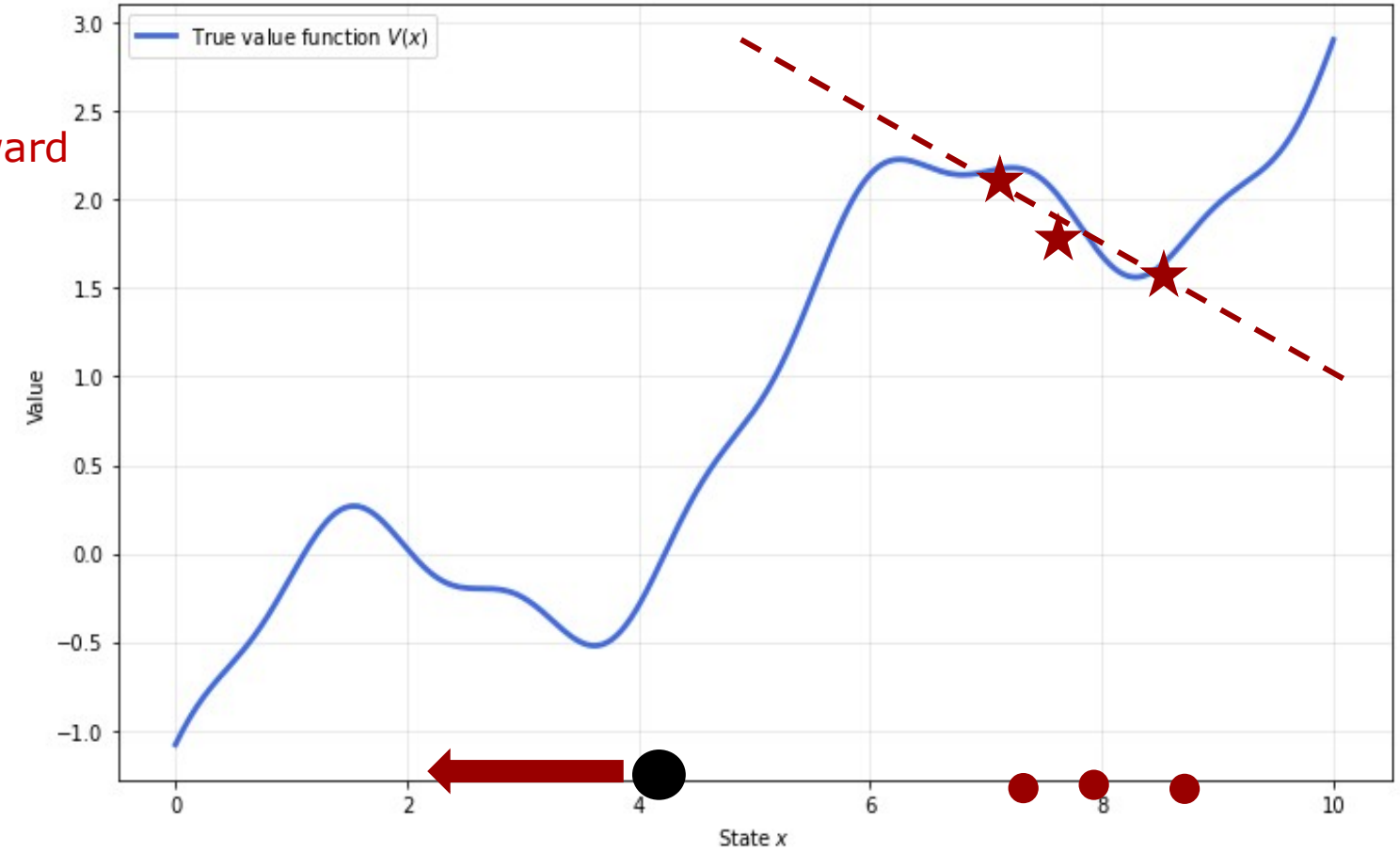
subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

● $x_{n,t=0}$

- - - $\hat{V}(\boldsymbol{y}_{n,t=1}, \boldsymbol{w}_{n,k,t=1}; \boldsymbol{\eta}_{t=1})$ with random samples



Forward Pass

Random samples can be inconsistent across stages.

You want to sample in the area where you actually intend to go

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

Initialize $\boldsymbol{\eta}_t$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,t+1} = f(\boldsymbol{y}_{n,t}, \sum_k p_k \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Forward pass

For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

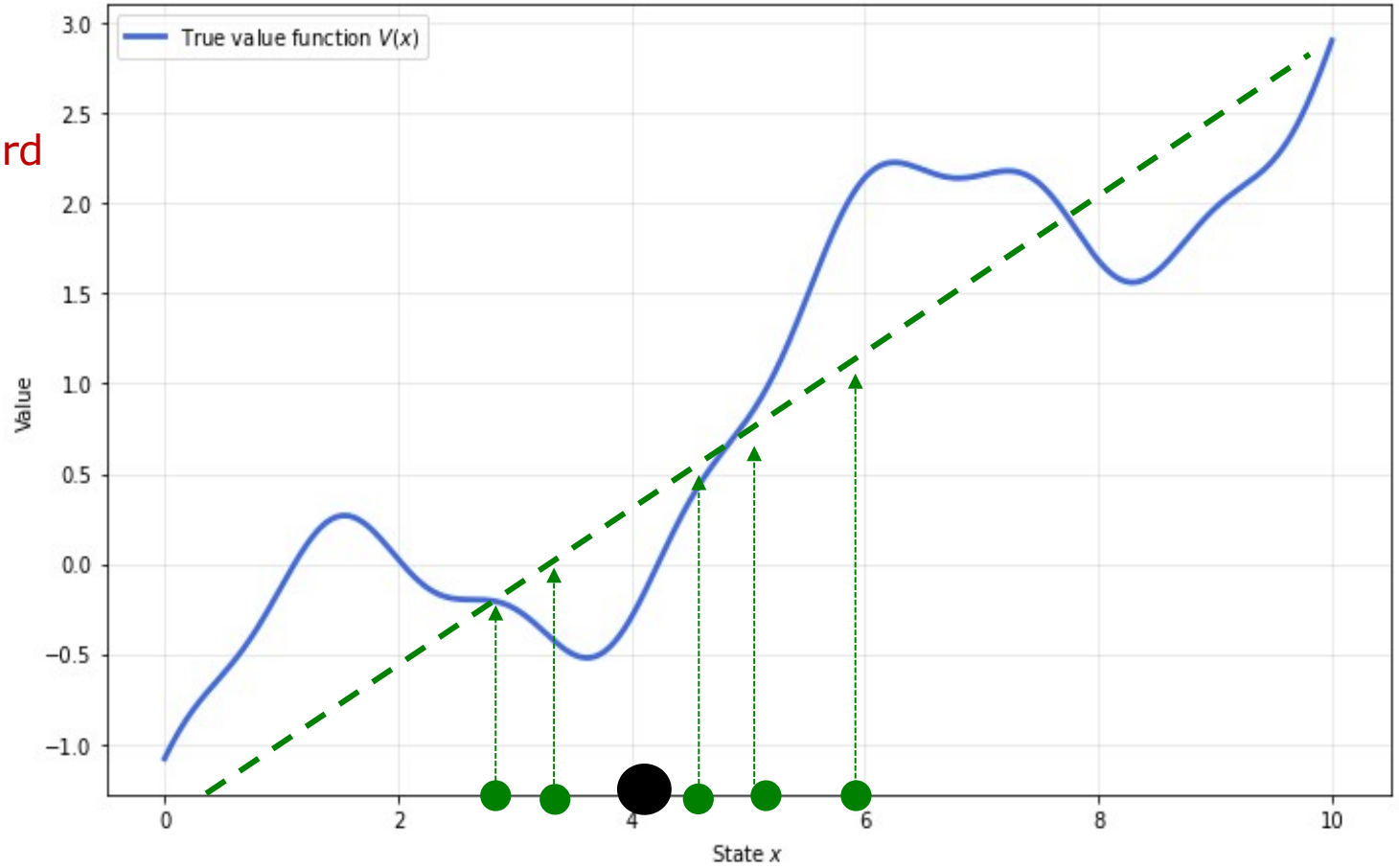
Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$



Forward Pass

Random samples can be inconsistent across stages.

You want to sample in the area where you actually intend to go

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

Initialize $\boldsymbol{\eta}_t$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,t+1} = f(\boldsymbol{y}_{n,t}, \sum_k p_k \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Forward pass

For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

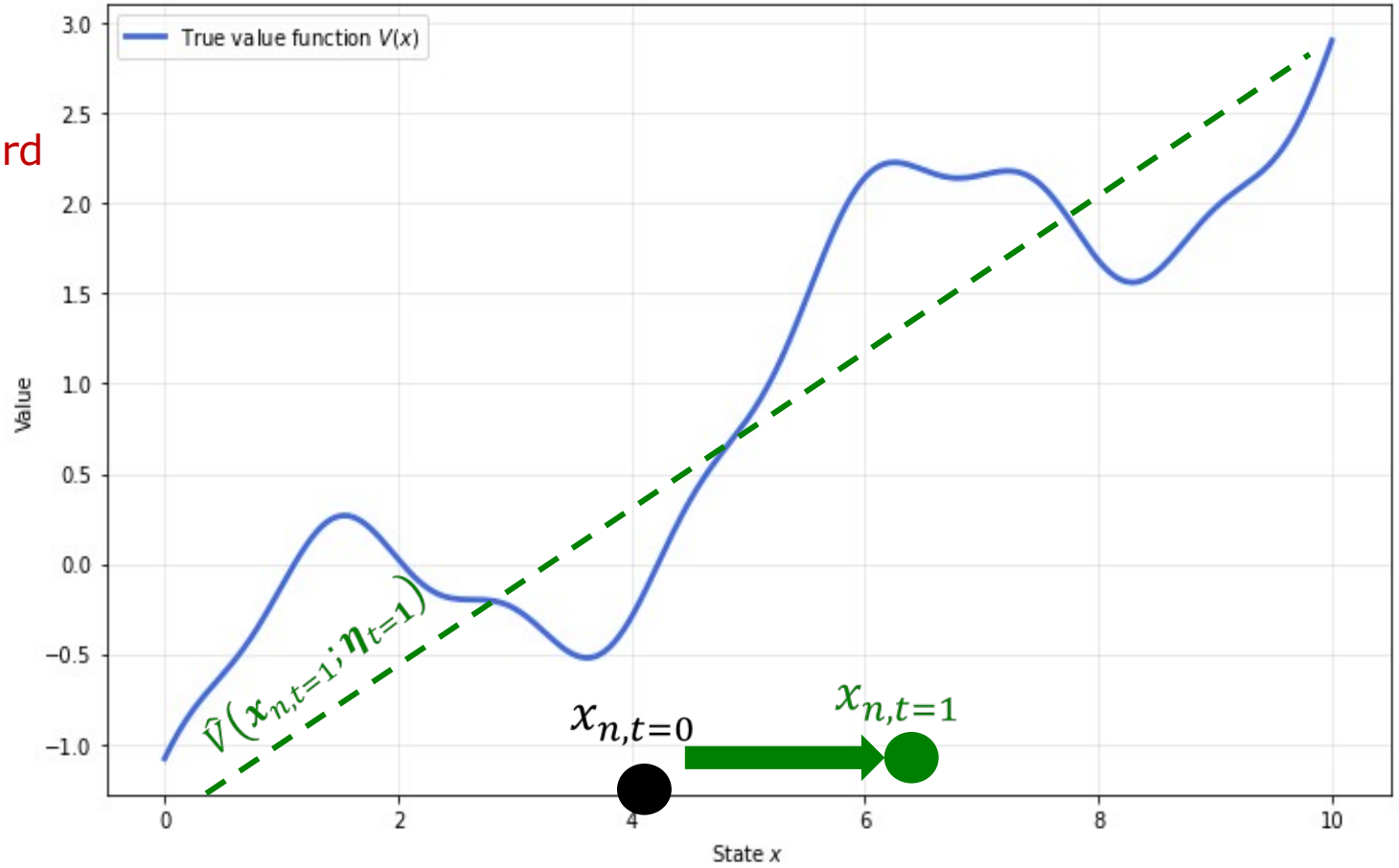
Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$



Forward Pass

Random samples can be inconsistent across stages.

You want to sample in the area where you actually intend to go

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

Initialize $\boldsymbol{\eta}_t$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$\hat{V}(\boldsymbol{x}_{n,t=2}; \boldsymbol{\eta}_{t=2})$$

Forward pass

For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

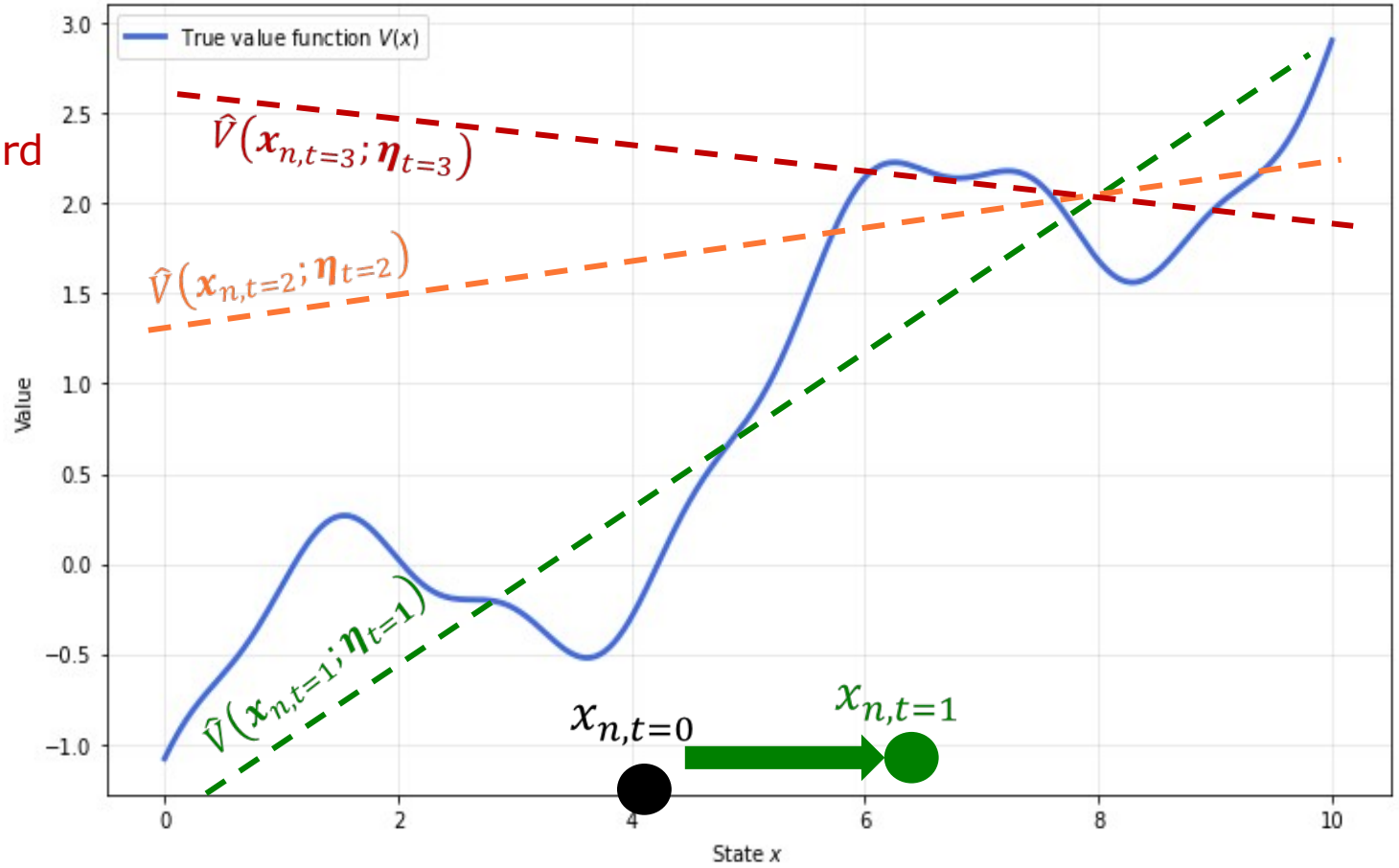
Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$



Forward Pass

Random samples can be inconsistent across stages.

You want to sample in the area where you actually intend to go

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

Initialize $\boldsymbol{\eta}_t$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$\hat{V}(\boldsymbol{x}_{n,t=2}; \boldsymbol{\eta}_{t=2})$$

Forward pass

For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

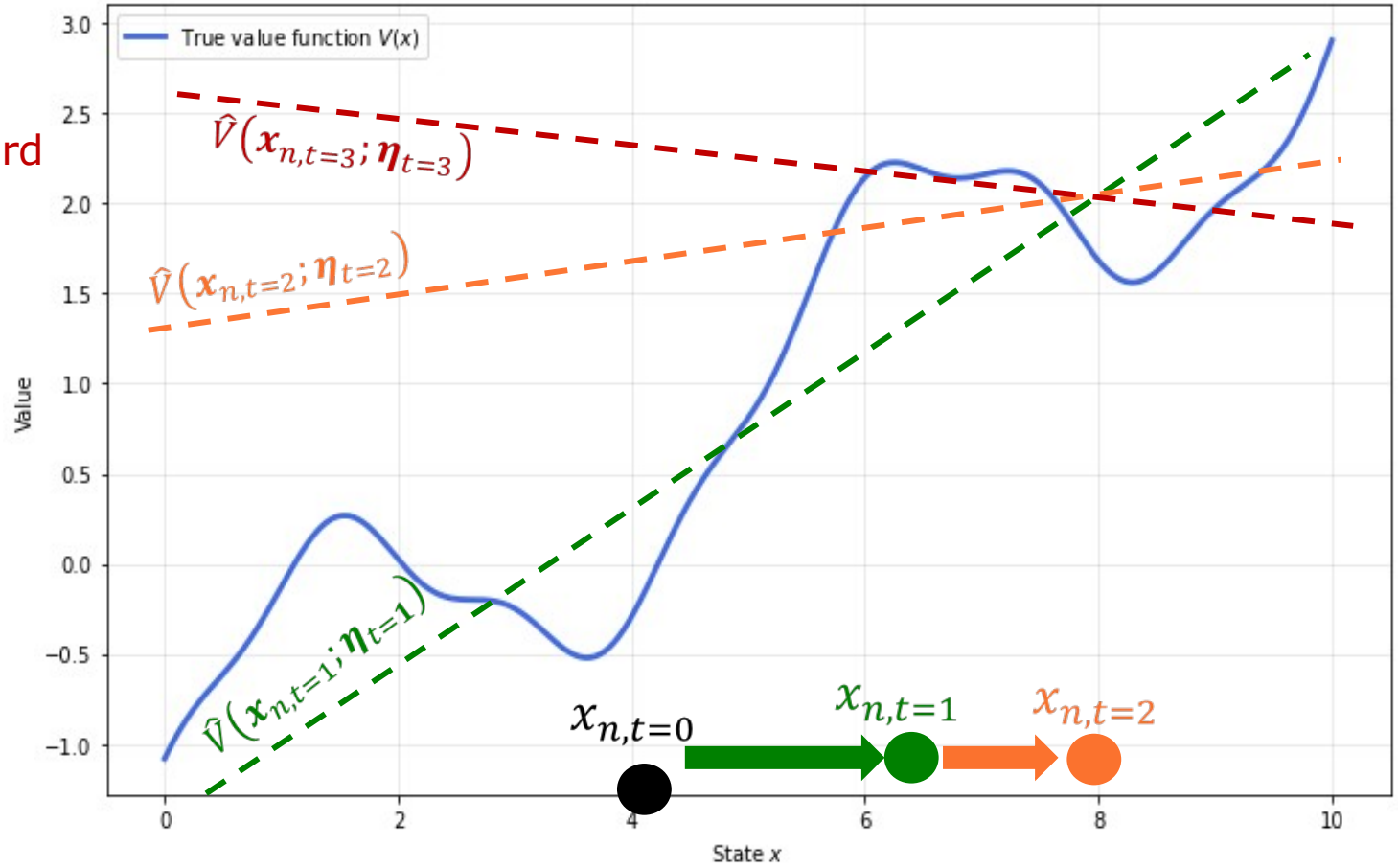
Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$



Forward Pass

Random samples can be inconsistent across stages.

You want to sample in the area where you actually intend to go

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

Initialize $\boldsymbol{\eta}_t$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$\hat{V}(\boldsymbol{x}_{n,t=2}; \boldsymbol{\eta}_{t=2})$$

Forward pass

For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

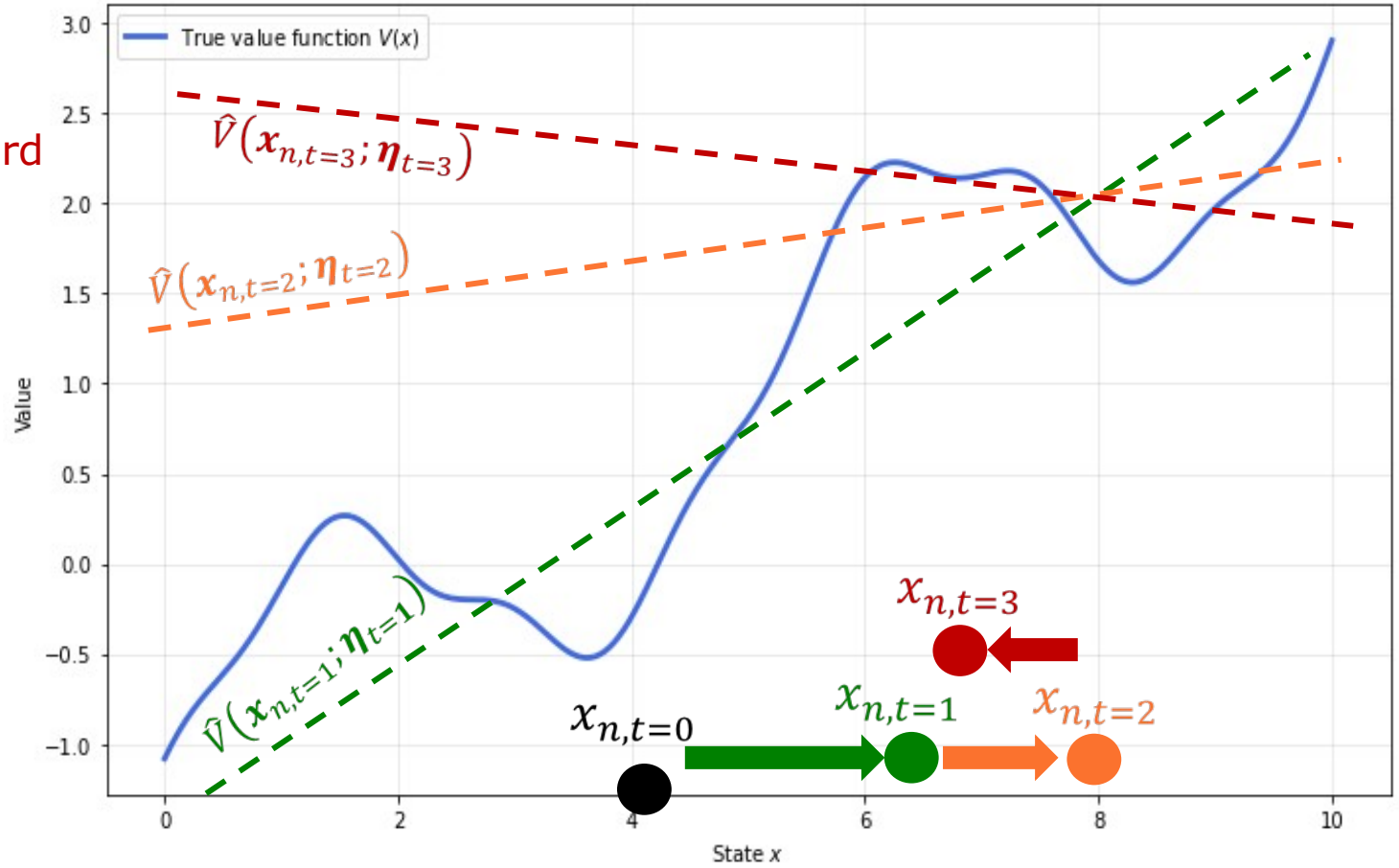
Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$



Forward Pass

But…
I drew samples based on an initial policy (forward pass);
and then changed the policy (backward pass)!
So my samples are no longer consistent with the new policy

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \dots\}$:

Initialize $\boldsymbol{\eta}_t$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,t+1} = f(\boldsymbol{y}_{n,t}, \sum_k p_k \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Forward pass

For each $t \in \{T, T-1, T-2, \dots\}$:

For each n:

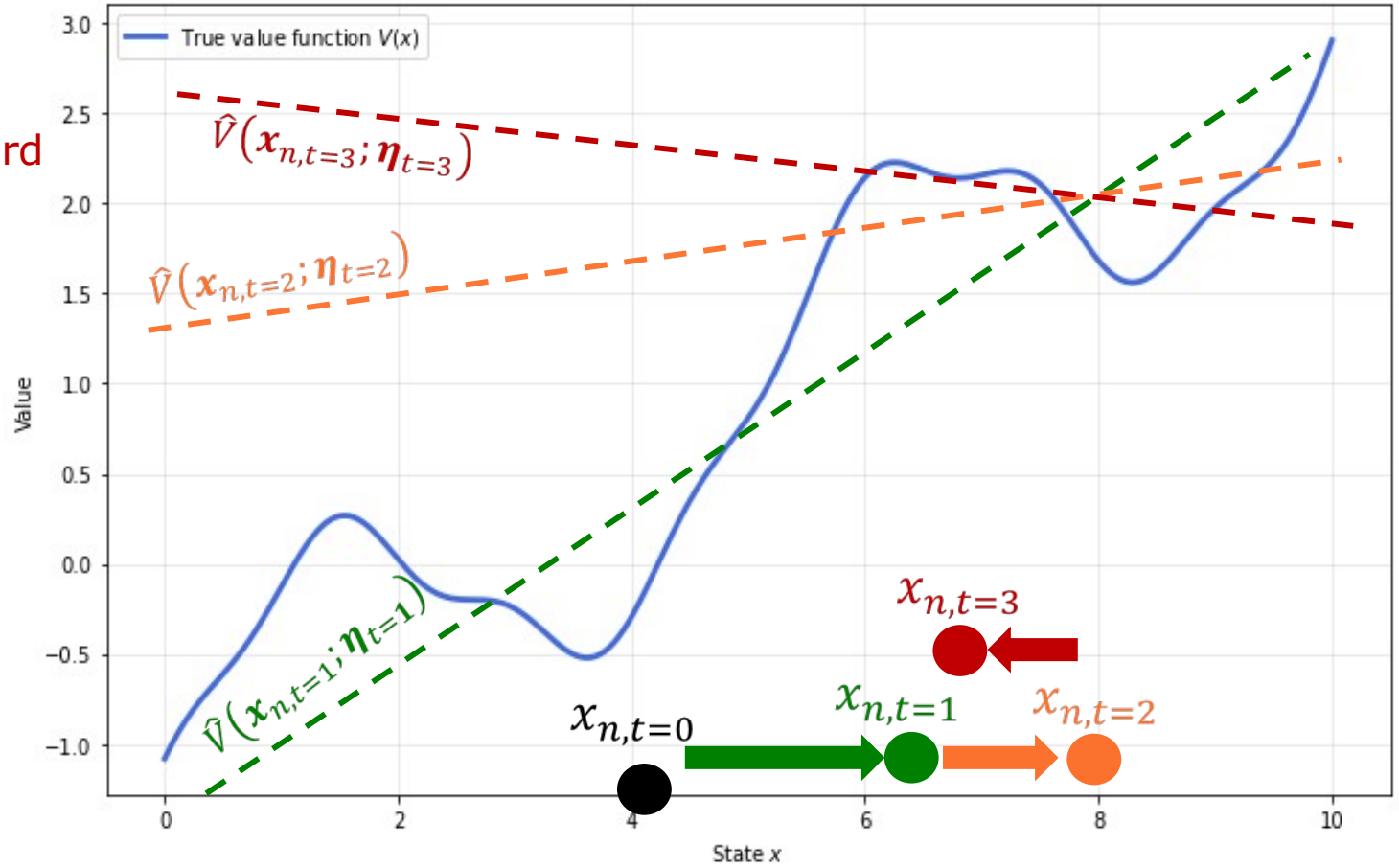
Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$



Forward Pass

But…
I drew samples based on an initial policy (forward pass);
and then changed the policy (backward pass)!
So my samples are no longer consistent with the new policy

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \dots\}$:

Initialize $\boldsymbol{\eta}_t$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,t+1} = f(\boldsymbol{y}_{n,t}, \sum_k p_k \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Forward pass

For each $t \in \{T, T-1, T-2, \dots\}$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

Backward pass

A Forward-Backward Algorithm

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \dots\}$:

Initialize $\boldsymbol{\eta}_t$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving **Bellman**

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,t+1} = f\left(\boldsymbol{y}_{n,t}, \textstyle\sum_k p_k \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t}\right)$$

Forward pass

Compute targets $\mathrm{V}^*(\boldsymbol{x}_{n,t})$

and

Update $\boldsymbol{\eta}_t$

using Fitted Value Iteration or Approx. Backward Induction

For each $t \in \{T, T-1, T-2, \dots\}$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \textstyle\sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

Backward pass

A Forward-Backward Algorithm

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

Initialize $\boldsymbol{\eta}_t$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving Bellman

Forward pass

Compute targets $\mathrm{V}^*(\boldsymbol{x}_{n,t})$

and

Update $\boldsymbol{\eta}_t$

using Fitted Value Iteration or Approx. Backward Induction

Backward pass

A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

$\boxed{\text{Initialize } \boldsymbol{\eta}_t}$

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving Bellman

Forward pass

Compute targets $\mathrm{V}^*(\boldsymbol{x}_{n,t})$

and

Update $\boldsymbol{\eta}_t$

using Fitted Value Iteration or Approx. Backward Induction

Backward pass

A Forward-Backward Algorithm

*this visualization, implicitly assumes that the true Value Function is the same at each stage, which in general is not true.

For I iterations

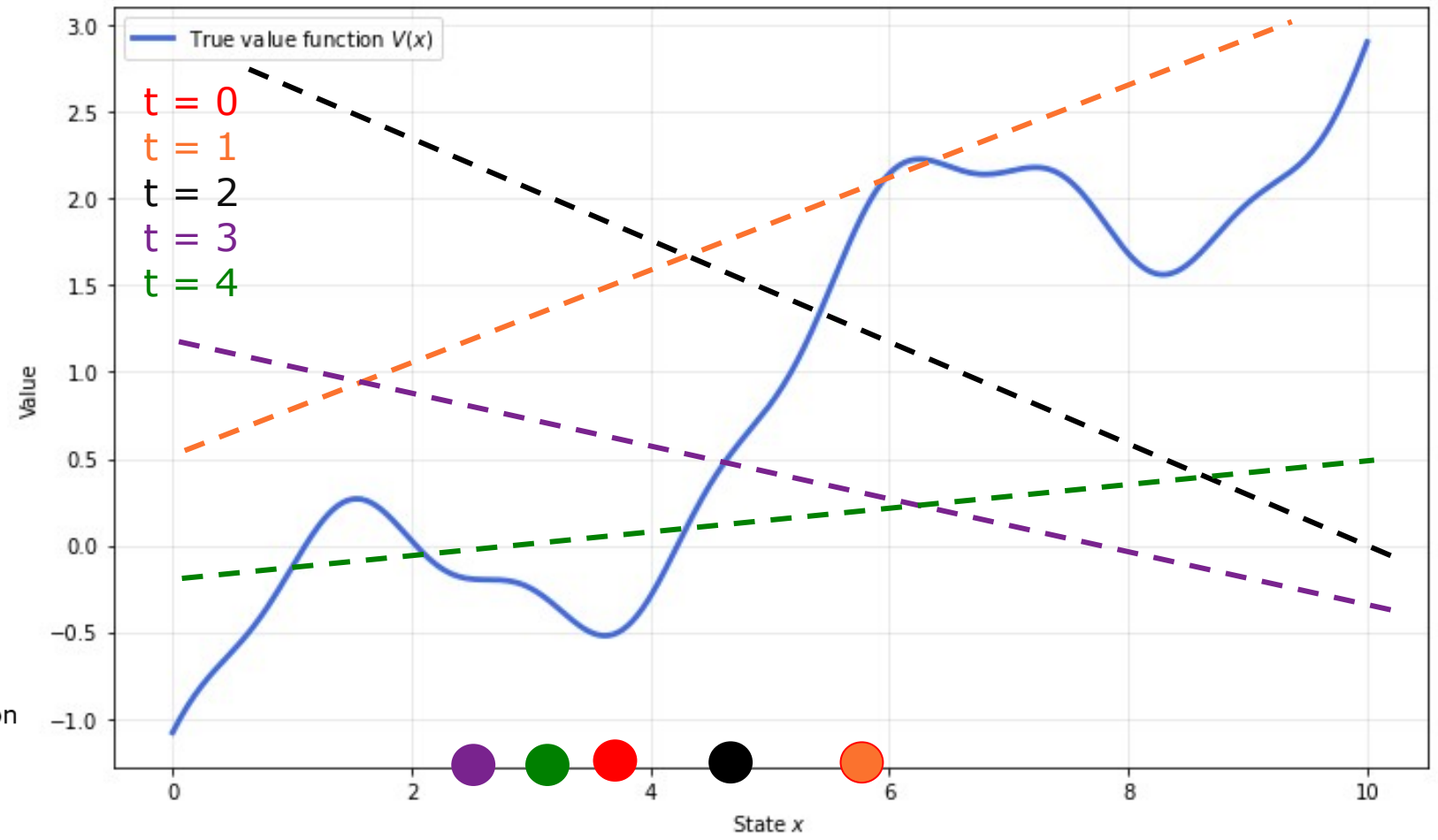
Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

Create a $y_{n,t+1}$ by solving Bellman



Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction

A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

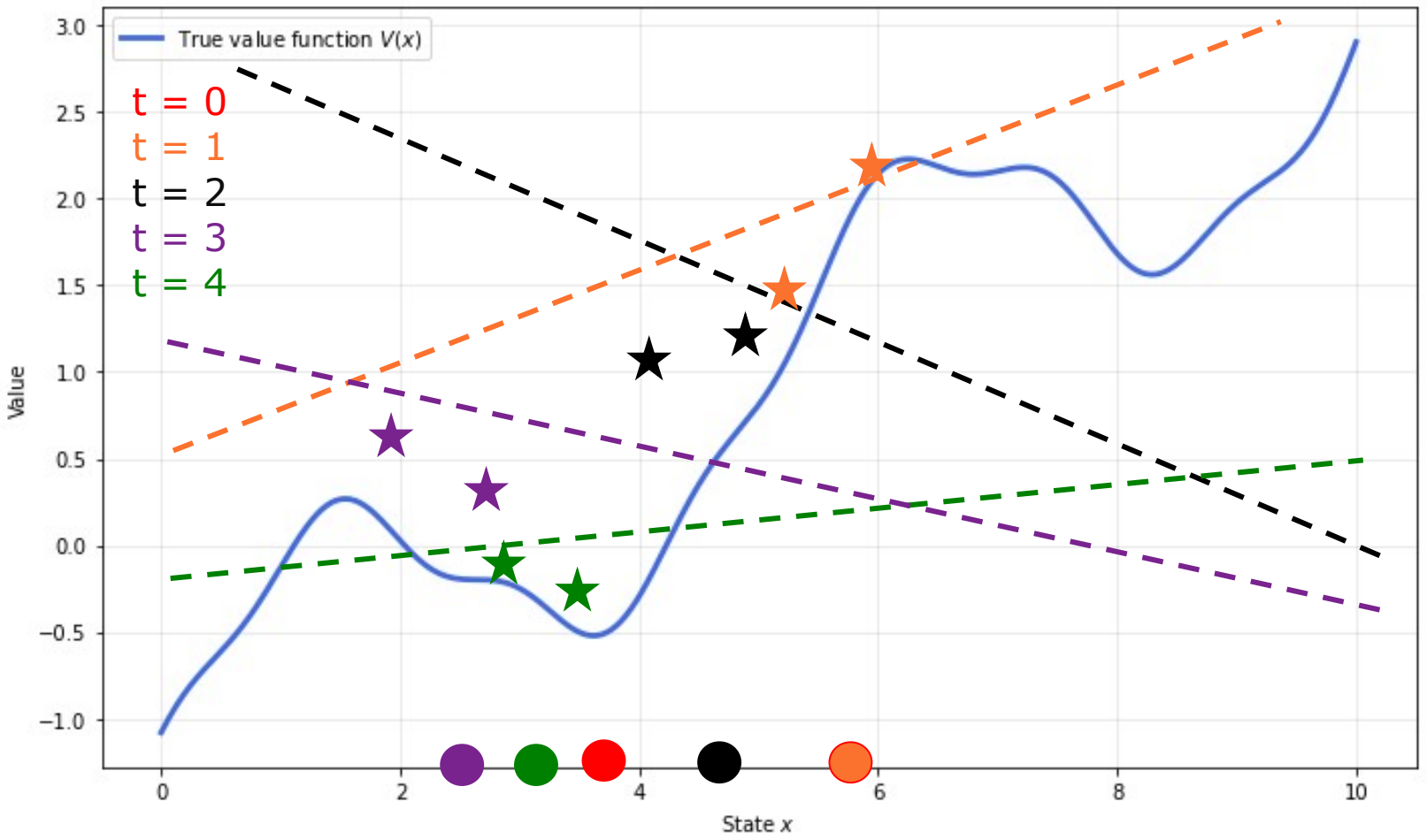
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Inductio



A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

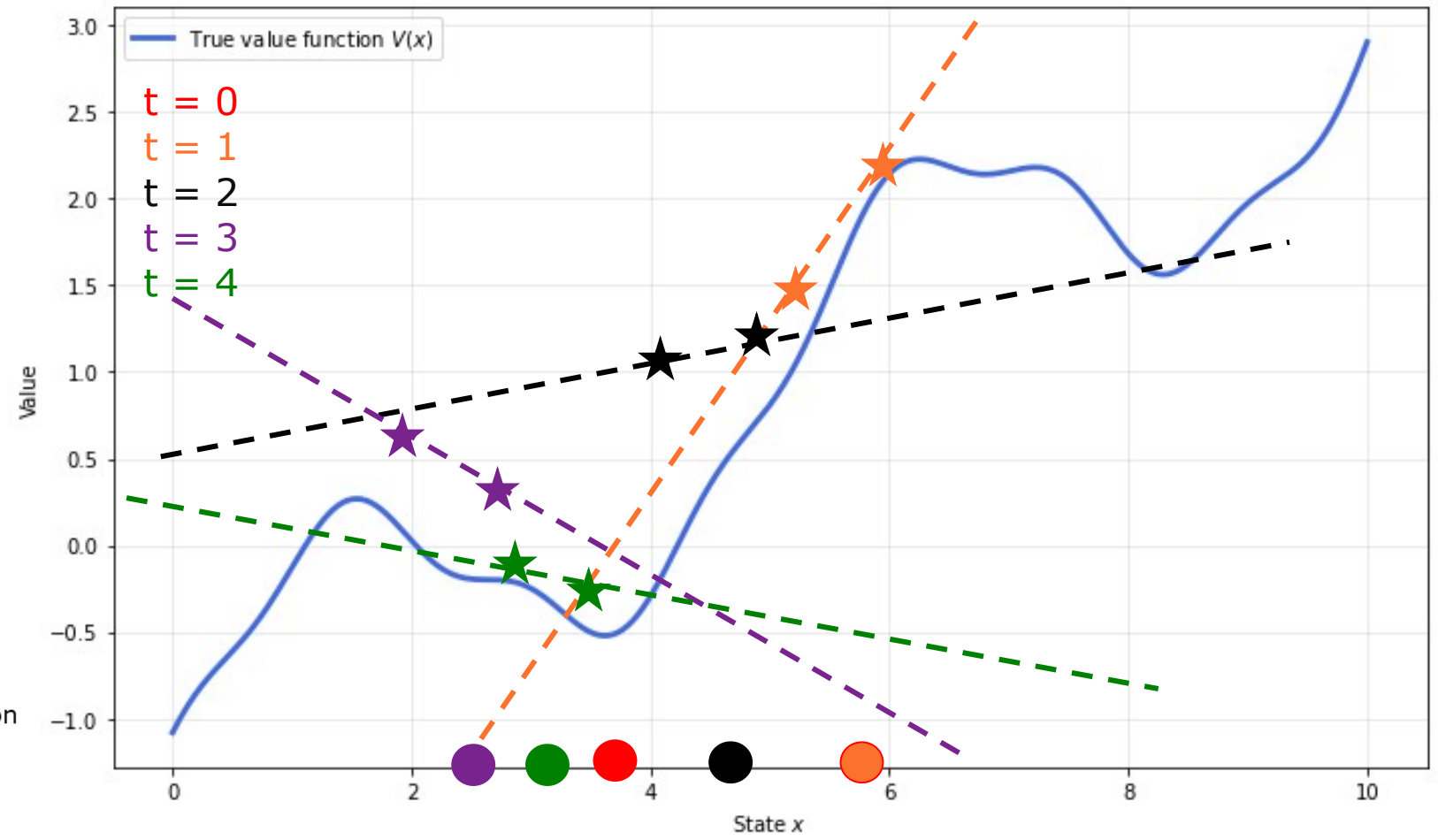
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

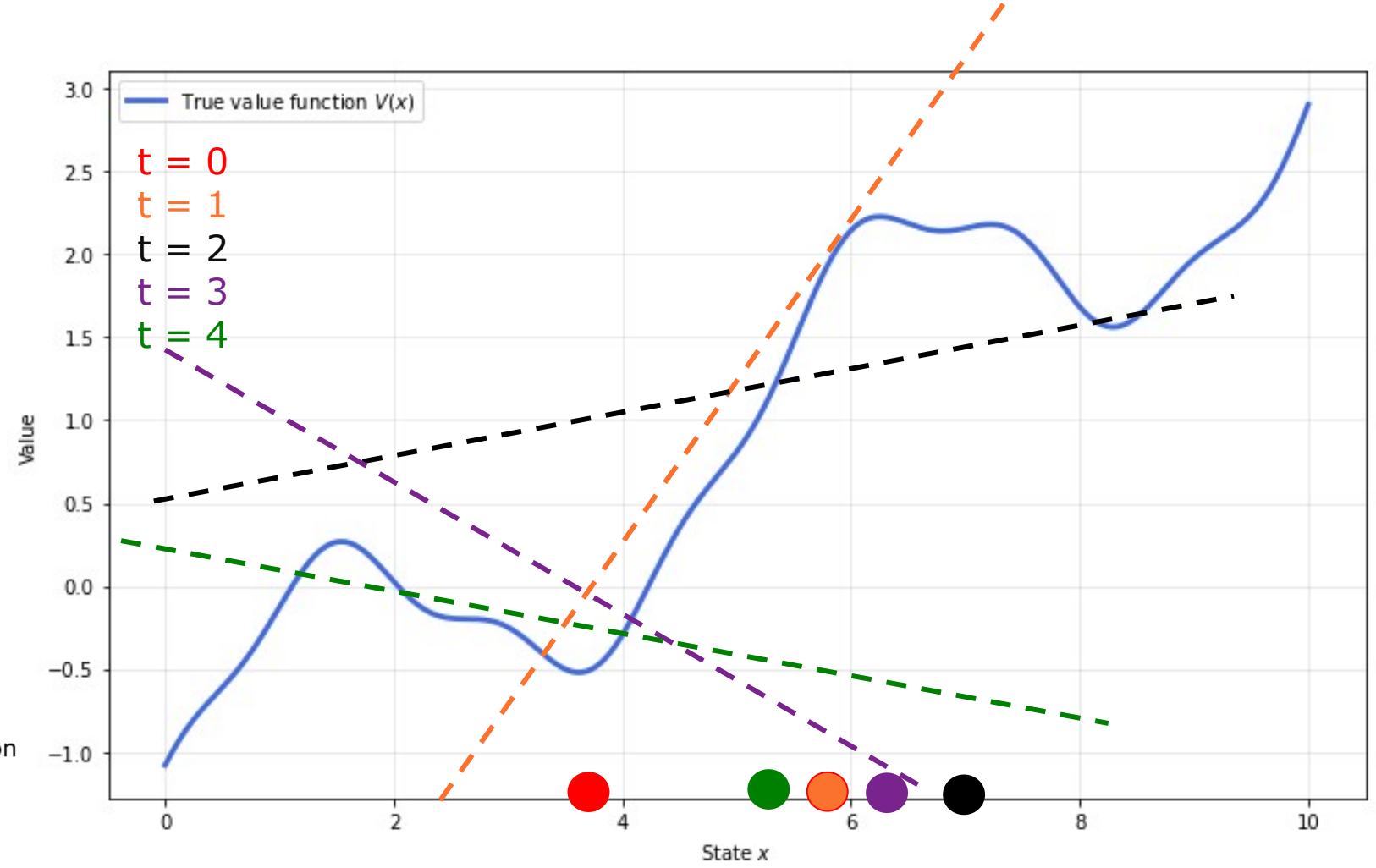
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

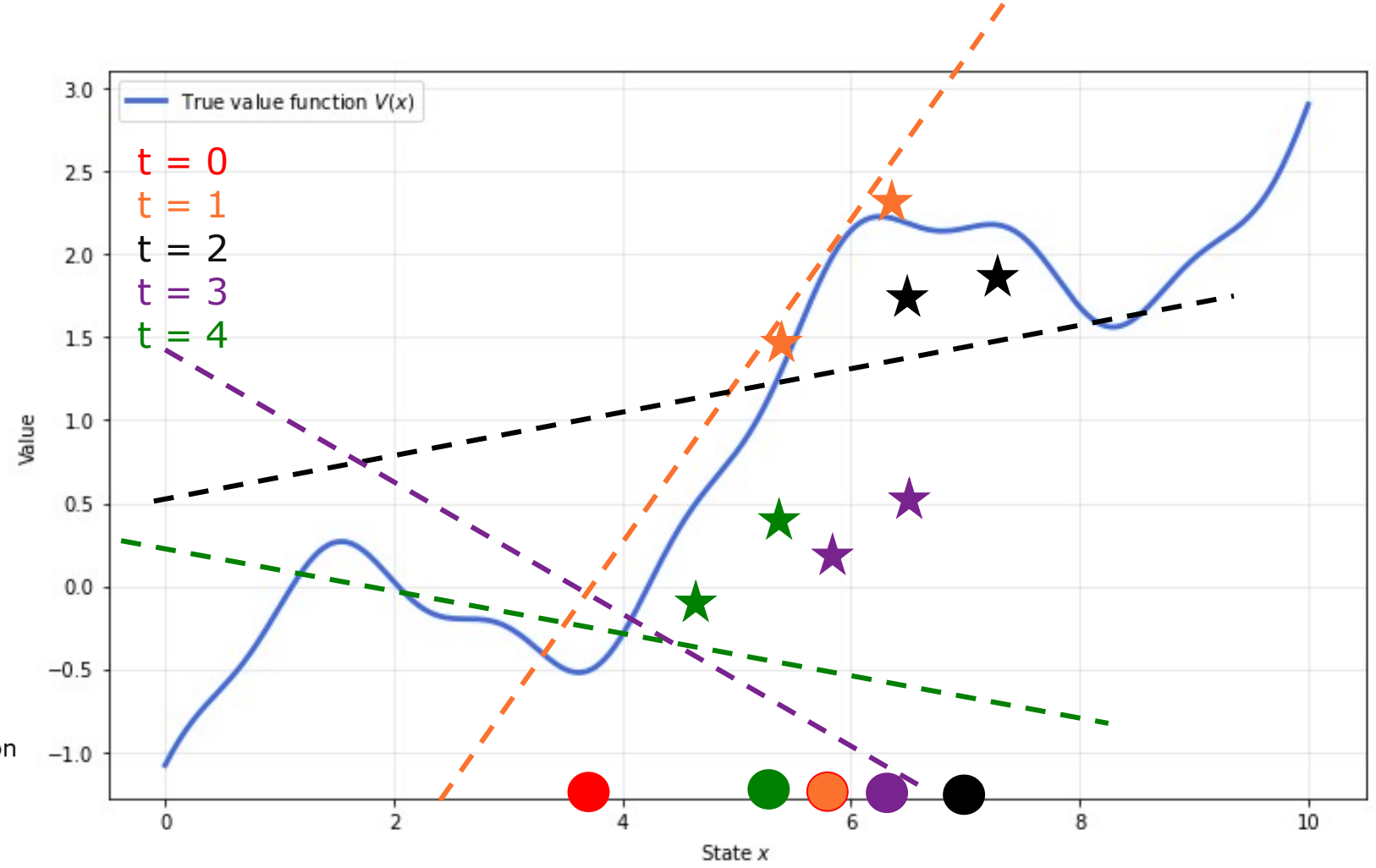
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

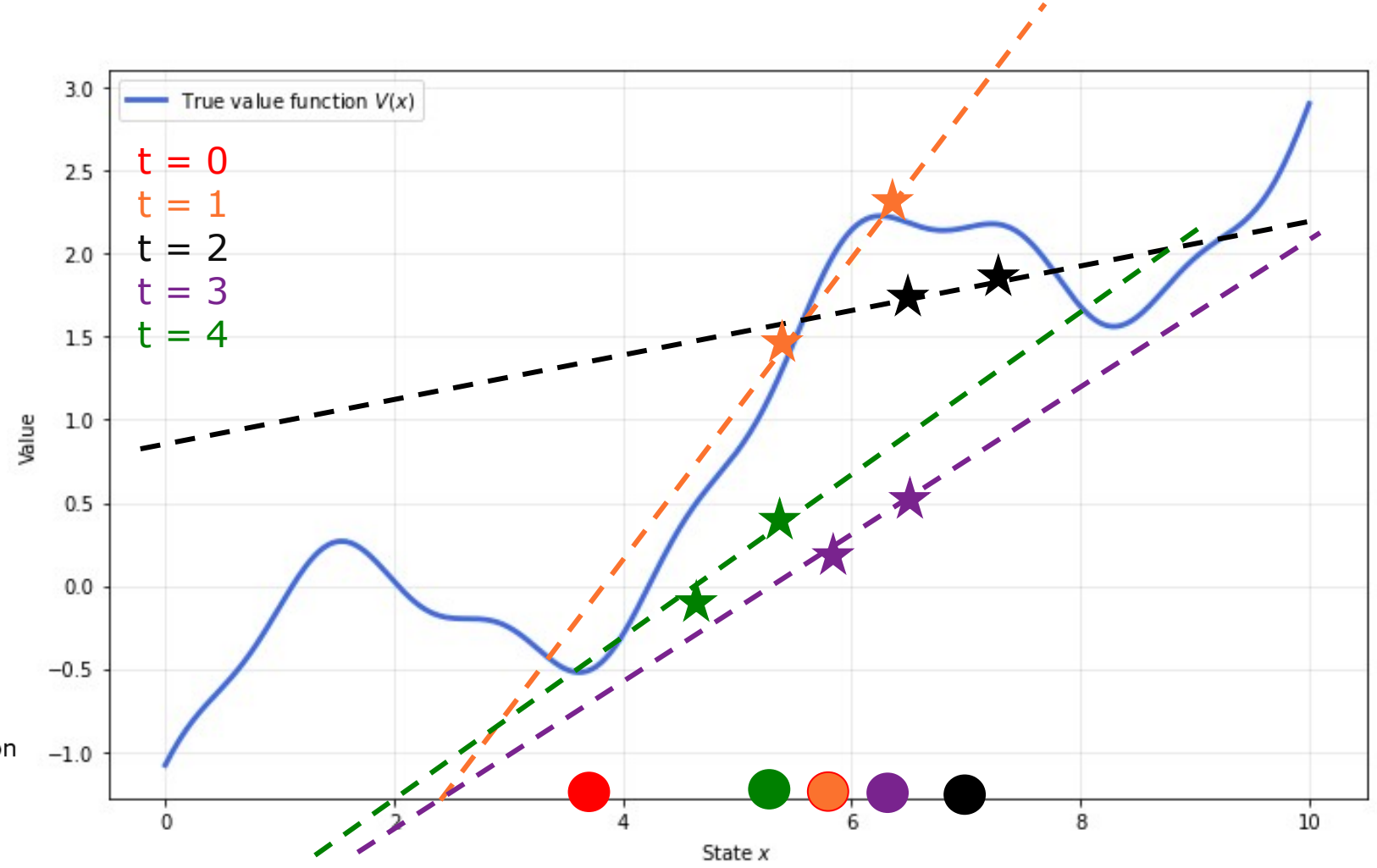
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

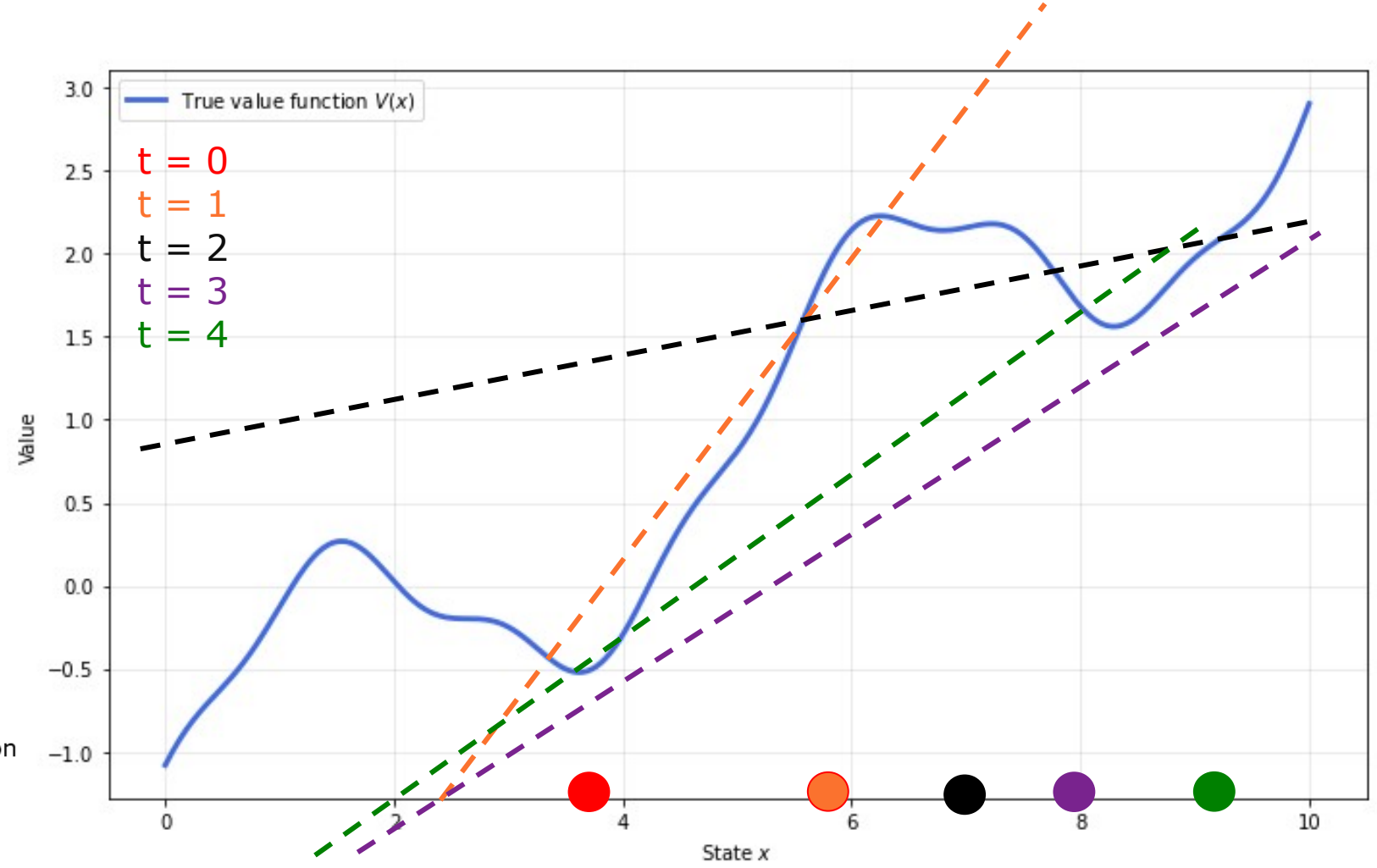
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

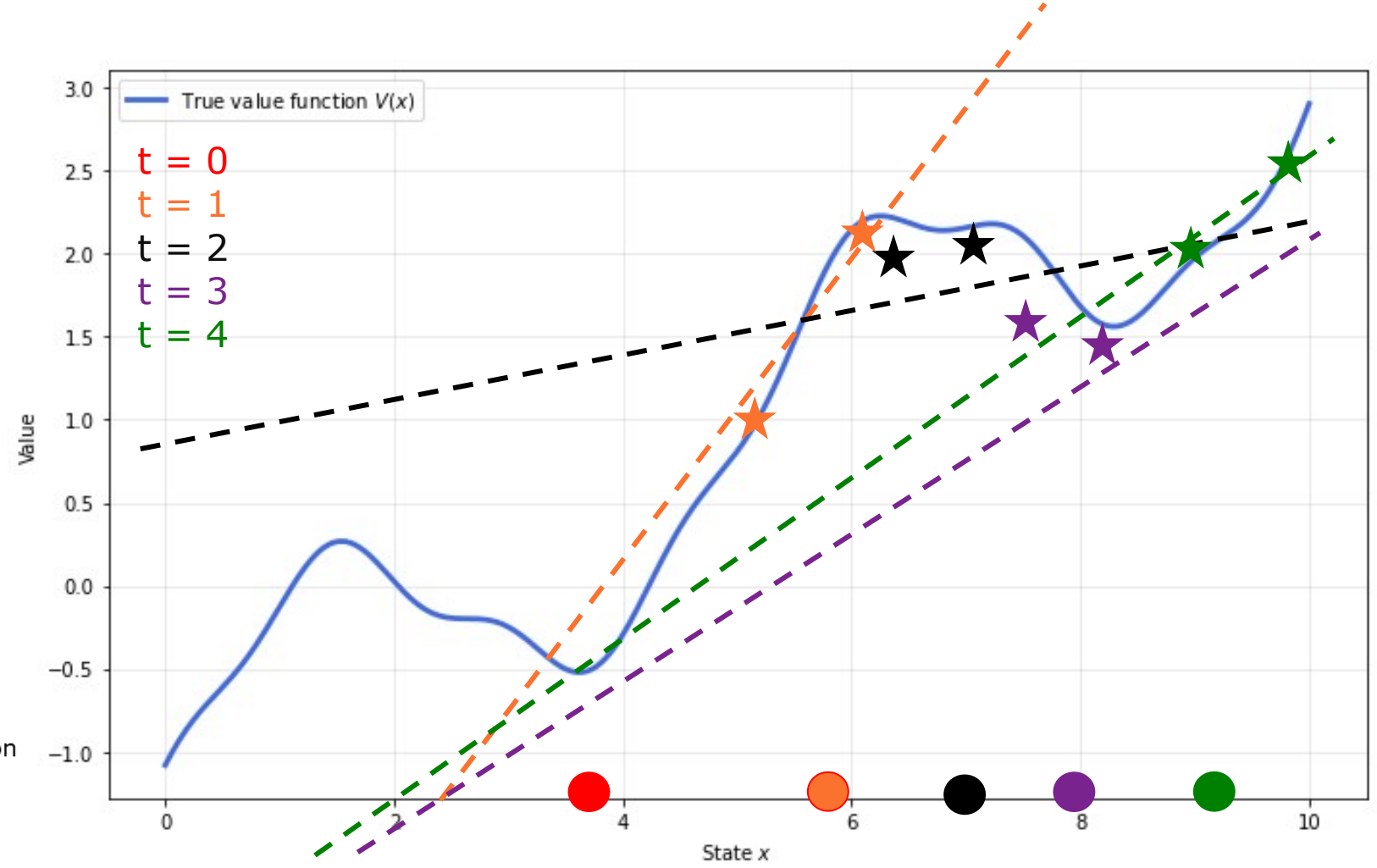
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

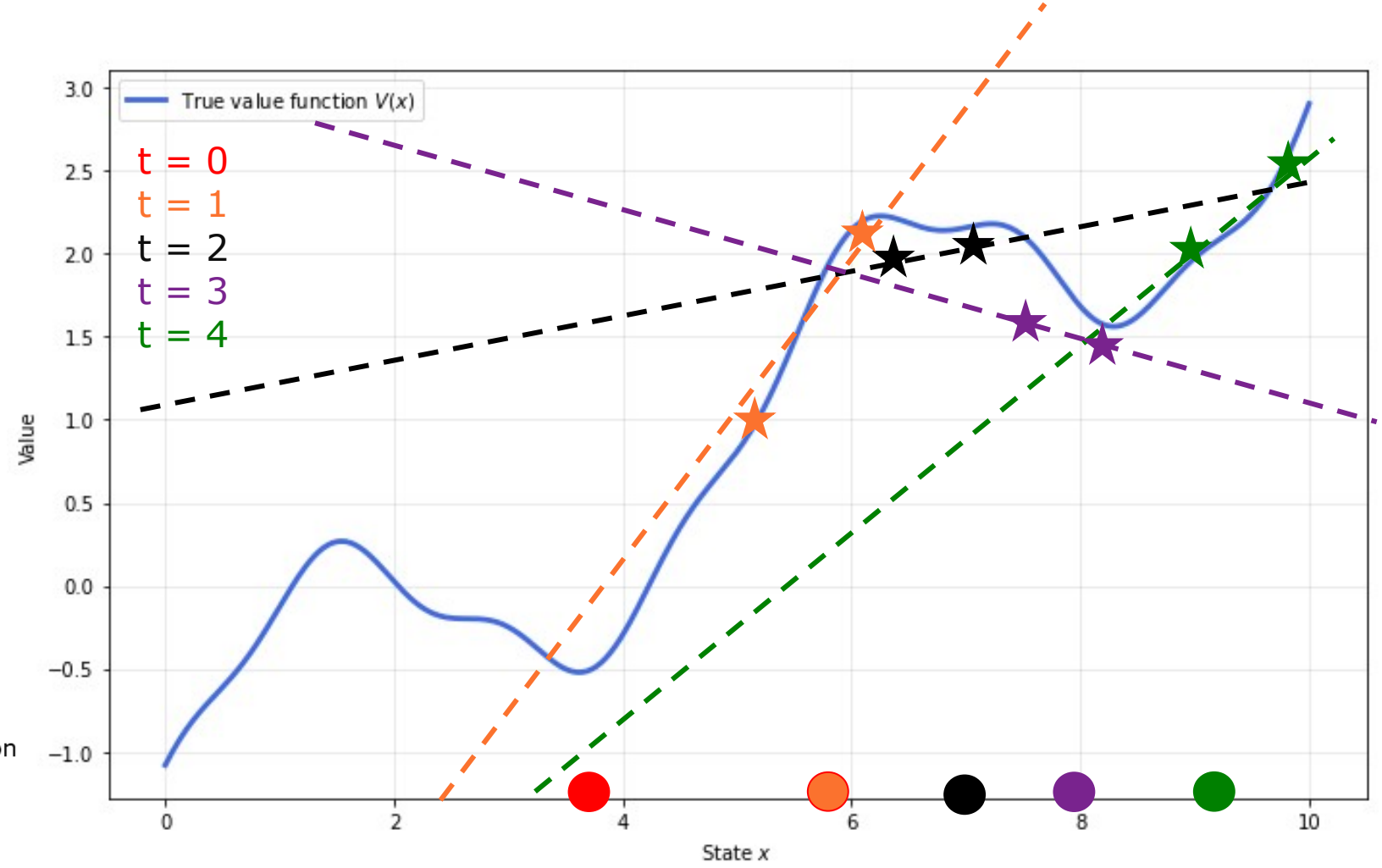
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

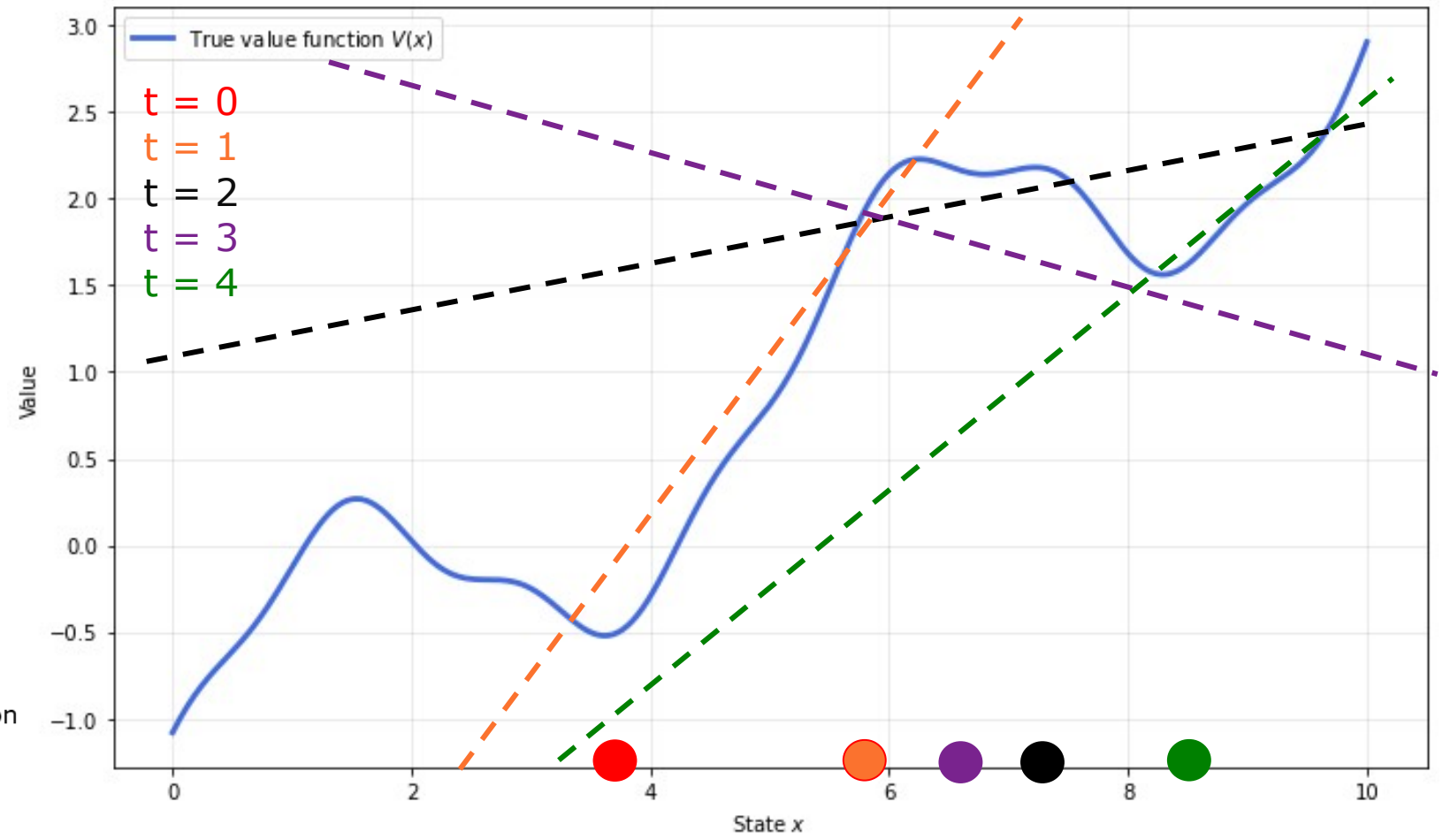
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

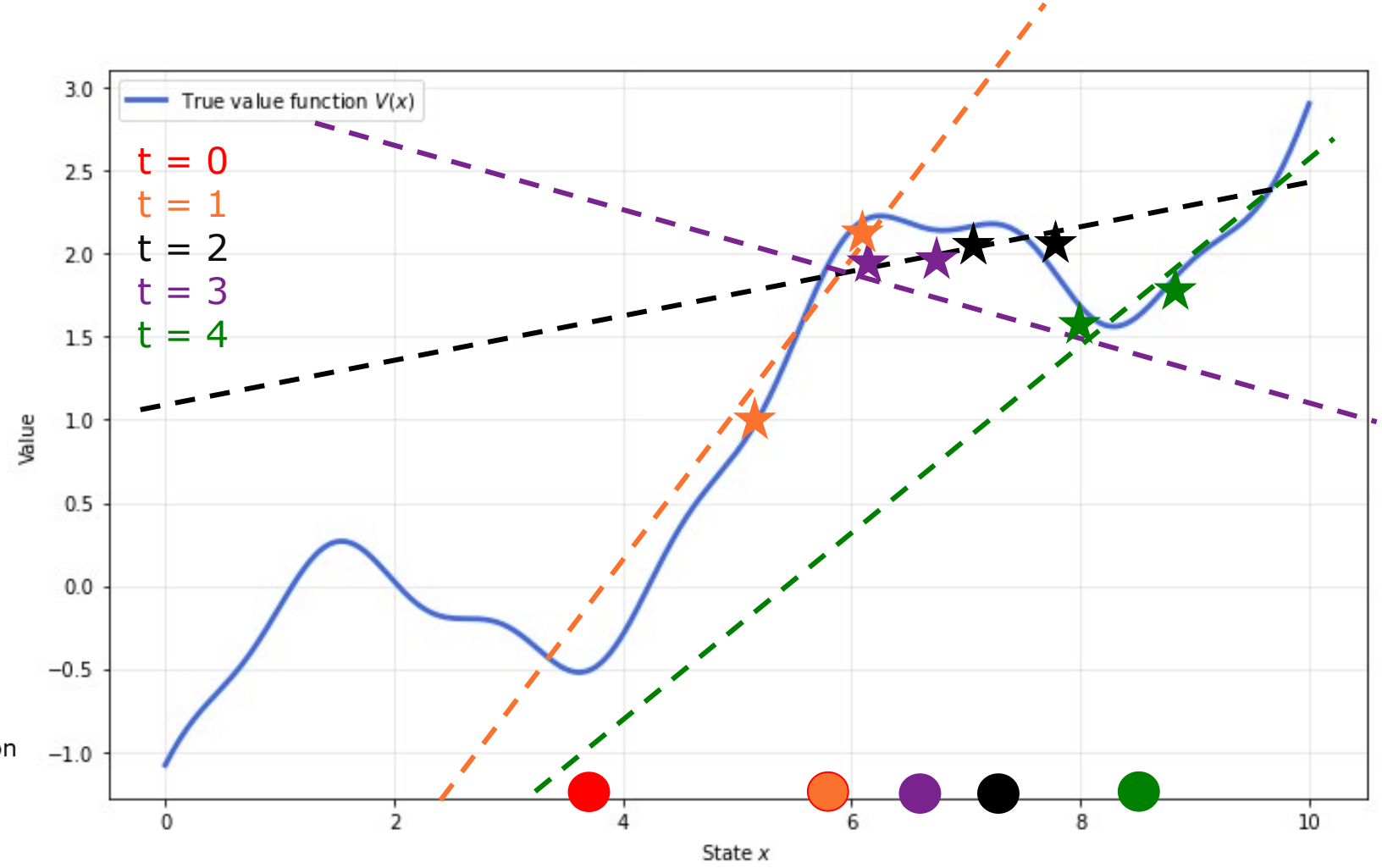
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

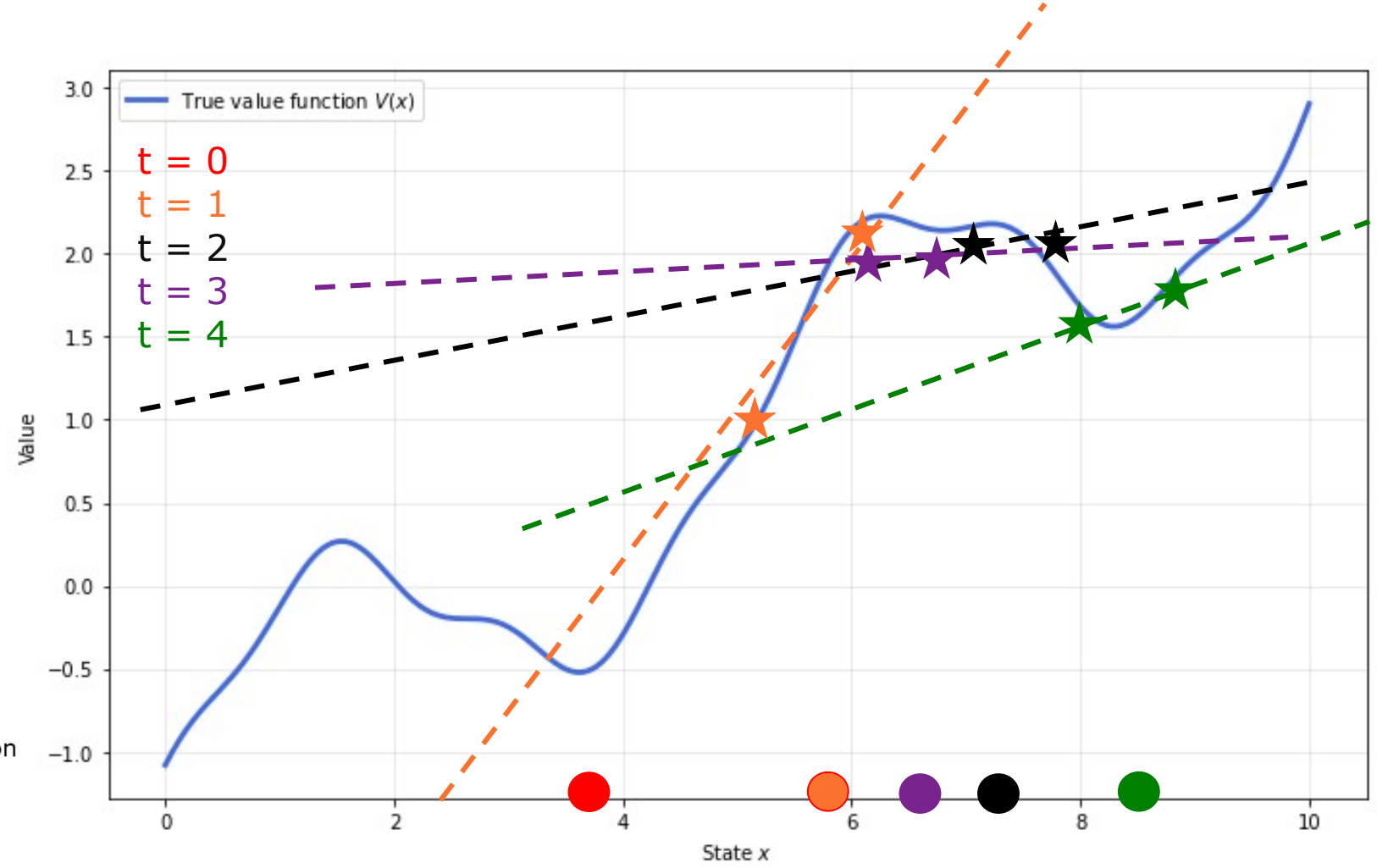
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

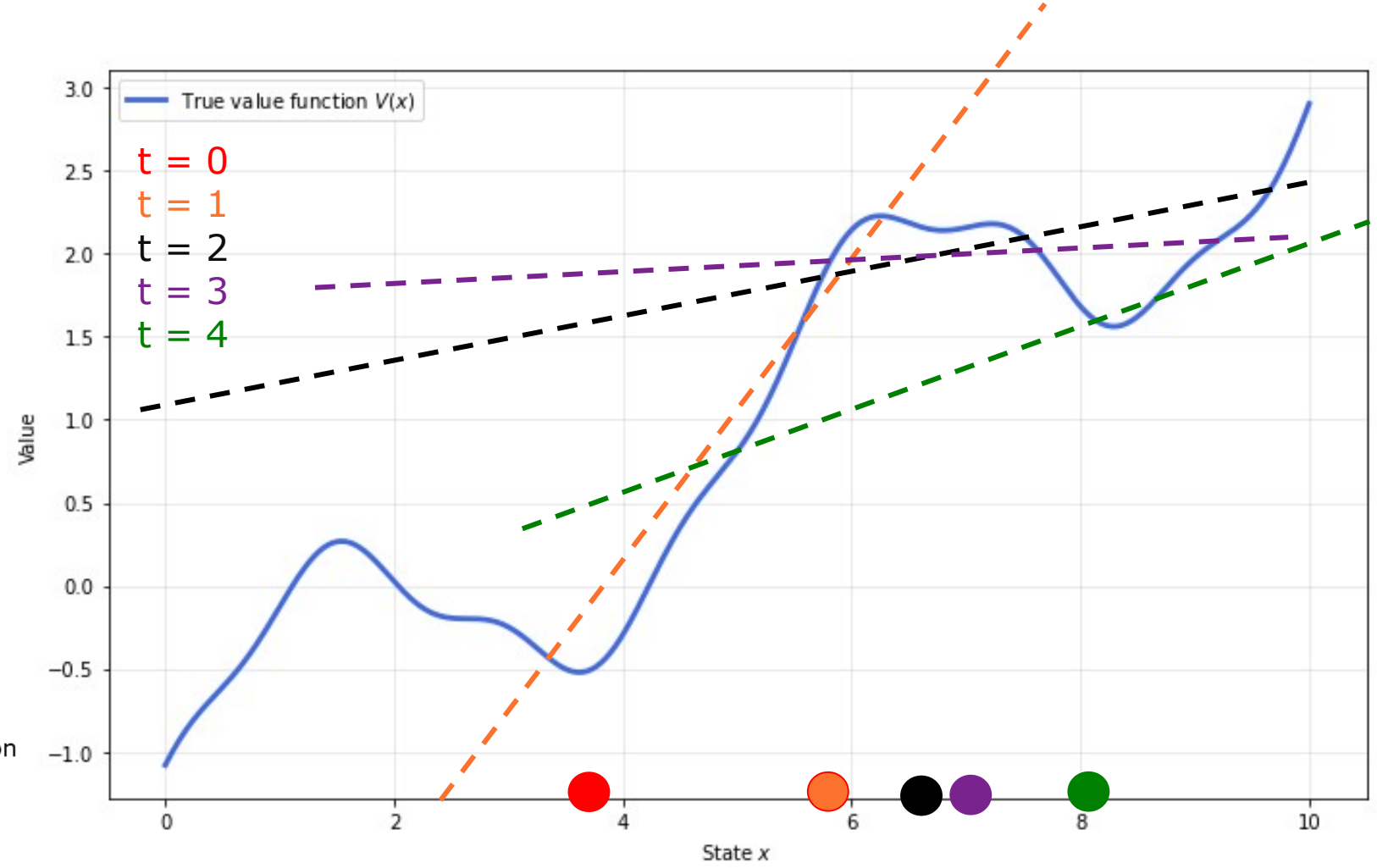
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

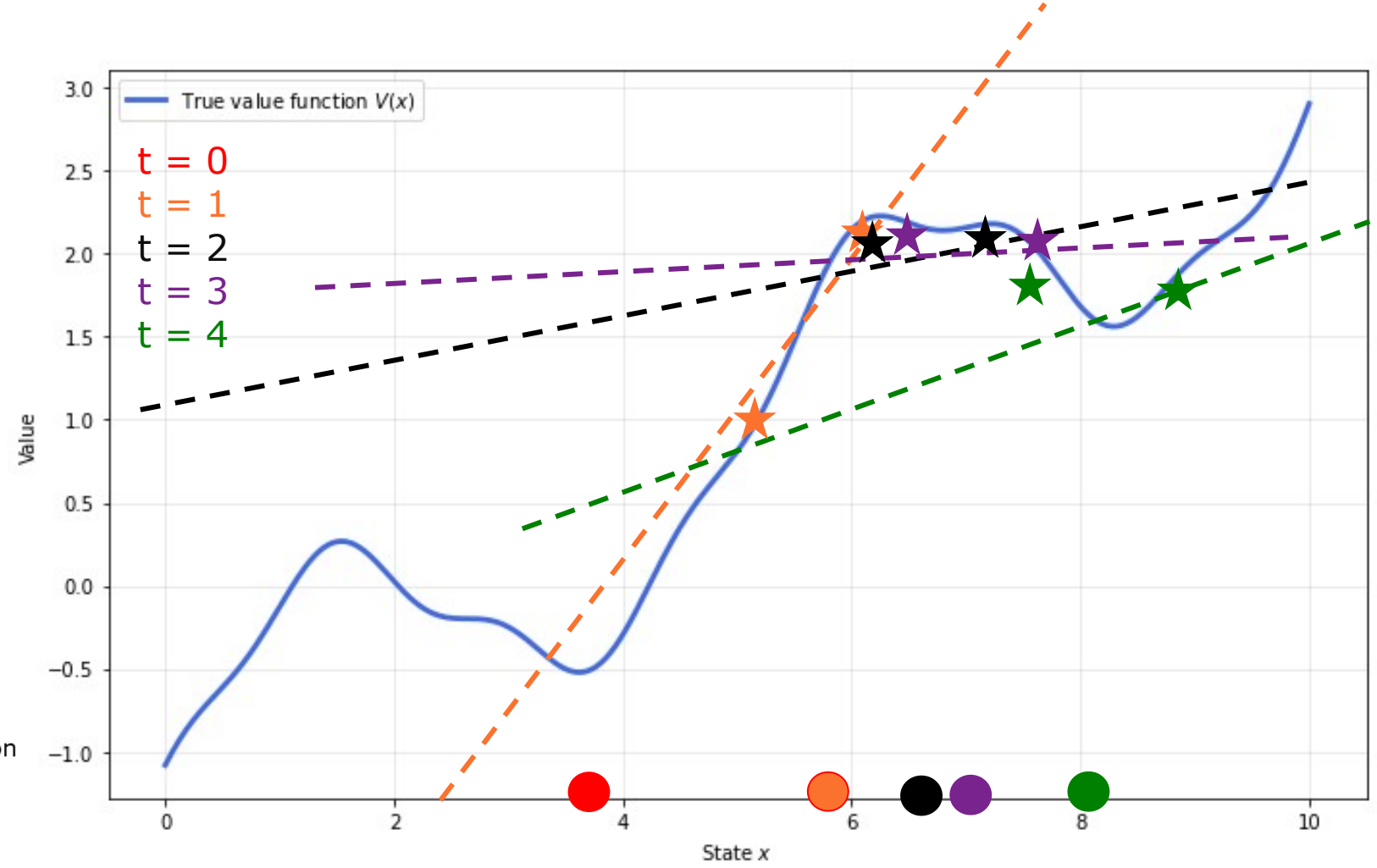
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

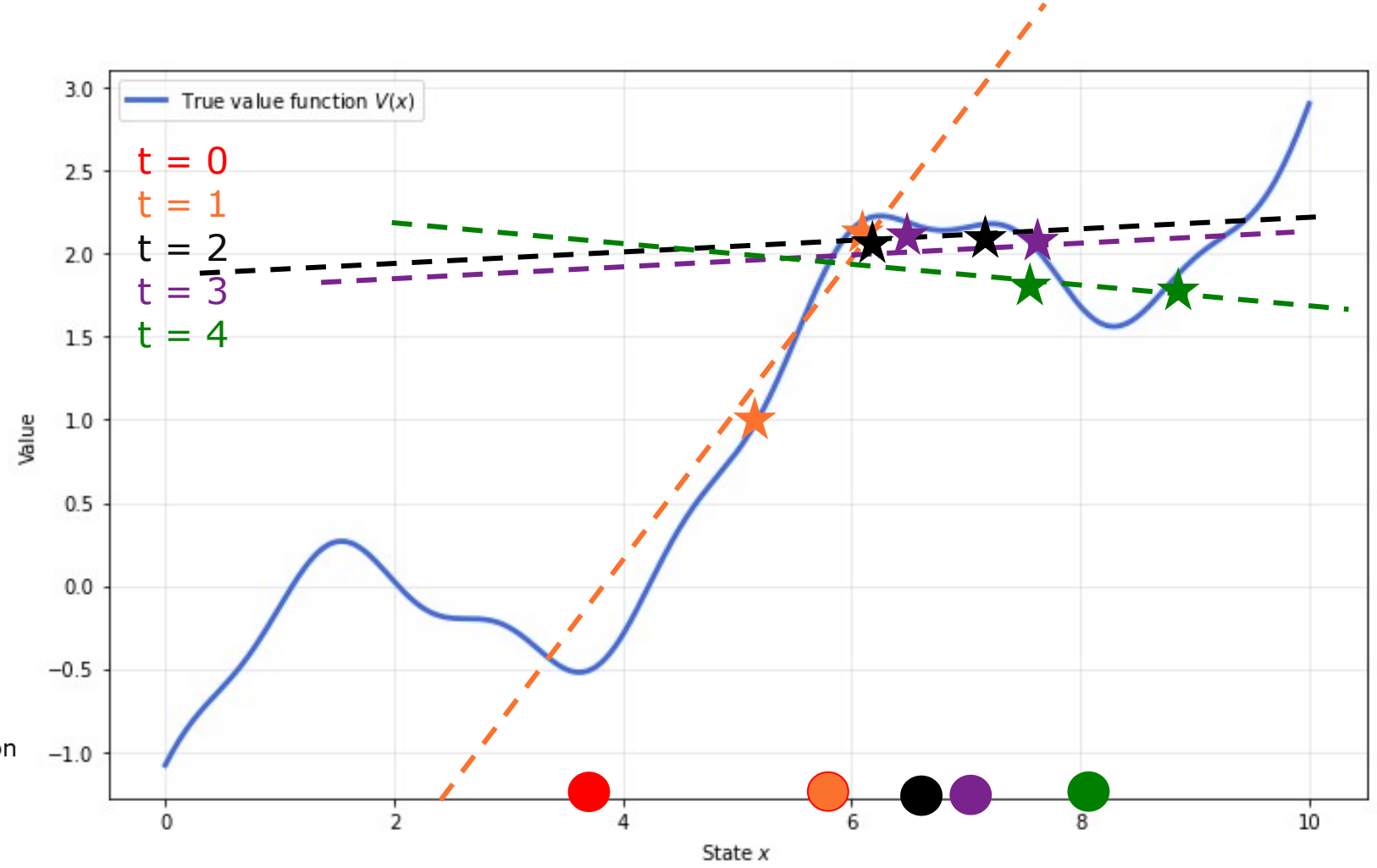
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

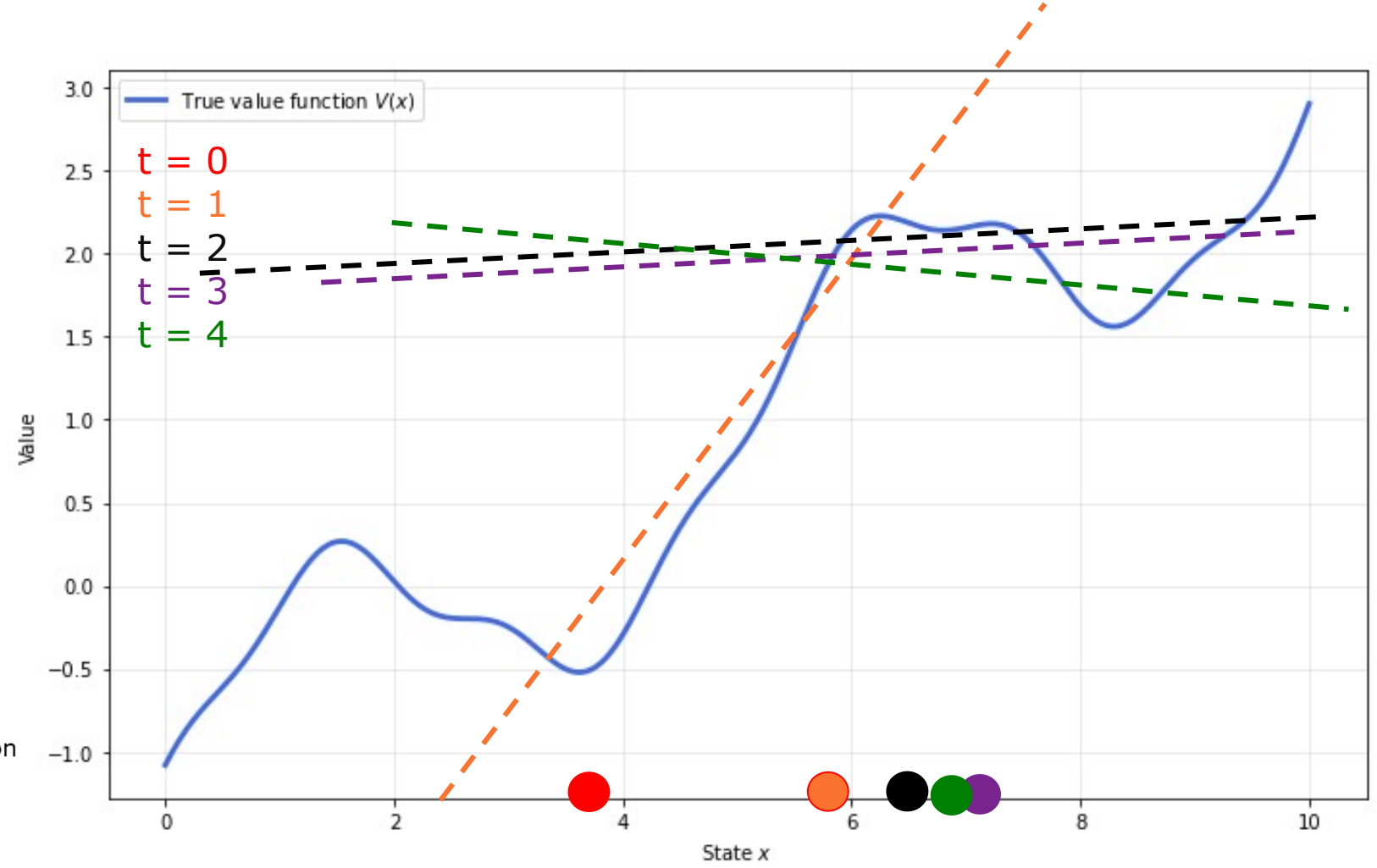
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

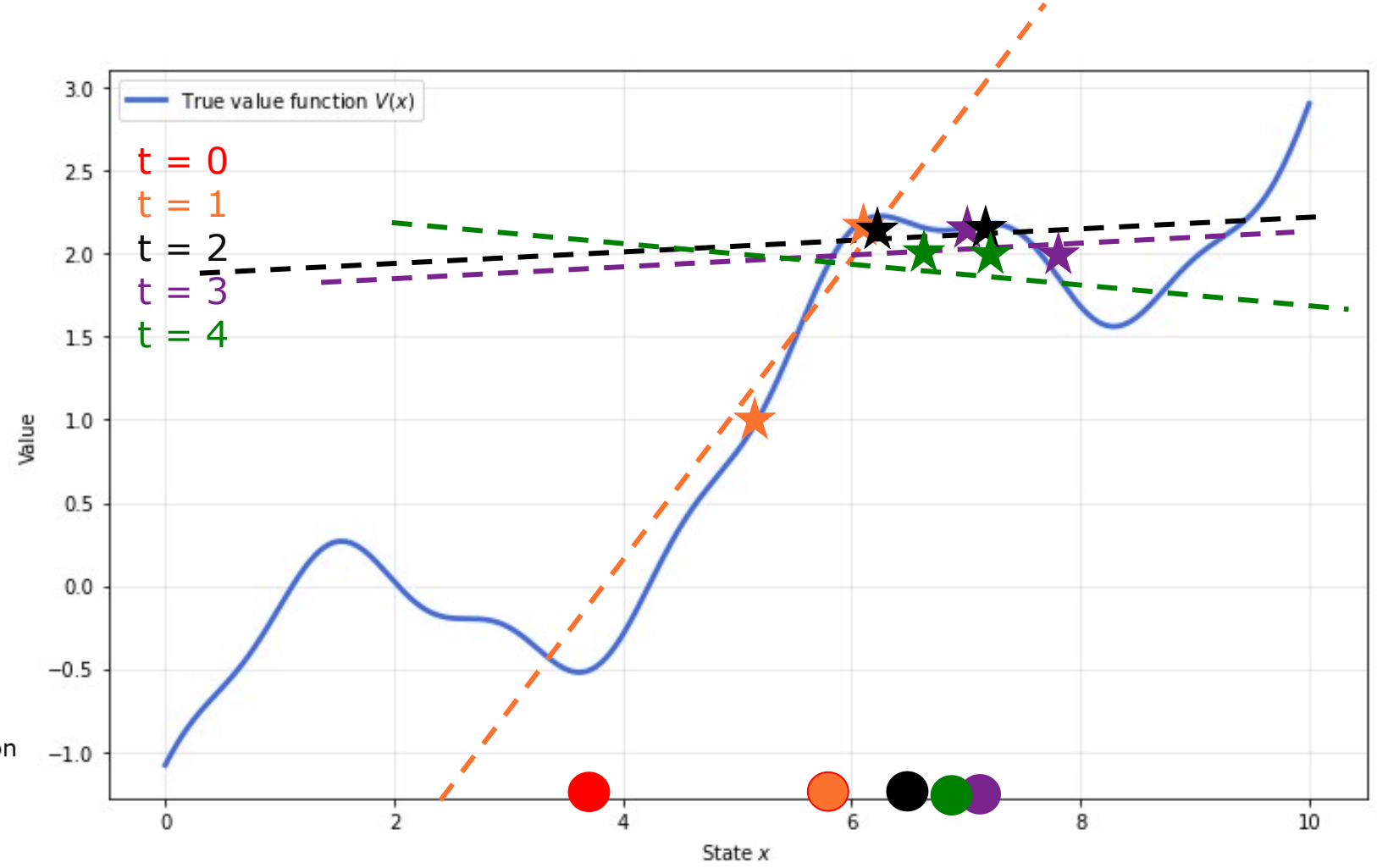
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



A Forward-Backward Algorithm

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

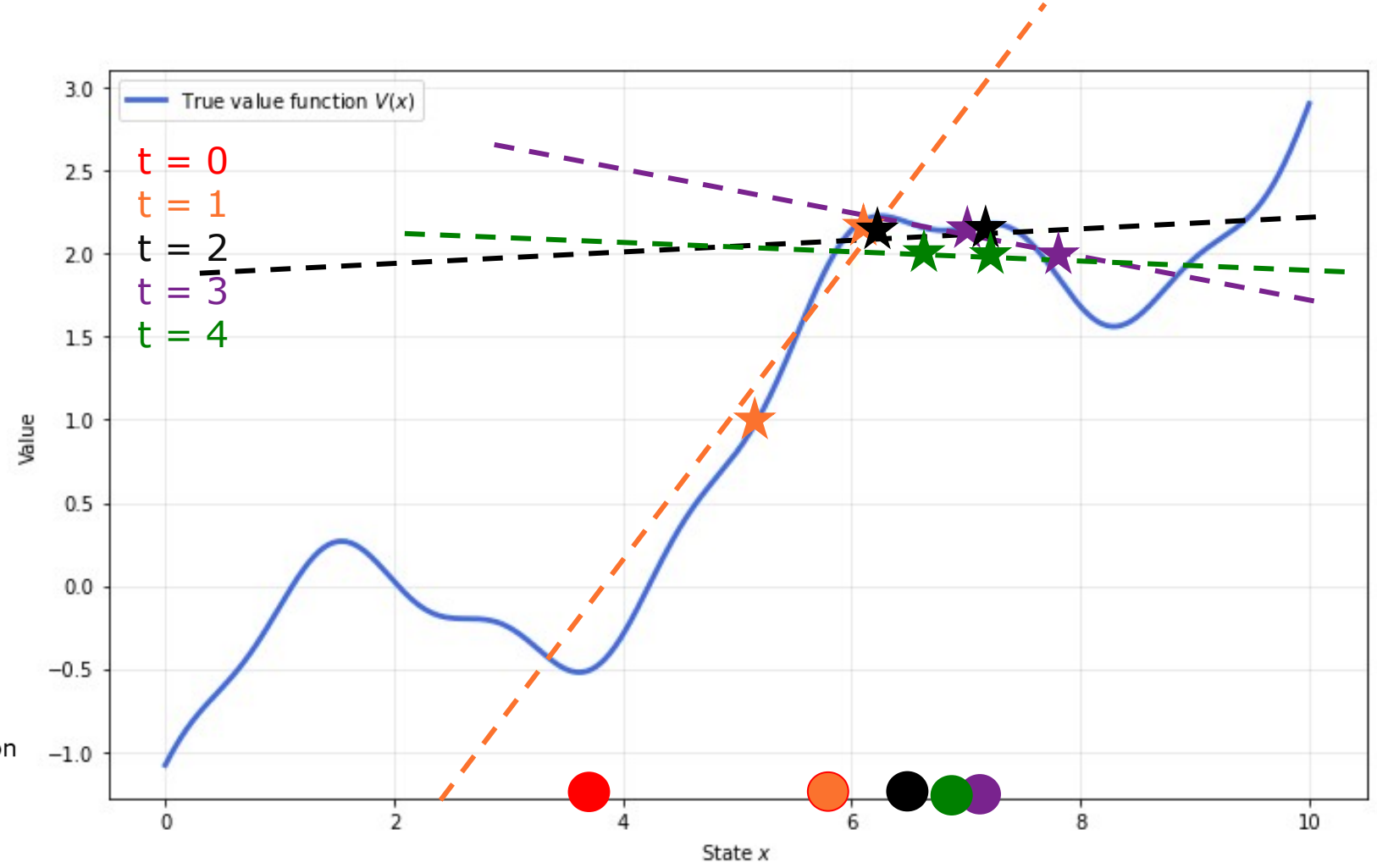
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



A Forward-Backward Algorithm

The sampled states and the etas move less and less.
Also, they kind of lead the system to a high-value area.

For I iterations

Sample N initial states $\{x_{n,0}\}_n = \{(y_{n,0}, w_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $w_{n,k,t+1} \sim P(\cdot \mid w_{n,t})$

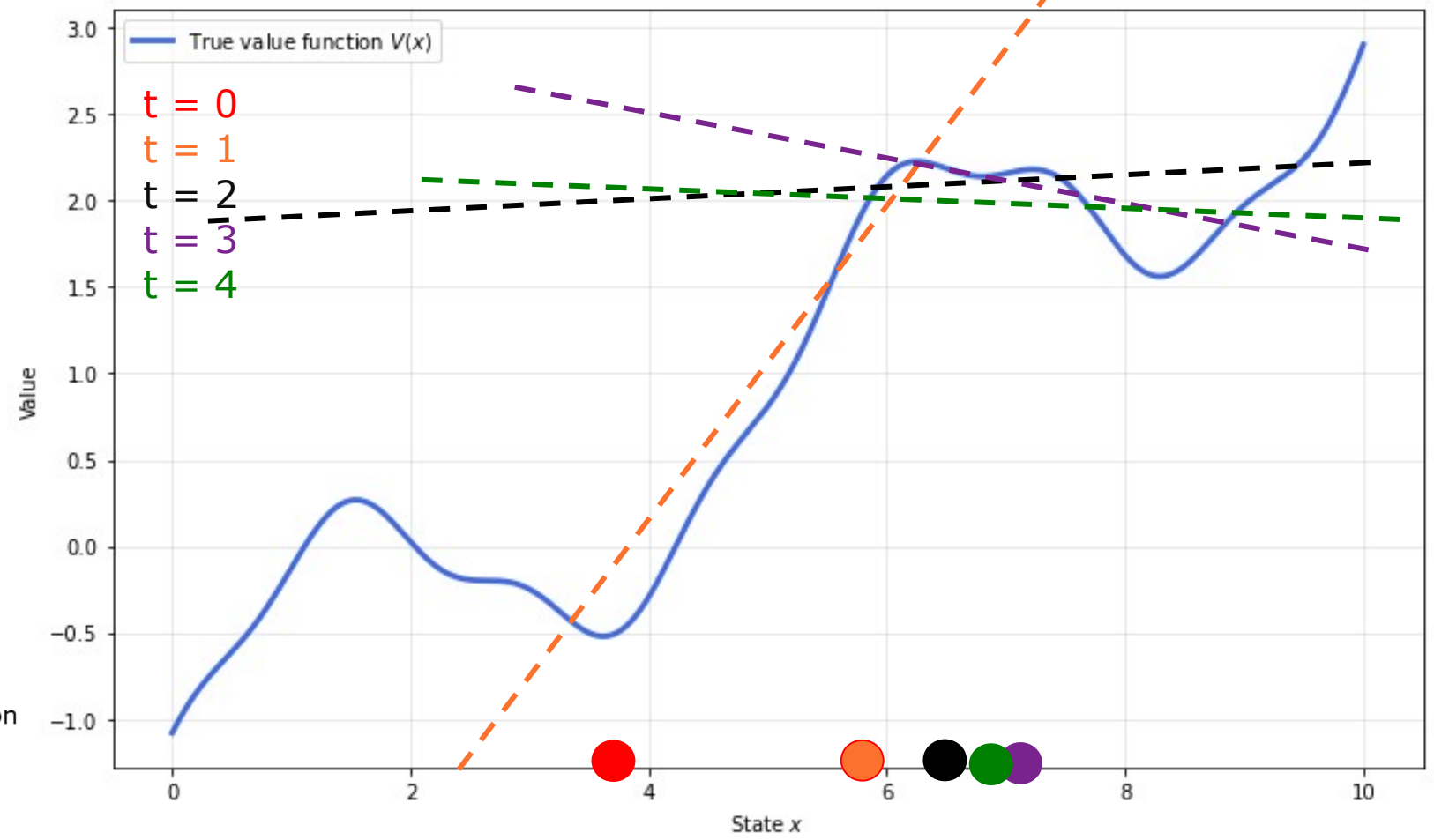
Create a $y_{n,t+1}$ by solving Bellman

Compute targets $V^*(x_{n,t})$

and

Update η_t

using Fitted Value Iteration or Approx. Backward Induction



Variant B:
Approximate Policy Iteration

Variant B: Approximate Policy Iteration

Initialize $\boldsymbol{\eta}_t$

For I iterations

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,t+1} = f(\boldsymbol{y}_{n,t}, \sum_k p_k \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Forward pass

For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{u}_{n,t})$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

Backward pass

We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$…

…and learn $\boldsymbol{\eta}_t$.

Step 1: Value Function Approximation

In the previous variant, we change the policy (by updating η_t) within the backward pass, i.e. "on the fly".

Variant B: Approximate Policy Iteration

Initialize $\boldsymbol{\eta}_t$

For $i \in I$ (iterations)

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,t+1} = f(\boldsymbol{y}_{n,t}, \sum_k p_k \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Forward pass

For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

Compute target $V^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}^i_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{u}_{n,t})$

~~Update $\boldsymbol{\eta}_t$ as:~~

~~$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^T \boldsymbol{x}_{n,t} - V^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$~~

Backward pass

We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$…

…and learn $\boldsymbol{\eta}_t$.

Step 1: Value Function Approximation

In the previous variant, we change the policy (by updating η_t) within the backward pass, i.e. "on the fly".

Alternatively, we can fix the policy:

$$\pi^i(x_t) = argmax \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}^i_{t+1}) \right\}$$

during the backward step, and update only the values (→ "policy evaluation")

Variant B: Approximate Policy Iteration

Initialize $\boldsymbol{\eta}_t$

For $i \in I$ (iterations)

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}\left(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}^{i,j=1}\right) \right\}$$

subject to: $\boldsymbol{y}_{n,t+1} = f\left(\boldsymbol{y}_{n,t}, \sum_k p_k \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t}\right)$

Forward pass

For $j \in J$ (evaluation sweeps)

For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}\left(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \tilde{\boldsymbol{\eta}}_{t+1}^{i,j}\right)$$

Update $\tilde{\boldsymbol{\eta}}_t^{i,j}$ as: (I already know $u_{n,t}$ from the forward pass)

$$\tilde{\boldsymbol{\eta}}_t^{i,j} = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\} \ \forall t$$

Backward pass

We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)\ldots$
…and learn $\boldsymbol{\eta}_t$.

Step 1: Value Function Approximation

In the previous variant, we change the policy (by updating η_t) within the backward pass, i.e. "on the fly".

Alternatively, we can fix the policy:

$$\pi^i(x_t) = argmax \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}^i) \right\}$$

during the backward step, and update only the values (→ "policy evaluation")

Variant B: Approximate Policy Iteration

Initialize $\boldsymbol{\eta}_t$

For $i \in I$ (iterations)

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}\left(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \tilde{\boldsymbol{\eta}}_{t+1}^{i,j}\right)$$

Forward pass

For $j \in J$ (evaluation sweeps)

For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}\left(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \tilde{\boldsymbol{\eta}}_{t+1}^{i,j}\right)$$

Update $\tilde{\boldsymbol{\eta}}_t^{i,j}$ as:

$$\tilde{\boldsymbol{\eta}}_t^{i,j} = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\} \ \forall t$$

Policy Evaluation (with fixed actions)

We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)\ldots$
…and learn $\boldsymbol{\eta}_t$.

Step 1: Value Function Approximation

In the previous variant, we change the policy (by updating η_t) within the backward pass, i.e. "on the fly".

Alternatively, we can fix the policy:

$$\pi^i(x_t) = argmax\left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}^i) \right\}$$

during the backward step, and update only the values (→ "policy evaluation")

And only after evaluating the policy thoroughly… I improve it

Variant B: Approximate Policy Iteration

Initialize $\boldsymbol{\eta}_t$

For $i \in I$ (iterations)

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}\left(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \tilde{\boldsymbol{\eta}}_{t+1}^{i,j}\right)$$

Forward pass

For $j \in J$ (evaluation sweeps)

For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}\left(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \tilde{\boldsymbol{\eta}}_{t+1}^{i,j}\right)$$

Update $\tilde{\boldsymbol{\eta}}_t^{i,j}$ as:

$$\tilde{\boldsymbol{\eta}}_t^{i,j} = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\} \ \forall t$$

Policy Evaluation (with fixed actions)

Set $\boldsymbol{\eta}_t^{i+1} = \tilde{\boldsymbol{\eta}}_t^{i,j} \ \forall t$

Policy Improvement

We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)\ldots$
…and learn $\boldsymbol{\eta}_t$.

Step 1: Value Function Approximation

In the previous variant, we change the policy (by updating η_t) within the backward pass, i.e. "on the fly".

Alternatively, we can fix the policy:

$$\pi^i(x_t) = argmax\left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}^i) \right\}$$

during the backward step, and update only the values (→ "policy evaluation")

And only after evaluating the policy thoroughly… I improve it

Piecewise linear Value Function Approximation

Variant C: Piece-wise Linear Approximate Value Function

Initialize $\boldsymbol{\eta}_t$

For I iterations

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,t+1} = f(\boldsymbol{y}_{n,t}, \sum_k p_k \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Forward pass

For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t}) \; \forall k$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

Backward pass

We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$

…and learn $\boldsymbol{\eta}_t$.

Step 1: Value Function Approximation

What if an endogenous state variable $z_t \in \boldsymbol{x}_t$ is binary?

Variant C: Piece-wise Linear Approximate Value Function

Initialize $\boldsymbol{\eta}_t$

For I iterations

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,t+1} = f(\boldsymbol{y}_{n,t}, \sum_k p_k \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Forward pass

We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$

…and learn $\boldsymbol{\eta}_t$.

Step 1: Value Function Approximation

For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

Compute target $V^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

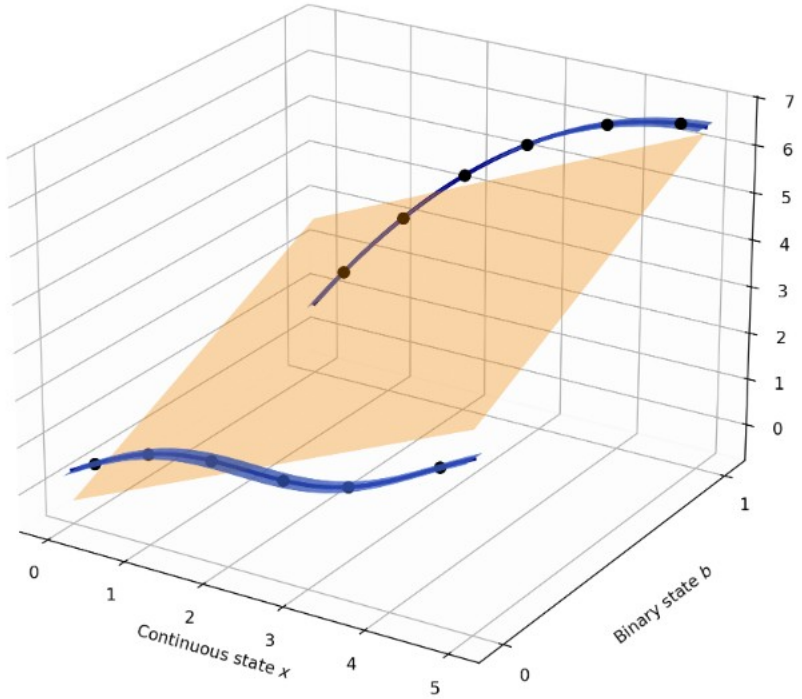
subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t}) \; \forall k$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - V^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

Backward pass

What if an endogenous state variable $z_t \in \boldsymbol{x}_t$ is binary?



Variant C: Piece-wise Linear Approximate Value Function

Initialize $\boldsymbol{\eta}_t$

For I iterations

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,t+1} = f\left(\boldsymbol{y}_{n,t}, \textstyle\sum_k p_k \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t}\right)$$

Forward pass

For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t}) \; \forall k$$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \textstyle\sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

Backward pass

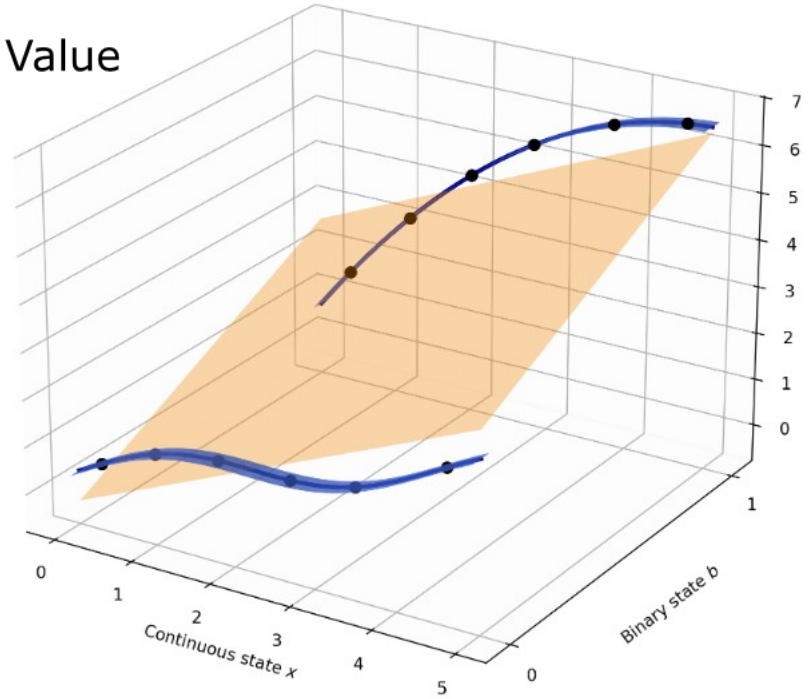
We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$
…and learn $\boldsymbol{\eta}_t$.

Step 1: Value Function Approximation

What if an endogenous state variable $z_t \in \boldsymbol{x}_t$ is binary?

For each t, we can fit two Value Functions; one for $z_t = 1$ and one for $z_t = 0$.



Variant C: Piece-wise Linear Approximate Value Function

Initialize $\boldsymbol{\eta}_t$

For I iterations

Sample N initial states $\{(\boldsymbol{y}_{n,0}, \boldsymbol{z}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}, z_{n,t+1}$ by solving:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,t+1}, z_{n,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, z_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \left(z_{n,t+1} \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}^1_{t+1}) + (1 - z_{n,t+1}) \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}^2_{t+1}) \right) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,t+1} = f\left(\boldsymbol{y}_{n,t}, \textstyle\sum_k p_k \boldsymbol{w}_{n,k,t+1}, z_{n,t}, \boldsymbol{u}_{n,t}\right)$$

$$z_{n,t+1} = g\left(\boldsymbol{y}_{n,t}, \sum_k p_k \boldsymbol{w}_{n,k,t+1}, z_{n,t}, \boldsymbol{u}_{n,t}\right)$$

For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

For each case $c \in \{1,2\}$: i.e. $c = 1$ if $z^c_{n,t} = 0$ and $c = 2$ if $z^c_{n,t} = 1$

Compute target $\mathrm{V}^c(\boldsymbol{x}^c_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}, z_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, z^c_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \left(z_{n,k,t+1} \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}^1_{t+1}) + (1 - z_{n,k,t+1}) \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}^2_{t+1}) \right) \right\}$$

$$\text{subject to: } \boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, z^c_{n,t}, \boldsymbol{u}_{n,t}), \forall k$$

$$z_{n,k,t+1} = g(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, z^c_{n,t}, \boldsymbol{u}_{n,t}), \forall k$$

For each case $c \in \{1,2\}$:

Update $\boldsymbol{\eta}^c_t$ as:

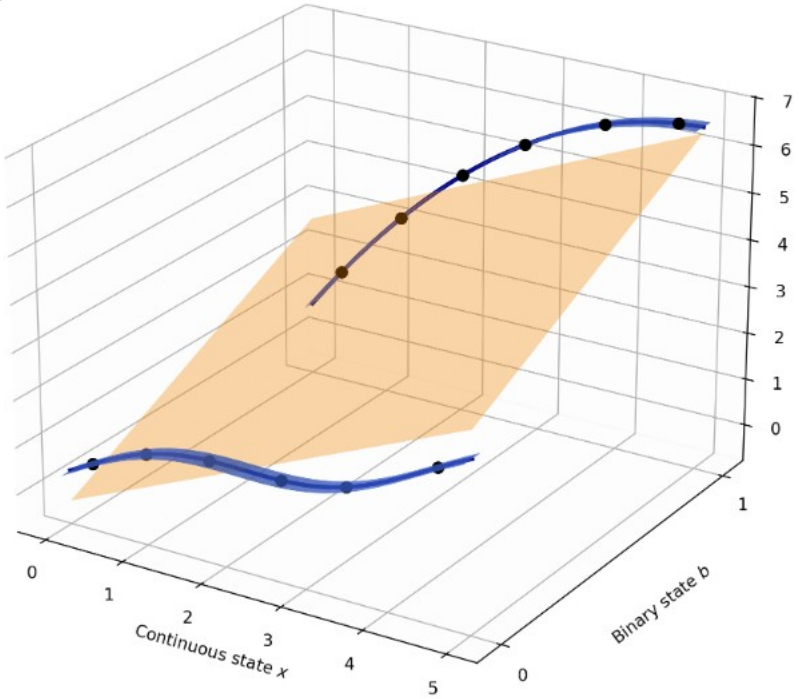
$$\boldsymbol{\eta}^c_t = \arg\min \left\{ \textstyle\sum_n \left((\boldsymbol{\eta}^c_t)^\top \boldsymbol{x}^c_{n,t} - \mathrm{V}^c(\boldsymbol{x}^c_{n,t}) \right)^2 \right\}$$

We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t)$

…and learn $\boldsymbol{\eta}_t$.

Step 1: Value Function Approximation



Non-linear Value Function Approximation

Variant D: Non-linear Approximate Value Function

Initialize $\boldsymbol{\eta}_t$

For I iterations

Sample N initial states $\{\boldsymbol{x}_{n,0}\}_n = \{(\boldsymbol{y}_{n,0}, \boldsymbol{w}_{n,0})\}_n$

For each $t \in \{0, 1, 2, 3, \ldots\}$:

For each n:

Create K samples $\boldsymbol{w}_{n,k,t+1} \sim P(\cdot \mid \boldsymbol{w}_{n,t})$

Create a $\boldsymbol{y}_{n,t+1}$ by solving:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,t+1} = f(\boldsymbol{y}_{n,t}, \sum_k p_k \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t})$

Forward pass

For each $t \in \{T, T-1, T-2, \ldots\}$:

For each n:

Compute target $\mathrm{V}^*(\boldsymbol{x}_{n,t})$ as:

$$\max_{\boldsymbol{u}_{n,t}, \boldsymbol{y}_{n,k,t+1}} \left\{ r(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,t}, \boldsymbol{u}_{n,t}) + \sum_k p_k \hat{V}(\boldsymbol{y}_{n,k,t+1}, \boldsymbol{w}_{n,k,t+1}; \boldsymbol{\eta}_{t+1}) \right\}$$

subject to: $\boldsymbol{y}_{n,k,t+1} = f(\boldsymbol{y}_{n,t}, \boldsymbol{w}_{n,k,t+1}, \boldsymbol{u}_{n,t}), \forall k$

Update $\boldsymbol{\eta}_t$ as:

$$\boldsymbol{\eta}_t = \arg\min \left\{ \sum_n \left((\boldsymbol{\eta}_t)^\top \boldsymbol{x}_{n,t} - \mathrm{V}^*(\boldsymbol{x}_{n,t}) \right)^2 \right\}$$

Backward pass

We approximate the Value Function:

Parameterize it as $\hat{V}(\boldsymbol{x}_t; \boldsymbol{\eta}_t) = \boldsymbol{\eta}_t^\top \varphi(\boldsymbol{x}_t)$

…and learn $\boldsymbol{\eta}_t$.

Step 1: Value Function Approximation

$\varphi(\boldsymbol{x}_t)$: "features" or "basis functions"

What functional form should we choose?

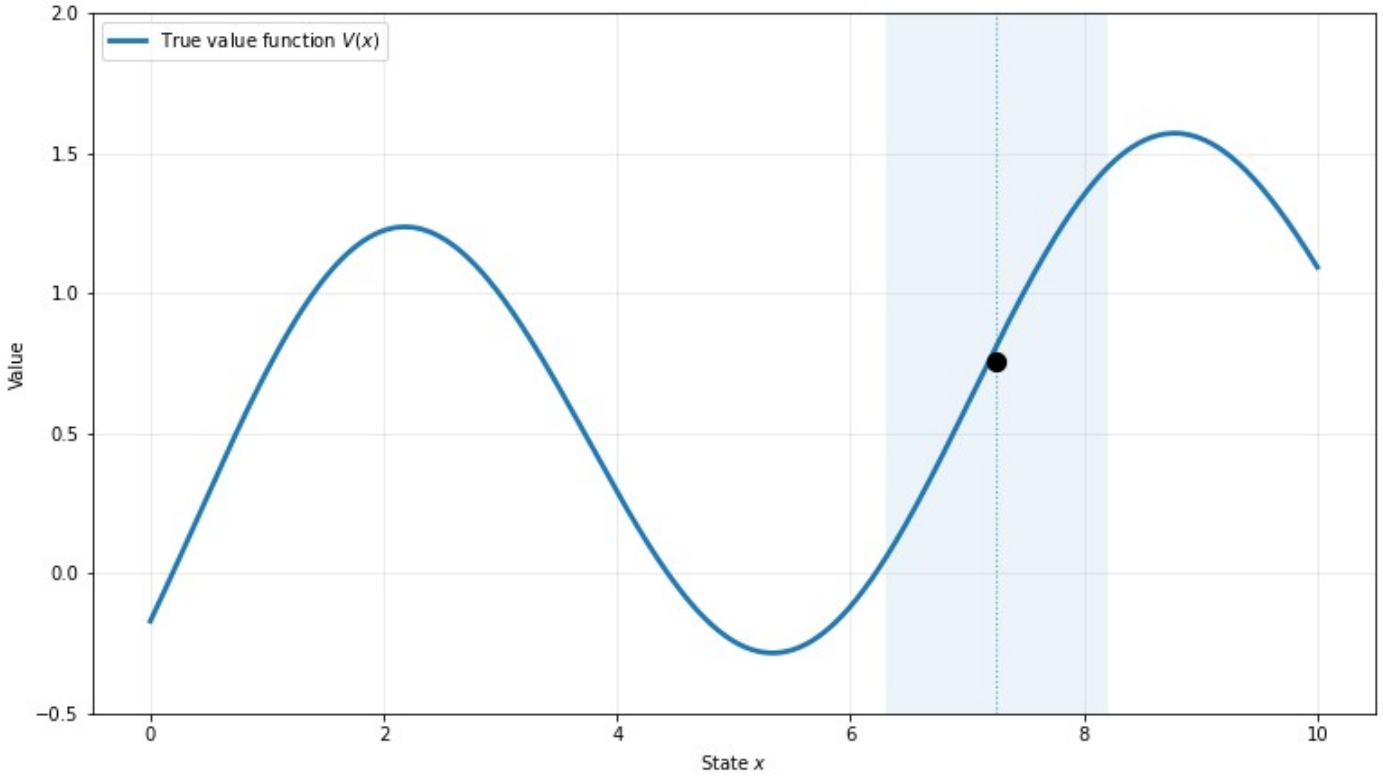
Linear / non-linear in $\boldsymbol{y}_t / \boldsymbol{w}_t / \boldsymbol{\eta}_t$?

~ Non-linearity in $\boldsymbol{w}_t$, is not a problem.

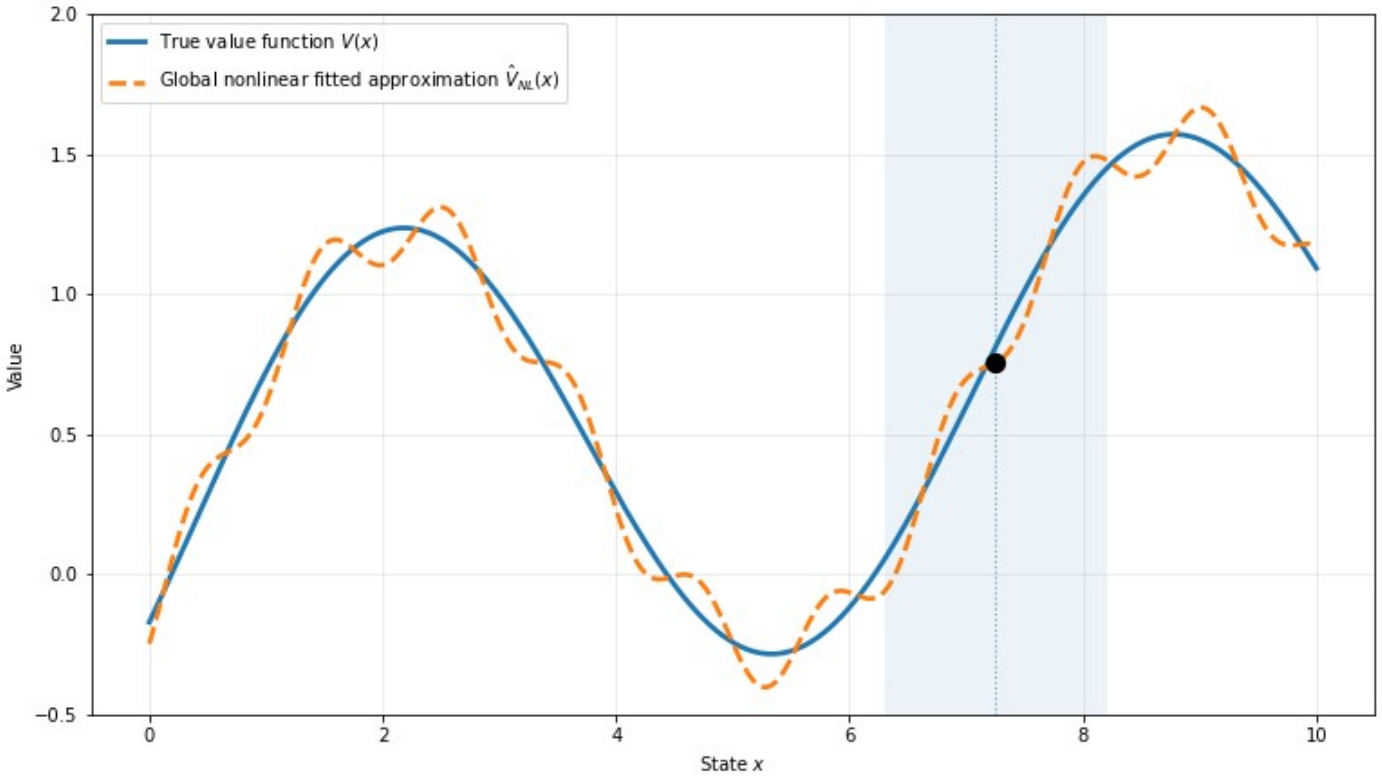
~ Non-linearity in $\boldsymbol{\eta}_t$, creates a challenge in the fitting; e.g. neural networks – not in scope.

~ Non-linearity in $\boldsymbol{y}_t$, creates a challenge in the Bellman update; but certain forms can be handled well.

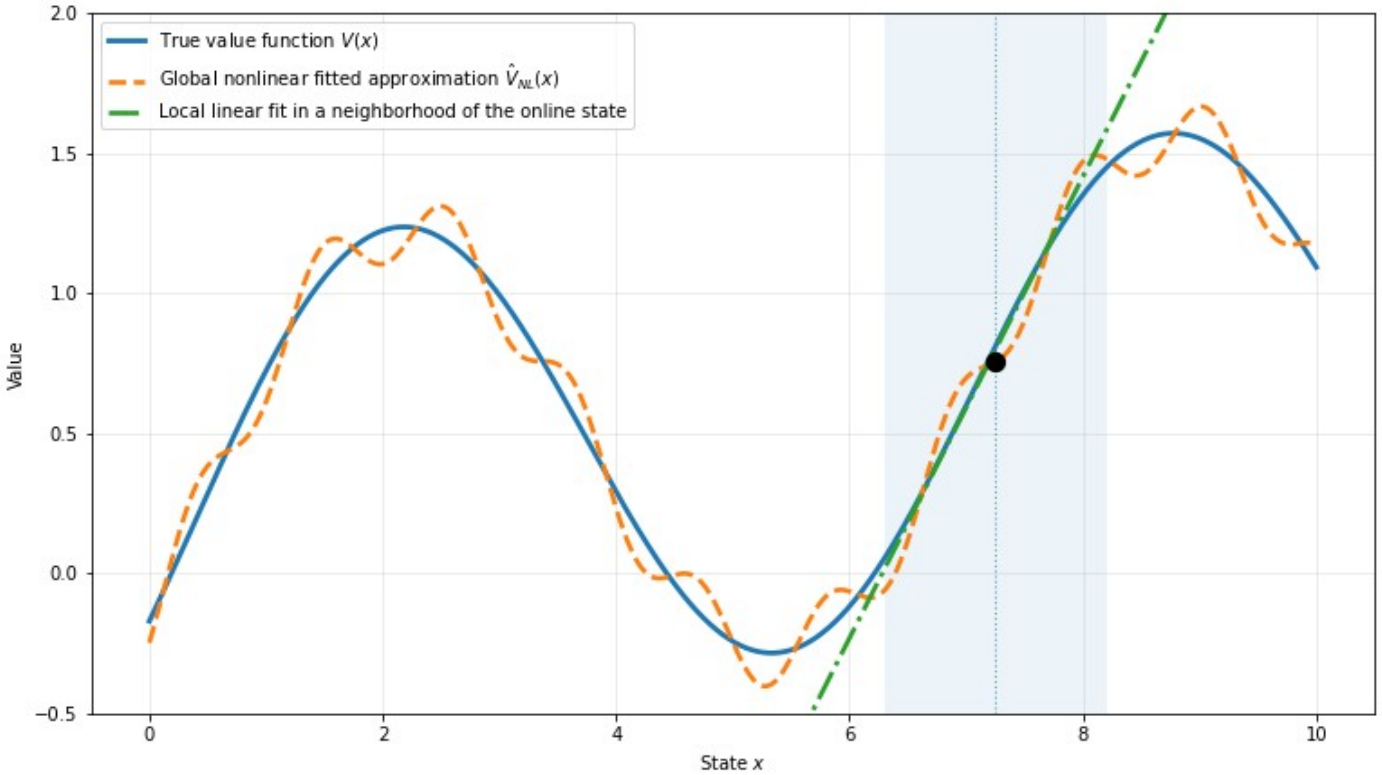
Extension A: Online Value Function Approximation



Extension A: Online Value Function Approximation



Extension A: Online Value Function Approximation



Extension B: Combine VFA with multi-stage SP

$$min\left\{\sum_{\omega\in\Omega} \mathrm{p}_\omega \sum_{t\in L} c\left(\boldsymbol{u}_{\omega,t}, \boldsymbol{x}_{\omega,t}\right)\right\}$$

subject to:

$$\boldsymbol{u}_{\omega,t} \in U\left(\boldsymbol{x}_{\omega,t}\right), \forall\omega \in \Omega, t \in L$$

$$\boldsymbol{y}_{\omega,t+1} = f\left(\boldsymbol{u}_{\omega,t}, \boldsymbol{y}_{\omega,t}, \boldsymbol{w}_{\omega,t+1}\right), \forall\omega \in \Omega, t \in L$$

$$\boldsymbol{y}_{\omega,t} = \boldsymbol{y}^{init}, \forall\omega \in \Omega, t = \tau$$

$$\boldsymbol{u}_{\omega,t} = \boldsymbol{u}_{\omega',t}, \forall\omega \in \Omega, \omega' \in \Omega_{\omega,t}, t \in L$$

Extension B: Combine VFA with multi-stage SP

$$min\left\{\sum_{\omega\in\Omega} \mathrm{p}_\omega \left(\sum_{t\in L} c\left(\boldsymbol{u}_{\omega,t}, \boldsymbol{x}_{\omega,t}\right) + V\left(x_{\omega,\mathrm{L}+1}\right)\right)\right\}$$

subject to:

$$\boldsymbol{u}_{\omega,t} \in U\left(\boldsymbol{x}_{\omega,t}\right), \forall\omega \in \Omega, t \in L$$

$$\boldsymbol{y}_{\omega,t+1} = f\left(\boldsymbol{u}_{\omega,t}, \boldsymbol{y}_{\omega,t}, \boldsymbol{w}_{\omega,t+1}\right), \forall\omega \in \Omega, t \in L$$

$$\boldsymbol{y}_{\omega,t} = \boldsymbol{y}^{init}, \forall\omega \in \Omega, t = \tau$$

$$\boldsymbol{u}_{\omega,t} = \boldsymbol{u}_{\omega',t}, \forall\omega \in \Omega, \omega' \in \Omega_{\omega,t}, t \in L$$

Questions